

Exploring the Architectural Design Space for Next-Generation Language Models: A Myrmidon-Swarm Review of 35 Candidate Mechanisms Against the Qwen3 and Qwen3.5 Baselines

Micah Villmow
`research@villmow.us` *

April 2026

Abstract

The rapid expansion of large language model (LLM) architectures has produced a wide design space of candidate mechanisms for improving efficiency, expressivity, and inference throughput. We present a systematic Myrmidon-Swarm review of 35 architectural candidates evaluated against three representative Qwen3/Qwen3.5 baselines: the 27B hybrid (Qwen3.5-27B, 16/64 full-attention layers interleaved with Gated DeltaNet), the 32B dense model (Qwen3-32B), and the 397B MoE hybrid (Qwen3.5-397B-A17B, 17B active parameters). Each candidate is reviewed across five dimensions: citation accuracy, computational complexity relative to baselines, literature completeness, comparative claim validity, and engineering feasibility.

Of the 31 originally reviewed ideas (groups 1.x through 5.x), 5 received high-priority or equivalent endorsement, 21 received promising further review, and 5 were assigned lower-priority due to prior-art saturation or unfavorable cost-benefit ratios. We additionally introduce four novel architectural candidates — an in-architecture autoregressive loop with learned stopping (6.1), a prefill/decode architectural split (6.2), a block-diffusion autoregressive decoder (6.3), and their combination (6.4) — and provide full Myrmidon-Swarm reviews of each. The highest-confidence immediately deployable ideas include INT4 block quantization (5.7), per-layer adaptive expert count (1.3), compressed dense layers via post-hoc SVD (2.2), and LSTM-gated attention for the Qwen3-32B KV cache (5.2).

All research documents, verification reports, and corrected synthesis artifacts are released alongside this paper.

1 Introduction

Modern large language model architectures have diversified far beyond the original Transformer [311]. The design space now encompasses dense attention, linear (recurrent) attention, mixture-of-experts (MoE) feed-forward layers, hybrid architectures interleaving multiple computational primitives,

*This paper was prepared with Claude Code using the Homeric Intelligence infrastructure.

and a growing toolkit of post-training compression techniques. Practitioners are confronted with dozens of published mechanisms promising improvements in throughput, memory efficiency, or model quality — yet systematic, apples-to-apples comparisons against the same production-grade baselines are rare. Most published results anchor to heterogeneous baselines, different context lengths, and variable hardware conditions, making direct comparison effectively impossible without re-implementing every method.

This paper addresses that gap by fixing three concrete production baselines drawn from the Qwen3 and Qwen3.5 model families [274, 275] and subjecting 35 architectural candidates to a uniform five-dimension review methodology. The three baselines span the major architecture classes currently deployed at scale.

Baseline A1 (Qwen3.5-27B) – Hybrid Linear-Attention. This 27-billion-parameter model combines Gated DeltaNet linear attention [329] on 48 of 64 layers with full softmax attention on the remaining 16, yielding a model with bounded KV cache growth and competitive long-context throughput. It is representative of the emerging class of hybrid recurrent-attentive architectures, exemplified also by Jamba [225], Mamba [189], Nemotron-H [263], and Griffin [36].

Baseline A2 (Qwen3-32B) – Dense Transformer. This 32-billion-parameter all-attention model uses 64-head grouped-query attention (GQA) with 8 KV heads and a large 25,600-dimensional MLP intermediate, producing a model whose inference is dominated by weight memory bandwidth. At 32K context the KV cache is ≈ 8.59

Baseline B (Qwen3.5-397B-A17B) – Hybrid Sparse MoE. This 397-billion-parameter model activates only ≈ 17 B parameters per token via fine-grained MoE routing (512 experts, $k=11$ active) combined with Gated DeltaNet for 45 of 60 layers. Its small KV cache (≈ 1.0 GB at 32K context) makes it the most bandwidth-efficient of the three baselines, and the most challenging target for any mechanism whose value proposition rests on KV cache reduction.

The Myrmidon-Swarm Methodology

Each candidate idea is reviewed by an independent lead agent coordinating five specialist sub-agents operating in parallel:

1. **Citation Verifier** — checks whether every citation accurately reflects the method, result, and context of the referenced paper; flags fabricated or misattributed results.
2. **Complexity Auditor** — verifies all Big-O complexity claims, FLOPs derivations, KV cache byte formulas, and quantitative throughput estimates against the canonical baseline specifications.
3. **Literature Gap Finder** — assesses the degree to which each idea overlaps with existing published work, classifying ideas as EXISTS ($\geq 85\%$ covered), PARTIAL (30–84%), or NOVEL ($< 30\%$ covered in any single prior work).
4. **Comparison Validator** — checks whether improvement claims are made relative to an appropriate and correctly specified baseline.

5. **Feasibility Checker** — assesses implementation complexity, required custom kernel development, training cost multipliers, and realistic engineering timeline.

The lead agent synthesizes the five specialist reports into a unified verdict (high-priority, candidate for investigation, or lower-priority) with a confidence score, a list of corrected claims, and a prior-art gap summary. This methodology produces 165 sub-agent verification reports alongside the 35 lead-agent summaries.

Contributions

This paper makes four primary contributions:

1. **Systematic review of 31 candidate architectural mechanisms** against three fixed Qwen3/Qwen3.5 baselines, with corrected complexity claims, citation verification, and uniformly formatted verdict summaries (Section 3).
2. **Four new architectural proposals with full Myrmidon-Swarm reviews:** in-architecture autoregressive loop (6.1), prefill/decode architectural split (6.2), block-diffusion autoregressive decoder (6.3), and their multiplicative combination (6.4), collectively offering up to a 4× decode FLOP reduction (Sections 3 and detailed analyses in the companion paper).
3. **A reproducible multi-agent review methodology** (the Myrmidon-Swarm pattern), including 165 specialist sub-agent verification reports released with the paper, enabling other researchers to audit or extend the reviews independently.

Paper Organization

Section 2 describes the three baseline models with precise specifications and KV cache calculations. Section 3 presents the catalog of all 35 candidate ideas organized into six thematic groups, with short descriptions and one-line verdicts. Section 6 consolidates implementation verdicts and priority recommendations.

2 Models and Baselines

All three baselines are drawn from the Qwen3 and Qwen3.5 model families released by Alibaba [274, 275]. Specifications are sourced from authoritative `config.json` files on Hugging Face and NVIDIA NIM model cards; they override any prior prelude document. All KV cache byte computations use the canonical formula:

$$\text{KV_bytes} = L_{\text{full-attn}} \times 2 \times H_{kv} \times d_{\text{head}} \times s \times 2 \quad (1)$$

where $L_{\text{full-attn}}$ is the number of full-attention layers (never the total layer count for hybrid models), H_{kv} is `num_key_value_heads` (never `num_attention_heads`), d_{head} is the per-head key/value dimension, s is the sequence length, and the final factor of 2 encodes bf16 (2 bytes per element).

2.1 Qwen3.5-27B — Baseline A1 (Hybrid Multimodal)

Qwen3.5-27B is a 27-billion-parameter hybrid model combining Gated DeltaNet linear attention with full softmax attention in a 3:1 ratio. Every fourth layer uses full causal self-attention; the remaining three quarters use Gated DeltaNet, which maintains a fixed-size recurrent state independent of sequence length [329]. The model supports multimodal inputs via a vision tower and uses multi-token prediction (MTP) heads. Its design is representative of the hybrid recurrent-attentive class that achieves competitive long-context throughput with bounded KV memory growth.

Table 1: *Qwen3.5-27B (Baseline A1) architectural specifications.*

Parameter	Value
Total layers (L)	64
Hidden dimension (d)	5,120
MLP intermediate (d_{ff})	17,408
Full-attention layers	16 of 64 (every 4th layer)
Q-heads / KV-heads	24 / 4; $d_{\text{head}} = 256$
Gated DeltaNet layers	48 of 64
V-heads / QK-heads	48 / 16; $d_{\text{head}} = 128$
Vocabulary size	248,320
Native context length	262,144 tokens
RoPE base	10,000,000; mRoPE [11,11,10]
Activation	SiLU
Normalization	RMSNorm
Data type	bf16
Total parameters	$\approx 27\text{B}$ (all active)

KV cache analysis. Only the 16 full-attention layers maintain a KV cache; Gated DeltaNet layers carry a fixed-size recurrent state (≈ 13 MB) independent of sequence length. Applying Equation 1:

$$\text{KV} = 16 \times 2 \times 4 \times 256 \times s \times 2 = 65,536 \cdot s \text{ bytes}$$

At $s = 32,768$ tokens: ≈ 2.15 GB. At $s = 262,144$ tokens: ≈ 17.2 GB.

MLP FLOPs per token per layer: $2 \times 5,120 \times 17,408 \approx 1.78 \times 10^8$ FLOPs.

2.2 Qwen3-32B — Baseline A2 (Dense Transformer)

Qwen3-32B is a 32-billion-parameter fully dense transformer. All 64 layers use full causal self-attention with grouped query attention (GQA), and all layers contribute to the KV cache. At 32K context the KV cache is ≈ 8.59 GB, which is dominated at batch size 1 by weight memory bandwidth (≈ 64 GB in bf16). This baseline is the primary target for weight compression and KV compression mechanisms.

Table 2: *Qwen3-32B (Baseline A2) architectural specifications.*

Parameter	Value
Total layers (L)	64
Hidden dimension (d)	5,120
MLP intermediate (d_{ff})	25,600
Attention heads (Q / KV)	64 / 8; $d_{\text{head}} = 128$
Vocabulary size	151,936
Native context length	40,960 tokens
RoPE base	1,000,000
Activation	SiLU
Normalization	RMSNorm
Data type	bf16
Total parameters	$\approx 32\text{B}$ (all active)

KV cache analysis. All 64 layers are full-attention; $H_{kv} = 8$, $d_{\text{head}} = 128$:

$$\text{KV} = 64 \times 2 \times 8 \times 128 \times s \times 2 = 262,144 \cdot s \text{ bytes}$$

At $s = 32,768$ tokens: ≈ 8.59 GB. At $s = 40,960$ tokens (native max): ≈ 10.49 GB.

[†] This figure arose from substituting $H_q = 64$ (query heads) for $H_{kv} = 8$ (key-value heads) The correct value is ≈ 8.59 GB. See

MLP FLOPs per token per layer: $2 \times 5,120 \times 25,600 \approx 2.62 \times 10^8$ FLOPs.

2.3 Qwen3.5-397B-A17B — Baseline B (Hybrid Sparse MoE)

Qwen3.5-397B-A17B is the flagship mixture-of-experts model in the Qwen3.5 family, activating $\approx 17\text{B}$ parameters per token from a 397B total. Its architecture groups 60 layers into 15 groups of 4, each comprising 3 Gated DeltaNet layers followed by 1 Gated Attention layer. The MoE feed-forward module at each layer deploys 512 fine-grained experts with $k=11$ active per token (10 routed plus 1 shared). With only 15 of 60 layers carrying a KV cache, and those layers using only 2 KV heads at $d_{\text{head}} = 256$, this baseline achieves the smallest KV footprint of the three. NVIDIA reports $8.6\times\text{--}19\times$ decoding throughput over Qwen3-Max using this design.

KV cache analysis. Only the 15 Gated Attention layers maintain KV; $H_{kv} = 2$, $d_{\text{head}} = 256$:

$$\text{KV} = 15 \times 2 \times 2 \times 256 \times s \times 2 = 30,720 \cdot s \text{ bytes}$$

At $s = 32,768$ tokens: ≈ 1.0 GB. At $s = 262,144$ tokens: ≈ 8.0 GB.

Table 3: *Qwen3.5-397B-A17B (Baseline B) architectural specifications.*

Parameter	Value
Total layers (L)	60
Layer groups	$15 \times (3 \text{ DeltaNet} + 1 \text{ Gated Attention})$
Hidden dimension (d)	4,096
Gated Attention layers (full KV)	15 of 60; $H_q = 32$, $H_{kv} = 2$, $d_{\text{head}} = 256$
Gated DeltaNet layers	45 of 60; 64 V-heads / 16 QK-heads, $d_{\text{head}} = 128$
MoE experts (total / active)	512 / 11 (10 routed + 1 shared)
Vocabulary size	248,320
Native context length	262,144 tokens (YaRN extensible to $\approx 1,010,000$)
Active / total parameters	17B / 397B
Data type	bf16

2.4 K2 Family — Baseline C (Dense Llama-arch, 72.55B)

The K2 model family (LLM360 [243]) represents a large-scale open-weight dense transformer baseline. K2-V2 and K2-Think-V2 share an identical architecture: 80 transformer layers, hidden dimension $d = 8,192$, MLP intermediate size $d_{\text{ff}} = 28,672$, grouped-query attention with 64 Q-heads and 8 KV-heads (GQA ratio 8:1), head dimension 128, and vocabulary size 250,112. Total parameter count is approximately 72.55B, all active at inference (dense, no MoE routing). The models use RMSNorm, SiLU/SwiGLU activation, and rotary position embeddings with $\theta = 10^7$.

K2-V2 supports a native context window of 524,288 tokens via LLaMA-3-style RoPE scaling (factor $4\times$ over the 131,072-token base). K2-Think-V2 extends to 262,144 tokens via YaRN (factor $2\times$). **Architecturally the two variants are identical;** K2-Think-V2 adds reasoning-trace tokens (`<think>`, `<think_fast>`, `<think_faster>`) only through its chat template, not through any weight or structural change. All quantitative analyses in this paper treat them as a single baseline.

Per-token complexity. Attention: $O(s \cdot d)$ compute; KV memory = $2 \times 8 \times 128 \times s \times 2$ bytes = $4,096 \cdot s$ bytes per layer (bf16). MLP: $2 \times 8,192 \times 28,672 \approx 4.70 \times 10^8$ FLOPs per token per layer. Total KV cache (bf16, all 80 layers): $80 \times 2 \times 8 \times 128 \times s \times 2 = 327,680 \cdot s$ bytes.

Key reference values.

- KV cache @ 32K tokens: ≈ 10.0 GiB
- KV cache @ 262K tokens: ≈ 80.0 GiB
- KV cache @ 524K tokens: ≈ 160.0 GiB
- Weight bandwidth (bf16): ≈ 145.1 GB

2.5 KV Cache Comparison Across Baselines

Table 4 summarises the KV cache footprint across all four baselines at reference context lengths. The contrast is stark: Baseline B achieves a $8.6\times$ reduction in KV memory relative to Baseline A2 at 32K context, purely through architectural choices (hybrid linear attention + aggressively low KV head count).

Table 4: KV cache size (bf16) across baselines at representative context lengths. All values computed from Equation 1 using canonical H_{kv} and d_{head} values.

Baseline	Layers	32K tokens	262K tokens	524K tokens	Max context
A1: Qwen3.5-27B (Hybrid)	16	≈ 2.15 GB	≈ 17.2 GB	N/A	262K
A2: Qwen3-32B (Dense)	64	≈ 8.59 GB	N/A	N/A	40K
B: Qwen3.5-397B-A17B (MoE)	15	≈ 1.0 GB	≈ 8.0 GB	N/A	262K
C (K2, 72.55B)	80	10.0 GiB	80.0 GiB	160.0 GiB	524K (K2-V2)

3 Catalog of Candidate Architectural Ideas

We organize the 35 candidate ideas into six thematic groups. For each idea we provide a concise mechanism description (2–3 sentences) and a one-line verdict drawn from the lead-agent summary. Section ?? (companion document) provides full per-idea analysis tables.

Verdict codes used below: **high-priority** — mechanism is well-validated, deployable within weeks to months, recommended for immediate engineering; **candidate for investigation** — mechanism has genuine novelty and positive expected value but requires targeted prototyping before committing; **lower-priority** — mechanism is either thoroughly covered by prior art, has an unfavorable cost-benefit profile, or exhibits fundamental technical barriers.

3.1 Adaptive and Learned-Structure Mechanisms (Ideas 1.1–1.7)

This group covers mechanisms that learn to adapt the structure of computation — expert counts, layer depth, layer type, or output vocabulary — from the training data rather than fixing these choices as hyperparameters.

3.1.1 1.1 Learnable Per-Token Top- k Expert Selection

Standard MoE routing activates a fixed k experts per token regardless of token difficulty. Idea 1.1 replaces the static k with a per-token learned value, routing complex tokens to more experts and simple tokens to fewer, reducing average active expert count (and therefore memory bandwidth and TPOT) proportionally to the achieved average $\bar{k}$. At $\bar{k} = 7$ for Baseline B ($k = 11$), this translates to an estimated $\approx 1.57\times$ reduction in expert-FFN bandwidth; empirical support from AdaMoE [342], D2DMoE [301], and DTop-p [205] confirms quality-neutral or quality-positive outcomes at moderate

k reductions up to 30B-class MoE models. Applicable only to MoE baselines (Baseline B); N/A for dense A1 and A2.

Verdict: high-priority — multiple ICLR/NeurIPS/ACL 2024–2025 results confirm soundness; remaining novelty is the 512-expert scale and hybrid-DeltaNet integration.

3.1.2 1.2 Per-Token Adaptive Depth (Static Prefix / Dynamic Middle / Static Postfix)

This idea partitions the transformer stack into three zones: a mandatory prefix of L_{pre} layers, a dynamic middle of up to L_{mid} layers where each token exits as soon as a lightweight router judges its representation sufficient, and a mandatory postfix of L_{suf} layers applied to all tokens. The static postfix guarantees a uniform final-projection quality regardless of exit depth, distinguishing the design from CALM [284] and LayerSkip [51]. At $\bar{T} = L_{\text{mid}}/3$ the expected TPOT reduction is $\approx 1.3\text{--}1.5\times$, but realizing the bandwidth benefit requires a conditional weight-loading kernel that physically skips loading skipped-layer weights.

Verdict: candidate for investigation — core concept proven (CALM achieves $3\times$ speedup; LayerSkip $2.16\times$); the prefix/postfix structure is novel; the blocking engineering challenge is the conditional weight-load kernel.

3.1.3 1.3 Per-Layer Adaptive Expert Count

Rather than a uniform k across all MoE layers, Idea 1.3 assigns a different (static, post-calibration) active count k_l to each layer, concentrating expert capacity where it matters most and reducing it where layers are empirically less sensitive. The static variant (LExI-style [27]) requires no retraining and can be deployed in 1–2 engineer-days on an existing checkpoint; LExI on Qwen1.5-MoE achieves +0.5% average accuracy over same-throughput uniform pruning with a 10% relative accuracy improvement at matched throughput. The dynamic variant (per-layer k_l adapts to input difficulty) is the novel frontier: no published paper covers this for hybrid MoE+DeltaNet architectures at 512-expert scale.

Verdict: high-priority — static post-training variant immediately actionable with demonstrated benefit; dynamic variant warrants follow-on investigation.

3.1.4 1.4 Learned Dense vs. Sparse (Dense/MoE) Layer Assignment

Existing hybrid architectures (DeepSeek-V3, DeepSpeed-MoE) determine which layers use dense MLP vs. sparse MoE through human ablation. Idea 1.4 proposes learning this assignment end-to-end during pretraining via a differentiable gate, eliminating the manual search. AutoMoE [204] demonstrates NAS-based assignment at NMT scale achieves parity with hand-designed hybrids; DS-MoE [335] shows $1.86\times$ speedup at comparable perplexity on 6B models. The gap is end-to-end gradient-based learning of the dense/MoE partition at LLM pretraining scale without a separate NAS phase.

Verdict: candidate for investigation — assignment problem is real and expensive to solve manually; AutoMoE proves NAS feasibility at small scale; gradient-based LLM-scale implementation is unvalidated.

3.1.5 1.5 Learned Sparsity Type (N:M Unstructured vs. MoE Routing)

Different sparsity modalities offer fundamentally different hardware profiles: N:M weight sparsity (e.g., 2:4) preserves arithmetic intensity and maps efficiently to Tensor Cores, while MoE routing reduces active parameter count but at low arithmetic intensity per expert. Idea 1.5 trains a per-layer differentiable gate (Gumbel-Softmax) that selects between these two sparsity modalities, learning the globally optimal assignment rather than imposing one type across all layers. The gate complexity and mode-collapse risk are HIGH; the combined model requires custom deployment infrastructure; and weight memory savings accrue only to N:M-assigned layers (MoE-assigned layers store all experts at full density).

Verdict: candidate for investigation — hardware distinction between modalities is real and consequential; proof-of-concept on 1–7B model needed before larger commitment.

3.1.6 1.6 Learned Layer Type (Attention / SSM / Convolution / Linear-Attention)

Baseline A1 already uses two computational primitives (Gated DeltaNet + full attention) via a hand-designed 3:1 layout. Idea 1.6 extends this to four primitives — full attention, SSM/Mamba-2, causal convolution, and linear attention — and replaces the hand-designed layout with a NAS-discovered assignment. Nemotron-H [263] (92% Mamba-2) achieves $3\times$ throughput over all-attention baselines at 65K context; Hymba [48] achieves $11.67\times$ cache reduction. No published work has searched over a 4-primitive space at 7B+ scale; dynamic content-adaptive routing per position is the higher-risk secondary objective.

Verdict: candidate for investigation — static 4-primitive NAS is a low-risk extension of validated 2-primitive hybrids; the $3\text{--}12\times$ speedup range from existing hybrids provides strong a priori motivation.

3.1.7 1.7 Dynamic Vocabulary / Per-Step Active Vocabulary Subset

The output projection $W_{\text{out}} \in \mathbb{R}^{d \times V}$ must be evaluated at every generation step; for Baseline A1/B with $V = 248,320$ and $d = 5,120$ this is a 2.55 GB matrix. Idea 1.7 restricts the active vocabulary to a context-conditioned subset $V' \ll V$ per step, reducing output-projection bandwidth by $\approx V'/V$. The bandwidth reduction is real (CSV-Decode [32] achieves $2.95\times$ end-to-end speedup via geometric certification), but the hard missing-token failure mode — when V' excludes the correct next token, the model cannot generate it — is structurally different from soft quantization errors. The highest-impact deployment is as a draft-model optimization in speculative decoding (FR-Spec [349], DynaSpec [343]) where the target model provides a full-vocabulary backstop.

Verdict: candidate for investigation — verdict upgraded from lower-priority; effective 2–5% standalone TPOT improvement; highest value in speculative decoding draft-model context.

3.2 Dictionary and Compressed-Parameter Mechanisms (Ideas 2.1–2.2)

This group covers output-layer compression (adaptive softmax) and weight matrix compression via low-rank factorization.

3.2.1 2.1 Hierarchical Frequency-Based Output Dictionary

A frequency-stratified cascade of output dictionaries — common tokens resolved from a small head tier D_1 , rare tokens triggering evaluation of subsequent tiers — is mechanically identical to adaptive softmax [182], which has been open-sourced since ICML 2017. The cascade reduces LM-head compute by up to $76\times$ for the common-token tier, but for production 32B models the LM head is only $\approx 2.5\%$ of total weight bandwidth, yielding $\approx 2\text{--}3\%$ net TPOT improvement. The highest-leverage application remains speculative decoding draft models, where the LM head is 30–52% of total weight and the leverage is proportionally higher.

Verdict: lower-priority — core mechanism EXISTS as adaptive softmax (ICML 2017, shipped); for large 32B+ production models the TPOT benefit is only $\approx 2\%$; no meaningful research novelty remains.

3.2.2 2.2 Compressed Dense Layers via Low-Rank Factorization

This idea replaces dense MLP weight matrices W with a low-rank factorization $U \cdot V$ plus a sparse residual correction R , targeting $3\text{--}16\times$ MLP weight bandwidth reduction depending on rank and architecture. Post-hoc SVD methods (SVD-LLM [313], ASVD [334]) are immediately deployable with conservative settings (5% compression, $\approx +0.10$ PPL on WikiText-2); training-native low-rank factorizations (Wei et al. 2024 [320], SLTrain [193]) achieve up to 73% training memory reduction at comparable perplexity. For Baseline A2 at rank $r = 256$, the MLP weight bandwidth drops from ≈ 64 GB to ≈ 4 GB — a $\approx 16\times$ reduction on the MLP-weight component, yielding an estimated $3\text{--}5\times$ TPOT improvement in the bandwidth-bound decode regime.

Verdict: high-priority — post-hoc SVD immediately applicable to Baselines A1 and A2; training-native variant with learned sparse residual has HIGH potential impact and MEDIUM implementation risk.

3.3 MoE, Internal-State, and DAG Mechanisms (Ideas 3.1–3.8)

This group explores full-block routing of complete transformer layers through MoE, modular expert lifecycle management, and novel recurrent flow-control structures including DAGs and finite state machines.

3.3.1 3.1 Layer-Level MoE (Full Transformer Block Routing)

Rather than routing only the FFN sublayer (standard MoE), Idea 3.1 routes each token to k of E independently parameterized complete transformer blocks (attention + FFN + norms), enabling per-token selection of distinct attention patterns. The mechanism enables genuine attention specialization not achievable with FFN-only MoE, and up to E/k -fold TTFT reduction at routed positions; however, each full-block expert generates its own KV entries, inflating KV cache by up to $k\times$ at routed positions — a fundamental production blocker. MLA-style per-expert latent KV compression or shared KV projections with per-expert output differentiation are required mitigation strategies before experimental validation is worthwhile.

Verdict: candidate for investigation — attention-specialization novelty is real; $k \times$ KV inflation is a production blocker requiring architectural analysis before prototyping.

3.3.2 3.2 Swappable / Hot-Swappable Expert Weights

Modular MoE expert weights that can be added, removed, or replaced post-training without full model reload enable incremental domain specialization and knowledge lifecycle management. For Baseline B (512 experts), a single-expert swap requires only 1/512 of total expert weight bytes (≈ 240 MB one-time I/O) and router re-calibration at $\approx 2\%$ of pretraining compute [293]. The gap relative to existing Branch-Train-MiX and Nexus methods is zero-shot production hot-swap (no router retraining, no server restart) at LLM scale with maintained routing quality; Free-MoE [178] and MoRAM [258] approach but do not fully close this gap.

Verdict: candidate for investigation — individual mechanisms (insert, remove, re-calibrate) exist; end-to-end live hot-swap without router re-calibration is the open engineering problem.

3.3.3 3.3 Dynamic Expert Router (Extensible-Cardinality Router)

A router designed from the start to accommodate a variable-size expert set — adding or removing expert modules post-training without retraining the full model — addresses the knowledge-lifecycle maintenance problem for production MoE systems. REAP [278] and MoNE [257] demonstrate near-lossless expert removal at up to 50% pruning; LEMoE [312] and MEMoE [247] demonstrate effective knowledge injection via expert insertion. The critical unresolved gap is cold-start routing quality: newly inserted expert rows are unreachable until router re-calibration (ReLU-routed rows initialized to zero), and no system achieves truly zero-shot high-quality routing to newly inserted expert slots.

Verdict: candidate for investigation — operational value is real; cold-start routing gap is the key open engineering problem; router KD re-calibration [280] costs only $\approx 0.01\%$ of pretraining compute.

3.3.4 3.4 Recursive Internal State (Latent Recurrent Depth)

A while-loop within the forward pass iteratively refines token activations in a shared-weight middle block up to $T_{\max}$ times, with a learned per-token early-exit gate, entirely in latent space without emitting intermediate tokens. The core mechanism EXISTS and is well-validated: Universal Transformers [39] (ICLR 2019), Huginn-3.5B [181] (3.5B parameters matching 50B-class models at $T = 32$), and AdaPonderLM [291]. Both TTFT and TPOT increase with T iterations (unavoidably); the value proposition is quality-per-compute-dollar, not raw speedup. The novel gap is combining recurrent depth with Gated DeltaNet-style linear attention inside the loop and training with RLTT-style trajectory reward [326].

Verdict: candidate for investigation (hybrid application) / **lower-priority** (standalone) — mechanism EXISTS; standalone recurrent depth adds no efficiency benefit, only quality-per-parameter; pursue only for reasoning-dominated workloads where latent CoT reduces output length by more than the recurrence overhead.

3.3.5 3.5 Gated Internal DAG (Learned If/Else Computation Flow)

Rather than executing all neurons in an MLP uniformly, a learned DAG of sub-computations gates different sequential paths per token, where each token traverses only the subgraph relevant to its content. Channel Gating (CNNs) [199] achieves $2.6\text{--}8\times$ FLOP reduction with minimal accuracy loss at this granularity; no published work demonstrates intra-layer gated DAG in a generative LLM with TPOT measurements. The sequential DAG critical path — d_{dag} gate evaluations that must complete before the token proceeds — is the fundamental latency obstacle requiring empirical characterization.

Verdict: candidate for investigation — structural distinction from parallel MoE is theoretically motivated; recommend a 2-level binary DAG within the MLP of a 7B model to characterize whether the critical path overhead dominates bandwidth savings.

3.3.6 3.6 Recursive Internal DAG (Turing-Complete Flow Control)

Combining a shared-weight recurrent loop (Idea 3.4), a learned gated DAG within each iteration (Idea 3.5), and a persistent state tensor updated across loop iterations yields a forward-pass architecture capable of implementing bounded-Turing-complete computation. Each loop component has individual published validation (Universal Transformers for loops; IPA-GNN [15] for DAG execution), but the full combination — learned Turing-complete flow control trained from data on language tasks — has not been demonstrated. Training stability for all three components simultaneously is the paramount engineering challenge.

Verdict: candidate for investigation — loop component alone (Idea 3.4) provides most of the empirical benefit; ablation comparing looped-only vs. looped+DAG should precede full 3.6 implementation.

3.3.7 3.7 Learnable State Machine (FSM-Governed Computation Routing)

An explicit learned discrete N -state FSM maintained across tokens governs which computation blocks (attention, MLP, both, or neither) execute for each token, enabling state-dependent routing with sequential context awareness beyond per-token independent gating. Theoretical motivation is strong: [250] prove that continuous recurrent states cannot reliably track discrete symbolic state; discrete FSM-structured models [304, 305] achieve near-perfect FSA emulation where SSMs fail. No published experiment validates FSM-governed block routing in an LLM at inference time; the estimated $1.33\times$ TPOT improvement (at $p = 0.5$, $r = 0.5$ routing) is derived from first principles.

Verdict: candidate for investigation — inference-speedup case is theoretically motivated; recommend soft FSM prototype in 1–3B model to validate state-tracking benefit before hardening to discrete routing.

3.3.8 3.8 Trainable Activation Functions (Per-Neuron Affine-Clamp)

Replacing fixed SiLU activations with per-neuron learned affine+clamp nonlinearities (4 scalar parameters: slope, bias, lower clamp α , upper clamp β) adds $\approx 0.04\%$ memory bandwidth overhead

per layer and delivers +0.7–5.3 GLUE points improvement at BERT/1B scale [164, 352]. However, at 27B–397B scale quality gains from richer activations are expected to diminish; the idea delivers no inference speedup (TPOT increases $\approx 0.04\%$); and the windowed variant has a fundamental conceptual flaw (hidden-dimension indices are permutation-invariant, so a “window” over neighboring indices is semantically arbitrary). All core components (PReLU, PACT, rational activations, PolyGLU) are independently published prior art.

Verdict: lower-priority — negligible inference overhead but also no inference speedup; quality improvement expected to diminish at scale; windowed variant has fundamental flaw; revisit only if learned clamps yield exploitable activation sparsity.

3.4 Composite Architecture Mechanisms (Ideas 4.1–4.7)

This group covers composite architectural modifications combining multiple ideas: FSM-conditioned layer modulation, cross-layer weight sharing, LoRA-based pretraining factorization, skip-connection restructuring, residual flow gating, multi-model routing, and compressed output vocabulary structures.

3.4.1 4.1 State Machine Core (Universal FSM Conditioning of All Layers)

A discrete N -state FSM maintained across the full forward pass updates at each token step and conditions every layer’s computation via FiLM-style scale/shift [270], as opposed to Idea 3.7 (routing blocks) or external memory (NTM/DNC). The overhead is small: $\approx 1\%$ TPOT increase and $\approx 0.26\%$ memory bandwidth increase at $N = 256$, $L = 64$; the expressiveness gap over continuous recurrent states is proven [250]. Idea 4.1 overlaps completely with Idea 3.7 — any implementation satisfying one satisfies the other — and should be developed jointly; the key difference is framing (universal conditioner vs. block-skip router).

Verdict: candidate for investigation — small overhead, compelling theoretical motivation, but NLP benefit at LLM scale is completely unvalidated; must be developed jointly with Idea 3.7 to avoid duplicating effort.

3.4.2 4.2 Shared Core Weights + Per-Layer LoRA

A single shared weight matrix reused across all transformer layers, with small per-layer LoRA adapters (rank r) providing the only source of layer-to-layer specialization, achieves $18\text{--}49\times$ reduction in unique MLP parameters at rank $r = 64\text{--}512$. ALBERT [212] demonstrates $1.7\times$ training speedup via parameter sharing; Relaxed Recursive Transformers [6] achieve 58.4% vs. 58.6% few-shot accuracy at 20% parameter reduction with rank-512 adapters from a pretrained checkpoint. The critical limitation is zero TPOT benefit without early-exit synergy: the shared weight must still be loaded from HBM $L = 64$ times per decode step, identical bytes to 64 independent matrices.

Verdict: candidate for investigation — storage and training memory reductions are real ($18\text{--}49\times$); TPOT benefit only materializes when combined with early exit (Idea 1.2); pursue as a bundled proposal.

3.4.3 4.3 LoRA Everywhere (Full-Model Native Low-Rank Factorization)

Every weight matrix parameterized as $W = W_{\text{core}} + A \cdot B$ from the start of pretraining yields training-time optimizer memory savings ($\approx 50\%$ Adam state reduction) and — if adapters are retained unmerged at rank $r = d/4$ — approximately $2\times$ weight memory reduction and $\approx 1.64\times$ decode throughput improvement (CoLA [235]). The critical honest framing: only the unmerged, pure-factorization path (4.3-C) provides inference benefits; merged variants and variants with a shared W_{core} offer zero inference benefit over a dense baseline. Quality parity at scale $\geq 27\text{B}$ has not been demonstrated (CoLA validates only up to 7B) and uniform rank may be catastrophic for V-projections and MLP down-projections (WeLore [321]: perplexity 1836 vs. 11.87 at 50% uniform rank reduction).

Verdict: candidate for investigation — 4.3-C inference benefit ($\approx 1.64\times$ TPOT) is validated at $\leq 7\text{B}$; non-uniform rank allocation and quality parity at 27B+ must be established before committing.

3.4.4 4.4 Skip-List Layers (Exponential-Interval Residual Connections)

This idea replaces per-layer residual connections with skip connections placed at exponentially spaced intervals ($L/2, L/4, L/8, \dots$), forcing layers to learn non-trivial transformations and providing hierarchical gradient highways analogous to FPN [228]. There is no TPOT or TTFT benefit (no weights are removed; residual additions are $O(d)$ vs. $O(d^2)$ for matmuls); the primary claim is a quality improvement from forced non-trivial layer learning. The central problem is severe gradient vanishing: a 32-layer sub-block without internal residuals matches the regime (>34 layers without shortcuts) where He et al. [195] demonstrated convergence failure for plain networks.

Verdict: lower-priority (pure form) / **high-priority** (hybrid form, adding exponential-interval connections *on top of* existing per-layer residuals as a Hyper-Connections-style supplement).

3.4.5 4.5 Learned Residual Flow Control (Static-Gate Layer Pruning)

Scalar gates per layer are trained to discover which residual updates to skip; after training, zero-gate layers are physically eliminated at inference, reducing TPOT and TTFT proportionally to the pruned-layer fraction. ShortGPT [248] establishes that 24–27% of layers in LLaMA-2-7B/13B are empirically redundant post-hoc (BI metric); GateSkip [211] validates that learned gating can find these layers dynamically. The key unknown is whether a static gate trained end-to-end produces the same or better pruning patterns than post-hoc BI-based pruning for large autoregressive LLMs.

Verdict: candidate for investigation — inference speedup path ($\approx 25\%$ TPOT reduction) is real; targeted 1B–3B ablation comparing post-hoc pruning vs. train-time gating would resolve the critical uncertainty cheaply.

3.4.6 4.6 Mixture of Models (MoMoRA)

MoMoRA routes individual tokens at generation time to entire heterogeneous pre-trained models (not FFN sub-network experts), combining two-level hierarchical routing, a persistent GRU-style recurrent state across model selections, and ACT-style gated output emission [185]. The core novelty

— token-level routing to complete heterogeneous full models within a jointly-trained shared-interface architecture — is not present in any single published work; closest prior work (FrugalGPT [22], RouteLLM [264]) operates at query granularity without shared interfaces or recurrent state. However, the gated emission mechanism is an honest negative for latency: average TPOT increases (not decreases) relative to any single baseline model at typical operating points.

Verdict: candidate for investigation (conditional) — token-level full-model routing with recurrent state is genuinely novel; gated emission increases average TPOT rather than reducing it; 3–5× training cost requires robust prior validation of the routing mechanism.

3.4.7 4.7 Compressed Output Dictionary

A single output vocabulary dictionary with a compressed representation (fewer bits per entry and/or context-conditioned active subset $V_{\text{active}} \ll V$) reduces output-projection memory bandwidth at decode proportionally to V_{active}/V . Static-frequency subsetting (adaptive softmax, FR-Spec [349]) is well-established; context-conditioned dynamic masking has been demonstrated on draft models in speculative decoding (DynaSpec [343]) but not validated as a standalone optimization on a primary 32B+ autoregressive model without a full-vocabulary correction backstop. For large dense models ($\geq 32\text{B}$), the standalone TPOT improvement is modest ($\approx 2.3\%$ for Baseline A2) because the LM head is only $\approx 2.4\%$ of total weight bandwidth.

Verdict: candidate for investigation — static-frequency variant EXISTS; context-conditioned variant applied to primary model LM head is a genuine but modest gap; highest value in speculative decoding draft-model context.

3.5 Attention, Quantization, and Block-Sparsity Mechanisms (Ideas 5.1–5.10)

This group covers mechanisms that reduce memory bandwidth or footprint at the attention sublayer (KV cache quantization, gated linear attention, grammar-structured KV eviction, sparse windowed attention, double-pass attention) and at the weight-matrix level (block quantization, block sparsity, mixed-precision typing).

3.5.1 5.1 TurboQuant — Quantization-Aware Training at the KV Cache Interface

Aggressive quantization (INT4 or lower) of KV cache entries, trained into the model via QAT at the KV interface only, reduces long-context decode bandwidth by up to 4× at modest accuracy cost. Post-hoc TurboQuant [340] (ICLR 2026) achieves near-lossless 3.5-bit KV compression on Llama-3.1-8B with zero measured LongBench accuracy loss; the QAT extension is scientifically interesting but requires $\approx 1.35\times$ training cost for benefits most relevant at sub-3-bit regimes where PTQ degrades. The primary beneficiary is Baseline A2 (dense, ≈ 8.59 GB KV at 32K; ≈ 68.7 GB at 262K if extended) — not A1 or B whose KV caches are already small.

Verdict: candidate for investigation — deploy TurboQuant post-hoc on A2 immediately (high ROI, low risk); run QAT ablation at 2–3 bit to characterize whether training-time learning closes the residual PTQ gap.

3.5.2 5.2 LSTM-Gated Attention (Full KV Cache Elimination)

Replacing softmax attention with LSTM-style data-dependent gating (as in GLA [330], RWKV [268], Griffin [36], xLSTM [11], or Gated DeltaNet [329]) eliminates the growing KV cache entirely, replacing it with a fixed-size recurrent state (≈ 50 MB for Baseline A2 regardless of context length vs. ≈ 8.59 GB at 32K). The mechanism EXISTS and is already 75% deployed in Baselines A1 and B; converting the remaining 25% full-attention layers in the Qwen3.5-27B hybrid eliminates the last ≈ 2.15 GB of KV cache. The open engineering question is whether the ≈ 10 – 22% quality gap on recall-intensive tasks (associative recall, needle-in-haystack, multi-hop QA) is acceptable for the target workload mix.

Verdict: candidate for investigation / HIGH PRIORITY for A2 — KV elimination is validated at scale; ≈ 8.59 GB $\rightarrow$ 50 MB reduction for A2 at 32K is the primary target; quality cliff on recall tasks must be characterized.

3.5.3 5.3 Grammar/FSM-Structured Attention (Sparse KV via Learned Masks)

A learned grammar or FSM constrains which tokens attend to which, storing only the grammar-relevant KV entries and thereby reducing KV cache from $O(s \cdot d_{kv})$ to $O(|S| \cdot d_{kv})$ per token where $|S| \ll s$. Fixed sparse patterns (sliding window, global tokens) are thoroughly validated; MoSA [272] and NSA [336] demonstrate learned sparse attention with production-quality hardware alignment; the specific novelty of Idea 5.3 is using an explicit differentiable FSM/grammar as the structural principle rather than heuristic attention-score eviction. At 10% KV retention for Baseline A2 at 32K context, the total TPOT ceiling is $\approx 1.1\times$ (KV is only $\approx 12\%$ of total bandwidth at 32K; the benefit scales to $\approx 1.9\times$ at 262K if extended).

Verdict: candidate for investigation — practical variant reduces to a reframing of NSA; incremental novelty over NSA requires explicit justification; block-aligned grammar tokens (NSA approach) recommended to avoid hardware-inefficiency of irregular sparse patterns.

3.5.4 5.4 Linked Attention (Dynamic KV Cache Eviction with Sparse Graph)

Dynamic inference-time KV eviction retaining only the top- k most relevance-scored token entries per head reduces KV memory footprint and decode-phase bandwidth at the cost of potential retrieval failures on non-recent, non-heavy-hitter tokens. H2O [344], Scissorhands [234], SnapKV [222], PyramidKV [18], and Quest [302] collectively demonstrate that 8–20% KV retention is sufficient for generation tasks with negligible quality loss. The novel contribution is a persistent, incrementally updated sparse adjacency structure (“linked graph”) with value-norm-aware relevance scoring [333], bridging offline eviction (SnapKV) and full online retrieval (InfLLM); primary open question is multi-turn and exact-retrieval quality (SCBench [221] demonstrates consistent failure of sub- $O(n)$ methods on string retrieval).

Verdict: candidate for investigation — pure attention-score eviction is a re-implementation; persistent sparse graph with value-norm-aware scoring is differentiated; deploy DuoAttention-style hybrid immediately, prototype incremental-graph formulation in parallel.

3.5.5 5.5 Ragged Window Attention (Per-Token Variable Contiguous Lookback)

Per-token variable-width contiguous lookback windows in causal attention, where each token position independently determines how many past KV entries to attend to, reduces KV cache footprint proportionally to the average window width. At $w_{\text{avg}} = 2\text{K}$ and $s = 32\text{K}$, the Qwen3-32B KV cache drops from ≈ 8.59 GB to ≈ 0.537 GB ($16\times$), enabling single-GPU serving at full 32K context. The variable window creates a ragged attention pattern requiring custom Triton/CUDA kernel support; the quality cliff on long-range coreference and multi-hop retrieval tasks at $w_{\text{avg}} = 2\text{K}$ must be characterized before committing.

Verdict: candidate for investigation — $16\times$ KV reduction is a compelling deployment win; quality cliff characterization on LongBench/RULER is a 2–4 day experiment; FlexAttention and FlashInfer may express ragged windows without full custom kernel development.

3.5.6 5.6 Double Attention (Two Sequential Self-Attention Passes Per Block)

Running two sequential self-attention passes per transformer layer enriches per-layer representation quality, potentially enabling depth reduction that offsets the per-layer compute increase. DaViT [46] (ECCV 2022) achieves +1.2–4.5% top-1 accuracy at $\approx 7\%$ extra FLOPs in vision; no experiments exist for decoder-only language modeling at scale. If double attention allows 30–50% fewer layers (supported by depth–width tradeoff literature [283]), then a 42-layer model with 2 attention passes per block matches a 64-layer single-attention model with $\approx 34\%$ KV cache reduction and $\approx 34\%$ TPOT improvement — this remains a hypothesis requiring empirical validation.

Verdict: candidate for investigation — vision precedent is promising; depth-reduction hypothesis is the load-bearing unknown; a 4–8 layer pilot at 300M–1B scale would resolve feasibility in ≈ 2 weeks.

3.5.7 5.7 Block-Level Compressed Weights (INT4 Quantization)

Block-wise weight quantization (GPTQ [172], AWQ [229], ATOM [347], QServe [230]) at INT4 reduces weight memory by $\approx 4\times$ (from ≈ 64 GB to ≈ 16.5 GB for Baseline A2 in bf16) and delivers $\approx 2\text{--}3\times$ practical TPOT improvement at batch=1 decode. This is post-training, requires only calibration data (no retraining), and has production-quality kernel implementations (MARLIN [173] reports $2.8\times$ TPOT on A6000; QServe reports $2.4\text{--}3.5\times$ on Qwen1.5-72B). The novel sub-variant is per-sub-matrix-tile independent low-rank factorization within a single weight matrix (BLAST [215] direction), which could improve quality at extreme compression ratios (< 2.5 bits/param) when combined with CALDERA-style quantization+low-rank [14].

Verdict: high-priority — INT4 block quantization is production-ready via GPTQ/AWQ/ATOM/QServe; per-tile low-rank sub-variant warrants further investigation at 27B–32B scale.

3.5.8 5.8 Block Sparse Weights (Structured Sparsity via BSR or 2:4 Format)

Zeroing out entire contiguous blocks of MLP/projection weight matrices allows sparse kernels to skip loading those blocks at decode, reducing weight bandwidth and storage. SparseGPT [171]

(ICML 2023) achieves near-lossless 50% unstructured sparsity at 175B scale; 2:4 structured sparsity via Wanda [295] degrades quality significantly at 7B (PPL +5.85) but moderately at 70B (PPL +1.86), and MaskLLM [165] (NeurIPS 2024) recovers much of this via learned mask training (PPL +1.60 at 7B). BSR-format block sparsity at 30B+ scale on a single GPU is the uncharacterized regime (BLaST [16] largest single-model result is 1B; $2.2\times$ is a 16-GPU 540B result).

Verdict: candidate for investigation — mechanism proven with excellent hardware support; quality–speedup tradeoff at 30B scale with BSR-format sparsity is uncharacterized; targeted experiment on Qwen3-7B or -14B resolves both quality and speedup cheaply.

3.5.9 5.9 Dynamic Per-Value Numeric Type (Per-Element Mixed Precision)

Assigning different numeric types (fp16/int8/int4) per weight element is theoretically appealing for memory reduction but practically blocked by metadata overhead: storing 1 bit of type tag per 4-bit weight incurs 25% overhead, partially negating compression; true per-element conditional dequantization is an anti-pattern on Tensor Cores that incur $\approx 500\times$ overhead vs. block-level processing. SpQR [42] (ICLR 2024) and OWQ [214] demonstrate that a small outlier fraction stored at elevated precision (3.1–3.13 bits effective) achieves <1% relative PPL loss; per-group or per-block typing (rather than per-scalar) is the viable implementation unit. The review reframes this idea as a per-block/per-group precision tool for accuracy recovery at aggressive average compression, not a throughput improvement tool.

Verdict: candidate for investigation — reframe as per-group/per-block dynamic type; per-scalar formulation is blocked by hardware constraints; SpQR/OWQ-style outlier isolation on Qwen3-class models with Triton kernels is the actionable path.

3.5.10 5.10 Block-Level Dynamic Type Compression ($k \geq 3$ Type Levels)

Compressing weight blocks with a shared scale header but per-element precision type flags within each block amortizes metadata overhead and enables mixed precision at fine granularity. The 2-level variant (binary outlier flag) EXISTS in SpQR (ICLR 2024) [42] and SqueezeLLM [206] (ICML 2024); hardware-level per-element variation within MX blocks EXISTS in MX+ [216] (MICRO 2025). The novel contribution is generalizing to $k \geq 3$ type levels with interleaved type flags enabling sequential memory access at decode — validated 2-level approaches (SpQR: <1% PPL loss at 4.6 bits) achieve the target TPOT improvement ($\approx 3\text{--}4\times$); $k \geq 3$ generalization is unvalidated and requires custom kernel development.

Verdict: candidate for investigation — replicate SpQR on Qwen3-32B to validate $3\text{--}4\times$ TPOT claim; extend to $k \geq 3$ type flags in a subsequent kernel engineering phase.

3.6 Autoregressive-Architecture Mechanisms (Ideas 6.1–6.5)

This group introduces five novel architectural proposals developed as part of this review. Unlike the ideas in groups 1–5, which analyze pre-existing proposals, these five represent original architectural combinations whose Myrmidon-Swarm reviews were conducted de novo alongside the correction of the baseline specification document.

3.6.1 6.1 In-Architecture Autoregressive Loop with Learned Stopping

This mechanism internalizes the autoregressive generation loop as a trained recurrent module inside the model graph, allowing the model to learn *when to stop generating* via a differentiable halt signal or a policy-gradient-trained `<|stop|>` token. A small GRU cell or MLP maintains a loop-state vector $h_{\text{loop}} \in \mathbb{R}^{d_{\text{loop}}}$ across generation steps, producing a scalar halting probability $p_{\text{halt}}(t) = \sigma(W_{\text{halt}} \cdot h_t)$; training uses truncated BPTT with a halting regularization loss. The per-step overhead is genuinely negligible ($\approx 0.12\text{--}0.18\%$ FLOPs), but the training challenge is real (BPTT through long sequences, doubly-nested DeltaNet recurrence for A1/B); prior art density is high (ACT [185], PonderNet [8], Universal Transformers [39], Looped Transformers [162]), with the novelty narrowing to the discrete stop-token applied to the outer AR sequence within a hybrid DeltaNet model.

Verdict: candidate for investigation — technically sound and motivated; novelty contribution is incremental given existing looped-transformer literature; prototype on A2 (pure-attention, no DeltaNet complexity) recommended.

3.6.2 6.2 Prefill/Decode Architectural Split

Two architecturally distinct sub-networks are co-trained: a wider/deeper prefill network optimized for compute-bound parallel context processing, and a narrower/shallower decode network optimized for memory-bound autoregressive generation, connected by a compressed state interface (DeltaNet recurrent state for hybrid models, cross-attention pooled tokens for dense models). For Baseline A2 ($L=64$), splitting at $L/2$ with a DeltaNet-only decode sub-network eliminates $\approx 50\%$ of the prefix KV cache and reduces decode TPOT by an estimated $\approx 41\%$; for Baseline B, active decode parameters drop from $\approx 17\text{B}$ to $\approx 8\text{--}9\text{B}$ per step. No exact prior art was found; the closest work (YOCO [296]) proposes dual decoders for KV reduction but does not pursue regime optimization (wider prefill, narrower decode) or DeltaNet recurrent state as the compressed interface.

Verdict: high-priority — PARTIAL NOVELTY; addresses a real production bottleneck ($\approx 50\%$ TPOT reduction, $\approx 50\%$ KV savings for A2); HIGH priority for hybrid architectures (A1/B) where DeltaNet provides a natural $O(1)$ compressed state interface.

3.6.3 6.3 Block-Diffusion Autoregressive Decoder

The decoder partitions the output sequence into blocks of k tokens; each block is generated via D denoising steps using bidirectional attention within the block and causal attention to committed prior blocks. At $k = 8$ and $D = 4$, the total forward pass count is $(T/k) \times D = T/2$ — a $2\times$ reduction vs. standard AR in the memory-bandwidth-bound regime (not $4\times$ as claimed in the original idea description). BD3-LM [261] (ICLR 2025 Oral) implements this exact mechanism with full theoretical justification and empirical validation; the idea as described is therefore not novel as a standalone contribution, though applying block-diffusion to hybrid DeltaNet architectures (A1/B), where DeltaNet state interacts with within-block masked diffusion, is unexplored territory.

Verdict: lower-priority (as novel contribution) / frame as “BD3-LM applied to hybrid recurrent-attention architectures” — the mechanism is fully covered by BD3-LM (ICLR 2025); remaining contribution is the hybrid-architecture application.

3.6.4 6.4 Combined: AR Loop + Prefill/Decode Split + Block Diffusion

Integrating Ideas 6.2 (prefill/decode split) and 6.3 (block diffusion) achieves a multiplicative FLOP reduction because the two optimizations act on orthogonal dimensions: the split reduces layers per forward pass by $L_d/L = 0.5$, and block diffusion reduces forward passes per output token by $D/k = 0.5$ (at $D = 4$, $k = 8$). Combined FLOP ratio = $(D/k) \times (L_d/L) = 0.25$ — a $4\times$ decode FLOP reduction — without increasing parameter count, applicable to all three baselines. Idea 6.1 (in-architecture stopping) adds $\approx 0\%$ FLOPs but enables the system to avoid generating tokens after task completion without external stopping logic. No prior work integrates all three components; quality at $D = 4$ denoising steps is documented as viable by BD3-LM [261] and Fast-dLLM [168].

Verdict: high-priority — mathematically sound; benefits multiply (not add); $4\times$ decode FLOP reduction achievable without parameter count increase; first pilot recommended on Qwen3.5-27B Hybrid (A1) due to natural DeltaNet state interface; timeline 6–12 months for a team with LLM training infrastructure.

3.6.5 6.5 Pre-Attention Expert Router (Router on Q/K/Activations)

Standard MoE models place the FFN expert router on the post-attention residual stream, so routing decisions are informed by attention-aggregated token representations. Idea 6.5 proposes relocating the router to operate *before* or *in parallel with* the attention sublayer, routing on the pre-attention activation vector, the Q projection, the K projection, or a combination of (Q, K). Because the router now sees non-attended, locally-derived token representations, expert selection is based on the token’s *intrinsic* type rather than its contextualised meaning. Prior art review confirmed this is **novel** for FFN expert routing; SwitchHead [31] tested Q/K routing for attention-head MoE and abandoned it, but no published work applies this to FFN expert selection. See §4.5 for full analysis.

The following subsections provide the complete per-idea analysis for all 35 candidate mechanisms. Each analysis follows a uniform structure: mechanism description, prior art with verified arXiv references, complexity analysis relative to the three baselines, integrated review findings (citation verification, Big-O audit, missing literature, technical correctness), and a final verdict with confidence scores. All KV cache values use the canonical formula derived in Section 2 with corrected GQA head counts.

3.7 Idea 1.1: Learnable Per-Token Top- k Expert Selection

Mechanism

Standard Mixture-of-Experts (MoE) layers use a fixed integer k —every token activates exactly k experts regardless of the token’s difficulty or the layer’s effective routing entropy. Idea 1.1 replaces the fixed- k selector with a lightweight *per-token k -predictor*: a small linear head (or 2-layer MLP) applied to the current token’s hidden state h_t that outputs a scalar confidence $c_t \in [0, 1]$. From c_t a discrete k_t is derived (e.g., via a threshold schedule), and the MoE router uses k_t active experts for that token. The predictor is jointly trained with the router’s load-balancing loss and the language-modelling objective; at inference only the predictor head adds overhead. For Baseline B (512 experts, $k = 11$), reducing the average activated expert count to $\bar{k} = 7$ decreases active-expert weight bandwidth

proportionally to $7/11 \approx 0.64\times$, yielding an estimated $1.4\text{--}1.56\times$ TPOT improvement on the expert-weight bandwidth term. The key engineering challenge at 512-expert scale is that variable- k dispatch breaks the static expert-parallelism assumptions embedded in existing FlashMoE/DeepSpeed-MoE kernels; per-token irregular dispatch requires custom grouped-GEMM kernels.

Prior Art and Related Work

The closest published antecedents cover the variable- k -per-token paradigm across several sub-families:

- **Expert Choice routing** [158]: experts select their top- k tokens (inverted routing); per-token expert count varies as an emergent property. Achieves $> 2\times$ training efficiency vs. Switch Transformer but is incompatible with autoregressive decode. arXiv: exttt2202.09368
- **AdaMoE** [151]: null-expert mechanism, top-3 true experts plus null slots; 14.55% FLOPs reduction, +1.69% ARC-C improvement. EMNLP 2024 Findings. arXiv: exttt2406.13233
- **DynMoE** [87]: per-token “top-any” gating; each token autonomously determines k_t via a learned threshold; competitive performance with fewer active parameters. ICLR 2025. arXiv: exttt2405.14297
- **ReMoE** [139]: relaxed top- k via a continuous sparsification objective; differentiable routing removes the need for straight-through estimation. ICLR 2025. arXiv: exttt2412.14711
- **D2DMoE** [135]: data-dependent routing, activates experts whose score exceeds $\tau \cdot \max R(z)$; up to 60% FLOPs reduction; near- $3\times$ speedup for converted MoE. arXiv (venue unconfirmed as NeurIPS 2024 main). arXiv: exttt2310.04361
- **DTop-p** [100]: PI-controller-based dynamic top- p routing; consistently outperforms fixed- k and fixed-threshold baselines (50.9 vs. 49.0 across 13 benchmarks). arXiv: exttt2512.13996
- **SeqTopK** [142]: routes top $T \cdot K$ expert-token pairs across all T tokens (sequence-level budget); up to 16.9% performance gain at $K = 2$; average TPOT per token unchanged. arXiv: exttt2511.06494
- **Huang et al. (ACL 2024)** [95]: attention-derived difficulty signal drives variable k ; average 1.76 experts vs. 2.0 baseline with +0.7% mean improvement. arXiv: exttt2403.07652
- **DA-MoE** [53]: attention-score-derived importance for routing, GLUE outperformance claimed. arXiv: exttt2409.06669
- **DynaMoE** [192]: six layer-scheduling strategies (ascending, descending, pyramid, wave, uniform) for dynamic k schedules; optimal schedule is task- and scale-dependent. arXiv: exttt2603.01697
- **DeepSeekMoE** [66]: shared-plus-routed expert split; DeepSeekMoE-16B matches LLaMA-2-7B at $\sim 40\%$ compute. arXiv: exttt2401.06066

Gap: No published experiment demonstrates learned per-token k at 512-expert scale with irregular grouped-GEMM dispatch in an autoregressive decode setting. LD-MoLE [160] (arXiv:

exttt2509.25684, ICLR 2026) applies a directly analogous mechanism to LoRA fine-tuning experts, partially closing the novelty gap. *Prior art overlap: partial* ($\sim 75\%$).

Complexity Analysis

Table 5: *Complexity impact of Idea 1.1 (Learnable Per-Token Top-k).*

Baseline	TPOT change	Notes
A1	$\approx \times 1$	25% full-attn layers unaffected; DeltaNet layers carry no MoE k -routing; net savings minimal
A2	N/A	Dense FFN; no expert routing
B	$\downarrow 1.4\text{--}1.56\times$	$\bar{k} = 7$ from $k_{\max} = 11$; active-expert weight bandwidth $\propto \bar{k}/k_{\max}$; custom grouped-GEMM required; gating overhead $\sim 0.03\%$ of layer FLOP

Big-O for a single MoE layer: $O(k_t \cdot d \cdot d_e)$ per token with variable k_t ; predictor head adds $O(d)$. TTFT: unchanged (prefill is compute-bound; weight loading is parallel). KV cache: unchanged (expert routing does not alter the attention KV footprint).

Verdict

high-priority. Static variable- k (post-hoc $\bar{k}$ reduction) on Baseline B is immediately deployable with 1–2 engineer-days of router recalibration. The learned predictor variant requires custom grouped-GEMM kernels and joint training; estimated 3–6 months.

Confidence scores: *Literature coverage* 7/10 *Technical soundness* 8/10 *Baseline comparison* 7/10
Overall $\sim 7/10$

Full analysis: research_1_1_learnable_top_k.md in the research corpus.

3.8 Idea 1.2: Per-Token Adaptive Depth (Early Exit)

Mechanism

Per-token adaptive depth partitions the model into three zones: a static *prefix* (first K layers, all tokens always execute), a dynamic *middle* (layers K to $L-K$, tokens may exit early), and a static *postfix* (last K layers, all surviving tokens execute for output projection). Within the middle zone, a lightweight exit classifier—a single linear head over the residual stream—computes a confidence score at each candidate exit point. If the score exceeds a learned threshold, the token bypasses all remaining middle layers and jumps to the postfix. TPOT speedup requires a *conditional weight-load kernel*: for a 40% early-exit rate at layer $L/2$, those tokens avoid loading $\sim$ half the model weights, reducing their individual bandwidth cost proportionally. Without that kernel, the weight-load proceeds for all tokens regardless of exit status, yielding near-zero wall-clock benefit. FlexiDepth

[76] measured actual speedup of 6.6–8.1% wall-clock vs. a projected theoretical $1.3\text{--}1.5\times$ —the discrepancy arises because synchronization overhead and exit-classifier cost dominate at moderate exit rates.

Prior Art and Related Work

- **ACT (Graves, 2016)** [184]: Adaptive Computation Time in RNNs; per-step halting probability; foundational. arXiv: exttt1603.08983
- **Universal Transformers** [68]: recurrent depth with ACT halting in transformers; +0.9 BLEU on WMT14. arXiv: exttt1807.03819
- **Depth-Adaptive Transformer** [71]: per-token depth adaptation at test time with confidence-based halting; tested on WMT14/WMT16.
- **CALM** [131]: Confident Adaptive Language Modelling; calibrated early exit at NeurIPS 2022; validated on summarisation and QA tasks. arXiv: exttt2207.07061
- **Mixture of Depths (MoD)** [128]: tokens routed to *bypass* layers (not exit); distinct from early exit. ICML 2024. arXiv: exttt2404.02258
- **LayerSkip** [72]: early exit with self-speculative decoding; $1.34\text{--}2.16\times$ speedup within 1–2% of full baseline. arXiv: exttt2404.16710
- **FlexiDepth** [76]: 20B+ scale; measured 6.6–8.1% wall-clock speedup. arXiv: exttt2603.23701
- **MoR (Bae et al., 2025)** [55]: Mixture of Recursions; early exit combined with weight-sharing.
- **ITT, AdaInfer, TIDE, DEL, SkipGPT**: multiple 2024–2025 works exploring dynamic exit criteria; TIDE year is 2026 (not 2025 as earlier drafts stated).

Critical errors identified in an earlier draft of the research document (review flags C1–C7): co-author “Hwanjun Song” in the FlexiDepth citation was fabricated (not an author of arXiv: exttt2603.23701); 59 GB bf16); TIDE year incorrect (2026 not 2025); TIDE author list wrong; one unverifiable citation; EE-LLM and Diminishing Returns paper missing. *Prior art overlap: partial ($\sim 60\%$).*

Complexity Analysis

Big-O: $O(K \cdot d^2)$ for static prefix/postfix plus $O(\bar{D} \cdot d^2)$ per token in the middle, where $\bar{D}$ is mean exit depth. TTFT: unchanged (exit logic does not apply to prefill in standard formulations). KV cache: exit tokens cease KV generation; cache footprint decreases $\propto$ exit rate.

Verdict

candidate for investigation. The conditional weight-load kernel is the critical unbuilt infrastructure. Training co-design (layer-dropout schedule + exit loss + router loss) adds significant complexity. Recommend Phase 2 investigation: build the kernel on a 7B model, measure wall-clock speedup empirically, then decide whether to proceed.

Table 6: *Complexity impact of Idea 1.2 (Per-Token Adaptive Depth).*

Baseline	TPOT change	Notes
A1	↓ 6–8% (wall-clock)	Theoretical 1.3–1.5×; realistic at 25% early-exit rate requires conditional weight-load kernel
A2	↓ 6–8% (wall-clock)	All 64 layers eligible; same kernel requirement
B	↓ 6–8% (wall-clock)	MoE routing overhead at each layer raises exit-classifier cost

Confidence scores: *Literature coverage* 6/10 *Technical soundness* 8/10 *Baseline comparison* 7/10
Citations 5/10 *Overall* 6.5/10

Full analysis: research_1_2_per_token_adaptive_depth.md in the research corpus.

3.9 Idea 1.3: Per-Layer Adaptive Expert Count

Mechanism

Standard MoE models use a fixed k at every layer. Per-layer adaptive expert count assigns a distinct k_l to each layer l based on that layer’s measured routing entropy—layers where nearly all routing weight concentrates on the top-1 or top-2 experts (low effective k) safely deploy with a smaller k_l than layers with flat routing distributions. In the *static* variant, post-hoc calibration on a held-out set identifies per-layer optimal k_l values with zero retraining cost. In the *dynamic* variant, a lightweight per-layer predictor adjusts k_l per token (analogous to Idea 1.1 but applied at the layer level rather than globally).

For Baseline B (60 layers, 512 experts, $k = 11$), an illustrative static assignment with 20 layers at $k_l = 6$, 30 layers at $k_l = 11$, and 10 layers at $k_l = 14$ yields:

$$\bar{k}_l = \frac{20 \times 6 + 30 \times 11 + 10 \times 14}{60} = \frac{120 + 330 + 140}{60} = \frac{590}{60} \approx 9.83$$

a 10.6% reduction in average active-expert bandwidth with zero training cost.

Prior Art and Related Work

- **LExI** [63]: layer-adaptive active expert inference; validated on Qwen1.5-MoE on H100 with vLLM; +0.5% average over 9 LM-Eval tasks at reduced k_l . arXiv: exttt2509.02753
- **Alloc-MoE** [116]: budget-aware expert activation allocation; ACL 2026. arXiv: exttt2406.09956
- **Huang et al. (ACL 2024)** [95]: task-difficulty signal drives variable k ; average 1.76 experts vs. 2.0 baseline. arXiv: exttt2403.07652
- **DA-MoE** [53]: attention-derived importance routing; GLUE outperformance. arXiv: exttt2409.06669

- **DiEP** [56]: differentiable expert pruning; per-layer expert count compressibility analysis. arXiv: exttt2509.16105
- **GRAPE** [155]: global-perspective sparse MoE pruning; per-layer calibration-based k_l reduction. arXiv: exttt2604.06542
- **MoE-I2** [148]: inter-expert pruning + intra-expert low-rank decomposition; EMNLP Findings 2024.
- **MoLA** [77]: more LoRA experts allocated to *middle* layers (not later layers as earlier drafts stated—a documented discrepancy corrected by review). NAACL Findings 2025.
- **AdapMoE** [157]: 25% reduction in activated experts; $1.35\times$ speedup; no accuracy loss on tested benchmarks. ASPLOS 2025. arXiv: exttt2408.10284
- **D-LLM (Jiang et al., NeurIPS 2024)** [99]: dynamic layer-level computation allocation in dense models; 45–50% compute reduction. (Full author list: Yikun Jiang, Huanyu Wang, Lei Xie, Hanbin Zhao, Chao Zhang, Hui Qian, John C.S. Lui.)
- **Optimal Expert-Attention Allocation** [111]: scaling-law analysis for heterogeneous expert allocation. arXiv: exttt2603.10379

Missing from original draft: DynMoE [87] (ICLR 2025, directly relevant); DynaMoE [192] (task/scale dependence of schedules); LAEP in Yuan3.0 Ultra [144] (arXiv: exttt2601.14327; 49% pretraining efficiency at 1.5T parameters); MoE-Spec [118] (arXiv: exttt2602.16052; per-layer budget caps in speculative decoding, 10–30% throughput). *Prior art overlap: exists ($\sim 80\%$).*

Complexity Analysis

Table 7: *Complexity impact of Idea 1.3 (Per-Layer Adaptive Expert Count).*

Baseline	TPOT change	Notes
A1	$\approx \times 1$	Dense MLP layers; no MoE routing
A2	N/A	Dense; no experts
B	$\downarrow 1.4\text{--}1.56\times$ (static); up to $1.65\times$ (dynamic)	Static: 1–2 engineer-days; validated at comparable MoE scale; dynamic requires fine-tuning

Per-layer big-O: $O\left(\sum_{l=1}^L k_l \cdot d \cdot d_e\right) = O\left(L \cdot \bar{k}_l \cdot d \cdot d_e\right)$. No change to TTFT (prefill compute-bound, weight loading parallelised). No change to KV cache.

Verdict

high-priority. Highest-feasibility non-trivial inference optimisation in the entire corpus. The static variant (calibrate k_l offline, redeploy) is immediately executable with 1–2 engineer-days on Baseline B. Dynamic variant is a Phase 2 task after static validation.

Confidence scores: *Literature coverage* 8/10 *Technical soundness* 9/10 *Baseline comparison* 8/10
Overall 8/10

Full analysis: research_1_3_per_layer_adaptive_expert_count.md in the research corpus.

3.10 Idea 1.4: Learned Dense vs. Sparse Layer Assignment

Mechanism

Each transformer layer receives a soft binary gate $g_l \in [0, 1]$ trained via Gumbel-Softmax or DARTS-style differentiable architecture search to choose between a dense FFN and an MoE (sparse) FFN. At the end of the search phase the gates are binarised (hard assignment) and the model is deployed with the fixed layer-type assignment. This replaces hand-designed hybrid ratios (e.g. Qwen3.5’s fixed 75% DeltaNet / 25% full-attention split) with data-driven architecture search.

Training cost is $\approx 1.5\text{--}3\times$ due to parallel forward passes through both layer types during gate-learning; assignment collapse (all layers converging to the same type) requires regularisation. The key correctness error from earlier draft: Baseline A1’s FFN layers are *dense MLP*, not MoE FFN; and Baseline B uses 512 experts with $k = 11$ (not 256 experts, $k = 8$, which is the DeepSeek-V3 configuration incorrectly inherited from an earlier baseline table).

Prior Art and Related Work

- **AutoMoE** [97]: NAS-driven dense–MoE assignment; parity BLEU with dense Transformer on WMT translation.
- **MoEfication** [153]: converts dense layers to MoE post-hoc without training; validates that the dense–MoE assignment can be solved empirically. arXiv: exttt2110.01786
- **CMoE** [29]: post-hoc MoE conversion with calibration data.
- **DARTS** [117]: differentiable architecture search; training-time analogue for the gate-learning phase. arXiv: exttt1806.09055
- **DS-MoE (Yuan et al., 2024)**: dense-to-sparse training; $1.86\times$ speedup reported; directly tests a form of the dense–sparse assignment.
- **ReMoE** [139]: smooth routing that continuously interpolates between dense and sparse regimes—the soft analogue of the binary gate in Idea 1.4.
- **ExpertWeaver**: structured expert assignment; claimed speed improvements (URL unverified in review as of 2026-04-13).
- **Mixtral-8x7B** [254]: all-MoE architecture baseline motivating sparse layer exploration.
- **DeepSeek-V3** [37]: arXiv: exttt2412.19437—note: an earlier draft cited the wrong URL; corrected here.

Missing: end-to-end gradient-based learning of the dense/MoE partition during standard LLM pretraining at scale has not been published. *Prior art overlap: partial ($\sim 55\%$).*

Complexity Analysis

Table 8: *Complexity impact of Idea 1.4 (Learned Dense vs. Sparse Layer Assignment).*

Baseline	TPOT change	Notes
A1	$\downarrow 1.5\text{--}2.5\times$ (ideal)	If gate assigns dense MLP layers to MoE; realistic at $k \cdot d_e = d_{\text{ff}}/8$ assumption—TTFT $\sim 1.5\times$ at $s = 8\text{K}$, not $\sim 3\times$ as earlier drafts stated
A2	$\downarrow 1.5\text{--}2.5\times$	Largest opportunity: 64 dense attention layers converted
B	marginal	Already 75% DeltaNet; gate search may refine the 25% full-attn share

TTFT at $s = 8\text{K}$: corrected from “ $\sim 3\times$ ” to $\sim 1.5\times$ —the overestimate assumed all layers convert. TPOT: $\sim 1.5\text{--}2\times$ improvement at the MLP bandwidth level only; attention bandwidth unchanged.

Verdict

candidate for investigation. Merge with Idea 1.5 into a *Unified Layer-Type Architecture Search* covering three options: dense / N:M sparse / MoE. No published LLM-scale end-to-end gradient learning of the partition. Phase 3 work (requires pretraining from scratch at 1–3B scale).

Confidence scores: *Literature coverage* 7/10 *Technical soundness* 8/10 *Overall* 7/10

Full analysis: research_1_4_learned_dense_sparse_layer.md in the research corpus.

3.11 Idea 1.5: Learned Sparsity Type Selection (N:M vs. MoE Routing)

Mechanism

Extends Idea 1.4 by adding NVIDIA 2:4 semi-structured N:M weight sparsity as a third option alongside dense and MoE. A per-layer gate $g_l \in \{\text{dense}, 2:4 \text{ sparse}, \text{MoE}\}$ is trained to select the sparsity modality that minimises per-layer perplexity subject to a FLOPs budget. At layers where routing entropy is high (flat routing distribution), 2:4 sparsity is preferable because MoE load-balancing fails; at layers where routing is sharp (concentrated expert preference), MoE is preferable. The combined deployment infrastructure must include: cuSPARSELt for 2:4 sparse GEMMs and standard MoE dispatch in the same model.

Mode-collapse risk is HIGH: with only gradient signals, all layers may converge to the cheapest option (dense or 2:4 sparse) and ignore MoE routing. Gate overhead at $d_{\text{dag}} \geq 4\text{--}5$ cancels the expected savings.

Prior Art and Related Work

- **MaskLLM** [73]: learned 2:4 mask via generative modelling; LLaMA-2-7B PPL 5.12 $\rightarrow$ 6.72 at 2:4 sparsity. NeurIPS 2024 Spotlight. arXiv: exttt2404.12875
- **SparseGPT** [177]: post-hoc N:M pruning at one-shot without retraining. ICML 2023. arXiv: exttt2301.00774
- **STUN** (Guo et al., 2025): structural-to-unstructured sparsity transition; ACL 2025.
- **Samoyeds** (EuroSys 2025): N:M sparsity with MoE; hybrid hardware path.
- **NVIDIA Ampere 2:4**: hardware-accelerated 50% weight sparsity; $\leq 1\%$ PPL degradation at 2:4 in bf16 inference.
- **MST**: multi-sparsity training baseline combining N:M and MoE within one training run.

Missing from draft: SparseGPT [177] (quality baseline for N:M sparsity); Switch Transformer (MoE quality baseline); SPOS (mode-collapse prevention via single-path sampling). *Prior art overlap: partial ($\sim 60\%$).*

Complexity Analysis

Table 9: *Complexity impact of Idea 1.5 (Learned Sparsity Type Selection).*

Baseline	TPOT change	Notes
A1	$\downarrow 1.5\text{--}2\times$ (ideal)	If 2:4 sparsity replaces dense MLP layers
A2	$\downarrow 1.5\text{--}2\times$	Similar; joint infrastructure cost
B	marginal	Already sparse; 2:4 on expert weights possible additional gain

Gate overhead: 8 FLOP/byte at the MoE-vs-2:4 crossover; gate itself adds $\sim 0.03\%$ FLOP overhead per layer.

Verdict

candidate for investigation. Merge with Idea 1.4 into a *Unified 3-Way Layer-Type Architecture Search* (dense / 2:4 sparse / MoE). No published LLM-scale experiments for training-time learned sparsity-type selection. Phase 3 work; high risk.

Confidence scores: *Overall* medium (no numerical score from review)

Full analysis: research_1_5_learned_sparsity_type.md in the research corpus.

3.12 Idea 1.6: Learned Layer Type (4-Primitive NAS)

Mechanism

Extends 2-primitive hybrid designs (attention + SSM or attention + linear attention, e.g. Qwen3.5, Jamba, Nemotron-H) to a 4-primitive architecture search space:

$$\mathcal{P} = \{\text{full attention, SSM/Mamba-2, causal convolution, linear attention}\}$$

A DARTS-style supernet trains all four layer types in parallel at each depth position; a Gumbel-Softmax or straight-through estimator selects the active primitive. After search, the architecture is distilled to a fixed-type hybrid model and retrained.

Nemotron-H (92% Mamba-2 + 8% attention) validates that 2-primitive hybrids match or exceed dense Transformer quality at $3\times$ throughput at 65K context length. The 4-primitive extension is uncharted at 7B+. Earlier draft TTFT “ $\downarrow \sim 2\text{--}3\times$ ” was corrected by review: actual improvement at 8K context for A2 is $\sim 1.05\text{--}1.10\times$ because TTFT is compute-bound, not bandwidth-bound; the large TPOT improvement ($\sim 2\text{--}4\times$ at 32K context) applies to bandwidth-bound decode, not prefill. DUET (arXiv: exttt2603.15530) is post-cutoff and was not flagged as such in the original draft—correction noted.

Prior Art and Related Work

- **Mamba (Gu & Dao, ICLR 2024)** [190]: structured SSM with selective state spaces; hardware-aware recurrent scan. arXiv: exttt2312.00752
- **Mamba-2 (Dao & Gu, ICML 2024)** [34]: structured state-space duality (SSD); group-value head; hardware-efficient. arXiv: exttt2405.21060
- **Jamba** [112]: Mamba + attention + MoE hybrid; 52B scale; ICLR 2025. arXiv: exttt2403.19887
- **Nemotron-H (NVIDIA 2025)**: $3\times$ throughput at 65K context; 92% Mamba-2 layers; “better or on-par accuracy.”
- **Hymba (Dong/NVIDIA, ICLR 2025)** [70]: SSM + attention parallel; +1.32% accuracy over Transformer baselines.
- **Zamba2** [80]: SSM-attention hybrid for long-context efficiency.
- **MAD (Poli et al., ICML 2024)** [126]: mechanistic architecture design; tests primitive composability.
- **Hybrid Architecture Review** [202]: arXiv: exttt2510.04800—empirical study of 2-primitive hybrids.
- **Composer** [30]: arXiv: exttt2510.00379—4-primitive NAS at small scale; directly relevant precedent.
- **DARTS** [117]: differentiable NAS backbone.

- **TransMamba (AAAI 2026)** [306]: attention–Mamba hybrid with differentiable switching.
- **BASED (Arora et al., ICLR 2024)** [54]: linear attention + sliding-window convolution hybrid. arXiv: exttt2402.18668

Missing from draft: Samba (3-primitive: SSM + attention + MLP), MoD per-layer routing, Griffin.
Prior art overlap: partial (~65%).

Complexity Analysis

Table 10: *Complexity impact of Idea 1.6 (Learned Layer Type, 4-primitive NAS).*

Baseline	TTFT change	TPOT change	Notes
A1	$\approx \times 1$	$\downarrow 1.1\text{--}1.3\times$	Already 75% non-attention; 4-primitive refines the 25% full-attn share
A2	$\approx 1.05\text{--}1.10\times$	$\downarrow 2\text{--}4\times$ at 32K ctx	Largest opportunity: 64 dense attention layers; TPOT bandwidth-bound
B	$\approx \times 1$	$\downarrow 1.0\text{--}1.2\times$	Already 75% DeltaNet; marginal gain

Verdict

candidate for investigation. Phase 3 idea: requires pretraining from scratch at 1–3B scale before committing to 27B+. The 2-primitive literature is mature; the 4-primitive extension is the novel contribution. Recommend starting with Composer-style NAS at 1B scale to validate the search space before scaling.

Confidence scores: *Literature coverage* 7/10 *Technical soundness* 6/10 *Baseline comparison* 6/10
Overall 6/10

Full analysis: research_1_6_learned_layer_type.md in the research corpus.

3.13 Idea 1.7: Dynamic Vocabulary Subsetting

Mechanism

At each decode step, instead of computing logits over the full vocabulary V (248,320 for Baselines A1 and B; 151,936 for A2), a lightweight context-conditioned predictor selects an active subset $V' \subset V$ of size $|V'| \ll |V|$ before the LM-head matrix multiplication. The LM-head weight matrix $W_{\text{out}} (V \times d)$ is indexed by V' , yielding a reduced GEMM of dimension $|V'| \times d$ instead of $V \times d$.

Critical corrected quantities (from review):

- A1/B vocab is $V = 248,320$ (not 151,936 as in earlier drafts using A2’s vocab size).
- W_{out} for A1: $248,320 \times 5120 \times 2 \text{ bytes} \approx 2.55 \text{ GB bf16}$ ($\approx 4.7\%$ of 54 GB total weight).
- W_{out} for B: $248,320 \times 4096 \times 2 \text{ bytes} \approx 2.04 \text{ GB bf16}$.

Standalone TPOT improvements (corrected): A1 $\sim 4.7\%$; B $\sim 6\%$ (not $\sim 2\%$ and $\sim 1\text{--}2\%$ as stated in the earlier draft). A2 standalone: $\sim 2.3\%$ (LM head $\approx 2.4\%$ of total bandwidth).

Critical failure mode: if V' excludes the highest-probability next token, the model cannot generate it. CSV-Decode [91] demonstrates a fallback mechanism guaranteeing exact top- k with $< 2\%$ fallback rate and 99.3% quality retention.

The highest-value deployment is as a draft-model optimisation in speculative decoding, where the LM head is a proportionally larger fraction of the total model size.

Prior Art and Related Work

- **Adaptive Softmax** (Grave et al., ICML 2017) [84]: frequency-stratified output softmax; 43.9 PPL on One Billion Word; open-sourced as `nn.AdaptiveLogSoftmaxWithLoss`. arXiv: exttt1609.04309
- **Baevski & Auli (2019)** [7]: adaptive input representations + adaptive softmax; 18.7 PPL on WikiText-103 (SOTA at time). arXiv: exttt1809.10853
- **D-Softmax** (Chen & Grangier, 2016) [25]: differentiated softmax with frequency-based cluster projection.
- **FR-Spec** (Zhao et al., 2025) [156]: frequency-regularised speculative decoding; $1.12\times$ speedup as draft-model optimisation.
- **DynaSpec** (2025): context-sensitive draft vocabulary; 98.4% of full-vocabulary mean accepted length.
- **CSV-Decode** (He et al., 2024/2025) [91]: context-sensitive vocabulary; exact top- k guarantee; $< 2\%$ fallback rate; 99.3% quality retention.
- **VocabTrim** (Goel et al., 2025) [81]: static frequency-based vocabulary pruning; near-lossless at top-40K.
- **SpecVocab** (2026): dynamic speculative vocabulary selection.
- **Balancing Coverage** (Ben Shoham, 2026): coverage-latency trade-off analysis for dynamic vocabulary in speculative decoding.
- **Holtzman et al. (2020, ICLR)** [93]: nucleus sampling; top- p dynamic vocabulary subset for generation quality—*missing from original draft; identified by review as directly relevant*.
- **Willard & Shin (2023)** [325]: grammar-constrained generation (structured vocabulary masking)—*missing from original draft*.

Prior art overlap: partial ($\sim 65\%$); static-frequency variant EXISTS.

Complexity Analysis

Table 11: *Complexity impact of Idea 1.7 (Dynamic Vocabulary Subsetting).*

Baseline	TPOT change	Notes
A1	↓ ~4.7%	$W_{\text{out}} = 2.55 \text{ GB} / 54 \text{ GB total}$
A2	↓ ~2.3%	$W_{\text{out}} = 1.56 \text{ GB} / 64 \text{ GB total}$
B	↓ ~6%	$W_{\text{out}} = 2.04 \text{ GB}$; active weights ~34 GB

TTFT: ~1% improvement (LM head is a small fraction of total prefill compute).

Verdict

candidate for investigation (stronger). Merge with Ideas 2.1 and 4.7 as a unified vocabulary compression investigation targeting the speculative decoding draft-model path. Standalone TPOT benefit is modest (4.7% for A1, 6% for B) but the draft-model path amplifies impact significantly.

Confidence scores: *Literature coverage* ~7/10 *Technical soundness* ~7/10 *Overall* ~7/10

Full analysis: research_1_7_dynamic_vocabulary.md in the research corpus.

3.14 Idea 2.1: Hierarchical Frequency-Based Dictionary

Mechanism

Stratify the output vocabulary into frequency tiers (e.g. top-10K high-frequency tokens, plus sparser long-tail lookup clusters), computing logits only over the frequent tier by default and deferring to less-frequent clusters on demand via a lightweight cluster-head softmax. The mechanism is mechanistically identical to *adaptive softmax* [84], which is already open-sourced as `PyTorch.nn.AdaptiveLogSoftmaxWithLoss`.

The idea’s proposed novelty—a trainable `<next_dict>` token to signal tier transitions—is demonstrably inferior to adaptive softmax: adaptive softmax achieves the same hierarchical lookup without consuming a vocabulary slot or adding an extra generation step.

Standalone TPOT benefit: ~2% for A2 (LM head $\approx 2.4\%$ of 64 GB total weight bandwidth). For A1 with corrected $V = 248,320$: ~4.7%.

Prior Art and Related Work

- **Adaptive Softmax (Grave et al., ICML 2017)** [84]: 43.9 PPL on One Billion Word; open-sourced. arXiv: exttt1609.04309
- **Hierarchical Softmax (Morin & Bengio, 2005)** [259]: binary tree-structured output softmax; foundational.

- **Noise-Contrastive Estimation (Mnih & Hinton, 2008)** [255]: approximate softmax for large vocabularies.
- **Mikolov et al. (2011)** [251]: hierarchical softmax in RNN language models; frequency-based tree construction.
- **D-Softmax (Chen & Grangier, 2016)** [25]: differentiated softmax with cluster-based frequency stratification.
- **Baevski & Auli (2019)** [7]: adaptive input + adaptive softmax; 18.7 PPL on WikiText-103. arXiv: exttt1809.10853
- **Learning to Screen (Chen et al., ICLR 2019)** [61]: vocabulary screening for LM output; top- k candidate selection before full softmax.
- **FR-Spec** [156]: frequency-regularised speculative decoding; $1.12\times$ speedup.
- **VocabTrim (Goel et al., 2025)** [81]: static frequency-based vocabulary pruning.
- **Vocab Frequency Imbalance (Chung & Kim, NeurIPS 2025)** [28]: frequency imbalance analysis confirming frequency stratification validity.
- **Coverage vs. Draft Latency (Ben Shoham, 2026)**: coverage–latency trade-off.

All 11 cited papers verified as real. *Prior art overlap: exists ($\sim 90\%$).*

Complexity Analysis

Table 12: *Complexity impact of Idea 2.1 (Hierarchical Frequency-Based Dictionary).*

Baseline	TPOT change	Notes
A1	$\downarrow \sim 4.7\%$	Corrected vocab $V = 248,320$; LM head $\approx 4.7\%$ of total weight
A2	$\downarrow \sim 2\%$	$V = 151,936$; LM head $\approx 2.4\%$ of 64 GB
B	$\downarrow \sim 6\%$	Corrected vocab $V = 248,320$; active weights ~ 34 GB

Verdict

lower-priority. Core mechanism EXISTS as open-source PyTorch primitive. $\sim 2\text{--}6\%$ TPOT gain is not actionable as a standalone investigation. Merge into unified vocabulary compression study with Ideas 1.7 and 4.7. 1–2 engineer-days to deploy PyTorch adaptive softmax as a drop-in.

Confidence scores: *Overall* 8.6/10

Full analysis: research_2_1_hierarchical_frequency_dictionary.md in the research corpus.

3.15 Idea 2.2: Compressed Dense Layers via Low-Rank Factorisation

Mechanism

Each FFN weight matrix $W \in \mathbb{R}^{d \times d_{\text{ff}}}$ is replaced by a product of two smaller matrices $U \in \mathbb{R}^{d \times r}$ and $V \in \mathbb{R}^{r \times d_{\text{ff}}}$ where $r \ll \min(d, d_{\text{ff}})$. At inference the two GEMMs cost $O(d \cdot r + r \cdot d_{\text{ff}})$ vs. $O(d \cdot d_{\text{ff}})$, a ratio of $2r/(d + d_{\text{ff}})$. For A2 ($d = 5120$, $d_{\text{ff}} = 25600$), compression to $r = 0.3 \cdot d_{\text{ff}} = 7680$ yields a $\sim 40\%$ TPOT reduction on the MLP bandwidth term.

Optionally, a sparse residual correction R is added: $W \approx U \cdot V + R$, where R is stored in COO/CSR format (CALDERA [59] uses this to recover quality at sub-3-bit regimes). The **cliff behaviour** is the primary risk: above $\sim 80\%$ parameter retention, quality loss is near-zero; below $\sim 60\%$ retention, PPL increases steeply.

ASVD correction from review: PPL $5.68 \rightarrow 5.78$ represents $\sim 1.8\%$ compression, not 20% as claimed in an earlier draft—the 20% figure was from a different evaluation setting. 2:4 sparsity on the residual R adds $7.7\times$ more bandwidth unless R is stored in $<1\%$ density COO/CSR format.

Prior Art and Related Work

- **LoRA (Hu et al., ICLR 2022) [94]**: *missing from earlier draft—identified by review as critical gap*; low-rank adaptation; standard baseline for all low-rank LLM work. arXiv: exttt2106.09685
- **ALBERT (Lan et al., ICLR 2020) [106]**: *missing from earlier draft—identified by review as critical gap*; cross-layer parameter sharing + embedding factorisation; -1.5 to -2.5 GLUE points for full sharing.
- **SVD-LLM (Wang et al., ICLR 2025) [138]**: truncated SVD on LLM weights; post-hoc, no retraining required. arXiv: exttt2403.07378
- **ASVD (Yuan et al., 2023) [150]**: activation-aware SVD; 10–30% compression without performance drop (PPL $5.68 \rightarrow 5.78$ at $\sim 5\%$ compression; the 20% compression claim in earlier draft was incorrect). arXiv: exttt2312.05821
- **CALDERA (Bhatt et al., NeurIPS 2024) [59]**: $W \approx Q + L \cdot R$ decomposition; outperforms all post-training compression at <2.5 bits/param. arXiv: exttt2405.18886
- **Wei et al. (NeurIPS 2024) [140]**: training-native low-rank; $1.35\times$ training speedup.
- **LoftQ (Li et al., 2023) [110]**: quantised + low-rank fine-tuning initialisation; better than QLoRA at 2-bit/mixed regimes. arXiv: exttt2310.08659
- **Monarch (Dao et al., 2022) [67]**: butterfly factorisation; $2\times$ training speedup on GPT-2. arXiv: exttt2204.00595
- **BLAST (Lee et al., 2024) [107]**: block-level adaptive structured matrices; $2\times$ compression with lowest degradation in class. arXiv: exttt2410.21262
- **SLTrain (Han et al., NeurIPS 2024) [88]**: sparse + low-rank training combination.

- **Nuclear Norm (Scarvelis et al., NeurIPS 2024) [130]:** nuclear norm minimisation for low-rank regularisation during training.

Prior art overlap: partial ($\sim 70\%$).

Complexity Analysis

Table 13: *Complexity impact of Idea 2.2 (Compressed Dense Layers, Low-Rank).*

Baseline	TPOT change	Notes
A1	$\downarrow 1.3\text{--}2.0\times$	Dense MLP layers; at $r = 0.3d_{\text{ff}}$, $\sim 3\times$ MLP bandwidth reduction; DeltaNet layers unaffected
A2	$\downarrow 4\text{--}6\times$ (MLP only)	At 32K ctx, weight bandwidth dominates KV; MLP is primary TPOT lever
B	$\downarrow 1.5\text{--}2.0\times$	Active expert weight bandwidth reduced; 397B stored at lower memory footprint

TTFT at $r = 256$: practical speedup $\sim 50\text{--}75\%$ of theoretical (overhead from two GEMMs vs. one, plus smaller FLOP-to-byte ratio).

Verdict

high-priority. Post-hoc SVD-LLM immediately deployable on Baseline A2 (no retraining). Training-native variant (Wei et al. 2024 style) is a Phase 2 task. Conservative compression ($>80\%$ parameter retention) is near-lossless.

Confidence scores: *Technical soundness* strong *Overall*: high confidence (high-priority confirmed)

Full analysis: `research_2_2_compressed_dense_layers.md` in the research corpus.

3.16 Idea 3.1: Layer-Level MoE (Full-Block Routing)

Mechanism

Instead of routing tokens to individual FFN sub-networks within a layer (standard MoE), route tokens to *entire transformer blocks* (attention + FFN), so that k of E full-block experts are selected per token per forward pass. TTFT improvement: $k/E \times$ full-model TTFT at prefill (only k/E fraction of expert weights loaded). TPOT: ambiguous—weight savings (k/E fraction loaded) vs. k -fold KV cache inflation (each block-expert maintains its own KV cache for causal attention).

For $k = 2$, $E = 8$: KV cache is $8\times$ larger than standard single-layer attention, because each of 8 full-block experts maintains a separate KV history. Alternatively, block-level routing could share a single KV cache but then each expert can only attend to $\sim 25\%$ of prior tokens (the tokens that were routed to it)—a novel failure mode documented in the review.

Critical correction from review: weight storage was underestimated $3.4\times$ in earlier draft because attention projection matrices (W_Q, W_K, W_V, W_O) were omitted; the full-block expert includes both attention and FFN parameters.

Prior Art and Related Work

- **Mixture of Depths (MoD, Raposo et al., ICML 2024)** [128]: per-token layer bypass (not block routing); validates variable-depth computation. arXiv: exttt2404.02258
- **SwitchHead (Csordás et al., NeurIPS 2024)** [65]: attention head MoE—heads rather than full blocks; closest published antecedent.
- **MoA (Zhang et al., EMNLP 2022)** [152]: Mixture of Attention; multiple attention modules per layer.
- **MoH (Jin et al., 2025)** [101]: Mixture of Heads.
- **Expert Choice (Zhou et al., NeurIPS 2022)** [158]: expert-side routing; $>2\times$ training efficiency; foundational MoE reference. arXiv: exttt2202.09368
- **DeepSeek-V3** [37]: large-scale MoE; 512 experts (standard FFN-level MoE); provides the baseline against which full-block routing is compared.
- **Shazeer et al. (2017)** [286]: original sparsely-gated MoE; missing from earlier draft (identified by review as critical omission).
- **Switch Transformer (Fedus et al., JMLR 2022)** [74]: missing from earlier draft; $k = 1$ routing; established MoE training stability baseline.
- **Mixtral-8x7B** [254]: FFN-level MoE baseline.
- **MoNE (Jain et al., NeurIPS 2024)** [96]: mixture of nerves—per-layer expert neuron allocation; adjacent concept.
- **SliceMoE (2025, EMNLP)** [289]: per-layer sliced expert assignment; partial block routing.

Prior art overlap: partial ($\sim 55\%$).

Complexity Analysis

KV cache: TPOT crossover at $\sim 334\text{K}$ tokens (where KV savings from k/E fraction dominate). Below that context length: KV inflation makes full-block routing worse than standard MoE.

Verdict

candidate for investigation (medium-high priority, but KV inflation is a production blocker at $<334\text{K}$ context). The sparse-KV failure mode is novel and must be characterised before committing

Table 14: *Complexity impact of Idea 3.1 (Layer-Level MoE, Full-Block Routing).*

Baseline	TPOT change	Notes
A1	↓ or ↑ depending on KV strategy	KV inflation blocks; weight bandwidth savings partially offset
A2	↓ k/E (weight) but ↑ E (KV)	Without KV solution: KV cache $E \times$ larger; net negative
B	N/A	Already FFN-level MoE; full-block routing would add attention duplication

engineering resources. No published experiments for full-block routing exist; validate at 1B scale first.

Confidence scores: *Overall*: medium (significant unresolved issues)

Full analysis: research_3_1_layer_level_moe.md in the research corpus.

3.17 Idea 3.2: Hot-Swappable Expert Weights

Mechanism

Enable post-deployment insertion, removal, and replacement of MoE experts without full retraining, by maintaining a growing expert pool and a recalibration-friendly router architecture. Expert insertion: train a new expert via LoRA fine-tuning on domain-specific data ($\sim 0.01\text{--}0.1 \times$ full training cost); extend the router’s projection matrix by 1 column. Expert removal: set routing weights for the target expert to zero and run router recalibration. Expert swap: swap one expert’s weights for another’s; router recalibration required.

Corrected swap size from review: ~ 1.5 GB per expert (not ~ 240 MB as in earlier draft—the discrepancy arose from confusing $d_e = d$ with $d_e \ll d$; Baseline B uses $d_e \approx 1024$, giving two matrices of $4096 \times 1024 \times 2$ bytes = 8.4 MB each, but the full expert library correction yields ~ 7.7 TB total, not ~ 1.2 TB as earlier stated).

No TPOT improvement: expert swapping does not reduce the inference bandwidth path. Value proposition: deployment agility—fine-grained domain capability (coding, maths, legal) can be added/removed without retraining the full model.

Prior Art and Related Work

- **Branch-Train-Merge (BTX)** [17]: expert specialisation via separate branch training and merging; baseline architecture for expert library.
- **Sukhbaatar et al. (2024)**: expert insertion + router fine-tuning; +18.8 pts math, +13.2 pts coding.
- **REAP (Cerebras, 2025)**: expert pruning at deployment; near-lossless at 50% pruning on code generation; 0.14 accuracy drop at 25% pruning.

- **LoRAMoE (ACL 2024)** [244]: LoRA-based expert composition—*missing from original draft; identified by review as closing the router-retraining gap.*
- **PHATGOOSE**: post-hoc expert assembly without router retraining—*missing from original draft; identified by review.*
- **DSMoE**: domain-specific MoE specialisation.
- **Expert merging (Muqeeth et al., 2024)** [123]: arXiv:2402.00433.
- **Sparse Upcycling** [105]: converting dense models to MoE by upcycling; baseline for expert initialisation.
- **What Gets Activated (arXiv:2601.10159)** [323]: expert activation patterns at deployment time.

Citation issues from review: 3 FAIL (anonymous keys, year error for arXiv:2601.10159). *Prior art overlap: partial; individual mechanisms exist; no end-to-end hot-swap system demonstrated.*

Complexity Analysis

Table 15: *Complexity impact of Idea 3.2 (Hot-Swappable Expert Weights).*

Baseline	TPOT change	Notes
A1	$\approx \times 1$	Dense MLP; expert swap not applicable
A2	$\approx \times 1$	Dense; not applicable
B	$\approx \times 1$	No TPOT improvement; swap adds router recalibration overhead ($\sim 0.01 \times$ training)

Verdict

candidate for investigation (operational idea, not inference efficiency idea). Deploy as a deployment agility feature alongside other ideas. Requires: LoRA expert training infrastructure + router recalibration tooling + hot-swap index management.

Confidence scores: *Overall*: medium (clear mechanism, missing engineering infrastructure)

Full analysis: research_3_2_swappable_experts.md in the research corpus.

3.18 Idea 3.3: Dynamic Expert Router

Mechanism

Replace the fixed per-layer linear router with a router that adapts its weights in response to deployment-time signals: domain shift, new expert pool membership, or performance feedback. A

lightweight router recalibration procedure (estimated $\sim 0.01\text{--}0.1\times$ full training cost) re-optimises routing weights given a new expert set or distribution shift. The router can incorporate context-aware signals (prompt prefix embedding, document-type classifier output) beyond pure hidden-state routing.

Critical review correction: the “cold-start gradient barrier” is a *strict* zero-gradient condition (router output layer logits for newly added expert are zero until the first calibration step), not a gradual warm-up as implied in the original draft. Load balancing is entirely absent from the original research document—a critical gap for production MoE deployment. Recalibration cost was overestimated $10\text{--}1000\times$ in the original draft.

Prior Art and Related Work

- **Shazeer et al. (2017)** [286]: sparsely-gated MoE; token-choice routing; foundational—*missing from original draft, identified as critical omission*.
- **Expert Choice (Zhou et al., NeurIPS 2022)** [158]: expert-side routing; $>2\times$ training efficiency.
- **MoEfication (Zhang et al., 2022)** [153]: foundational MoE conversion; missing from original draft. arXiv: exttt2110.01786
- **Sparse Upcycling (ICLR 2023)** [105]: dense-to-MoE router initialisation—*missing from original draft, direct precedent for cold-start*.
- **Branch-Train-Merge** [17]: direct precedent for router recalibration after expert addition—*missing from original draft*.
- **HyperRouter (Do et al., EMNLP 2023)** [69]: hypernetwork-based adaptive routing; meta-learns routing across task distributions.
- **ReMoE (Wang et al., ICLR 2025)** [139]: differentiable routing baseline.
- **REAP (Cerebras 2025)**: expert pruning with router recalibration; validates near-lossless pruning at 50%.
- **Router KD (arXiv:2603.02217)** [281]: router knowledge distillation for efficient recalibration.
- **MoKE/ARM (Cheng et al., ACL 2025)** [62]: adaptive mixture of routing experts.
- **LEMoe (Wang & Li, EMNLP 2024)** [315]: layer-ensemble MoE with adaptive routing.
- **MoEEdit (2026)** [256]: online expert editing with router adaptation.

Review flagged: 13/17 citations non-compliant (incomplete author lists, unconfirmed venues, anonymous keys). *Prior art overlap: partial ($\sim 60\text{--}65\%$).*

Table 16: *Complexity impact of Idea 3.3 (Dynamic Expert Router).*

Baseline	TPOT change	Notes
A1	$\approx \times 1$	Dense; not applicable
A2	$\approx \times 1$	Dense; not applicable
B	$\approx \times 1$	Router recalibration is a one-time offline cost; no per-token inference change

Complexity Analysis

Verdict

candidate for investigation. Operational idea with clear deployment value. Primary research question: does context-aware routing (beyond hidden state) improve routing quality without destabilising load balancing? Must fix all citation errors and add load balancing analysis before committing engineering resources.

Confidence scores: *Literature coverage* 5/10 *Technical soundness* 6/10 *Baseline comparison* 7/10
Overall 6/10

Full analysis: `research_3_3_dynamic_expert_router.md` in the research corpus.

3.19 Idea 3.4: Recursive Internal State (Implicit Reasoning Loop)

Mechanism

Allow the model to iterate its forward pass T times over a fixed set of *middle* layers before emitting the output token, enabling implicit “thinking” without explicit chain-of-thought tokens. The architecture is partitioned into: static *prelude* (first K layers), *recurrent zone* (middle L_{mid} layers, executed T times), and static *coda* (last K layers for output projection).

TPOT increases as:

$$\text{TPOT}_T = (1 + (T - 1) \cdot f_{\text{mid}}) \cdot \text{TPOT}_1$$

where f_{mid} is the fraction of per-token FLOPs in the recurrent zone. Arithmetic correction from review: at $T = 4$, $f_{\text{mid}} = 0.5$, the formula gives $1 + 3 \times 0.5 = 2.5\times$ TPOT increase (earlier drafts stated $1.5\times$ —incorrect).

Dynamic T via Adaptive Computation Time (ACT) [184] halting: a ponder network decides when the representation has converged, reducing average T below the maximum. The “overthinking” pathology documented by Kohli et al. (2026): excessive recurrence degrades quality— T_{opt} exists and must be found empirically.

Prior Art and Related Work

- **ACT (Graves, 2016)** [184]: Adaptive Computation Time; learned halting in RNNs; foundational. arXiv: exttt1603.08983

- **Universal Transformers** (Dehghani et al., 2019) [68]: T -step recurrent attention; +0.9 BLEU on WMT14. arXiv: exttt1807.03819
- **PonderNet** (Banino et al., ICML Workshop 2021) [57]: probabilistic halting mechanism; cleaner gradient flow than ACT.
- **Huginn-3.5B** (Geiping et al., 2025) [78]: $r = 32$ recurrent iterations with $(2, 4, 2)$ layer structure; matches ~ 50 B-class models on reasoning benchmarks; GSM8K 3.11% (no supervision, $r = 4$) vs. 24.87% (explicit CoT). arXiv: exttt2501.17157
- **Looped Transformers** (Saunshi et al., ICLR 2025) [129]: theoretical analysis of depth-recurrence equivalence.
- **Ouro** (Zhu et al., 2025) [159]: ouroboros recurrent architecture; validates looped computation at moderate scale.
- **COCONUT** (Hao et al., COLM 2025) [89]: continuous thought tokens (latent chain-of-thought); adjacent mechanism.
- **MoR** (Bae et al., NeurIPS 2025) [55]: Mixture of Recursions; combines early exit with recurrence.
- **AdaPonderLM** (Song et al., 2026) [133]: adaptive pondering in language models.
- **Two-Scale Latent Dynamics** (Pappone et al., NeurIPS 2025) [124]: slow/fast dynamics in recurrent LLMs.
- **SpiralFormer** (Yu et al., 2026) [149]: spiral recurrent architecture—citation flagged by review as non-compliant (no authors in original draft).
- **Kohli et al. (2026)** [104]: “overthinking” pathology; T_{opt} documented.

Prior art overlap: exists (Huginn, Ouro demonstrate the loop mechanism).

Complexity Analysis

Table 17: *Complexity impact of Idea 3.4 (Recursive Internal State). TPOT increases with T ; this is a quality-improvement, not efficiency, idea.*

Baseline	TPOT change	Notes
A1	$\uparrow 1.5\times$ at $T=2$, $\bar{f}_{\text{mid}}=0.5$	Efficiency-negative; quality gain depends on task
A2	$\uparrow 2.5\times$ at $T=4$, $f_{\text{mid}}=0.5$	Arithmetic correction applied
B	$\uparrow \sim 2.5\times$	Same formula; MoE layer recurrence adds router overhead per pass

Verdict

candidate for investigation (hybrid: standalone lower-priority for inference efficiency; candidate for investigation when combined with early exit, Idea 1.2). TPOT increases by up to $2.5\times$ at $T = 4$ —efficiency-negative as a standalone. Quality improvement (reasoning, planning) may justify the cost only for inference-compute-rich workloads. Hybrid with 1.2: early exit reduces average T ; net TPOT may be manageable.

Confidence scores: *Literature coverage* 8/10 *Technical soundness* 7/10 *Baseline comparison* 8/10
Overall 7/10

Full analysis: research_3_4_recursive_internal_state.md in the research corpus.

3.20 Idea 3.5: Gated Internal DAG

Mechanism

Replace the linear chain of transformer layers with a directed acyclic graph (DAG) of computation nodes (attention or FFN sub-blocks), where per-token routing gates determine which nodes execute. The effective compute depth equals the *critical path* through the active DAG nodes, which may be shorter than the full node count if branches are taken. TPOT improvement requires that the average critical path $<$ total node count.

Fundamental bottleneck: at batch-size=1 (single-sequence decode), the DAG critical path is a serial bottleneck. If any gate along the critical path fires, the latency is bounded below by that path’s depth—and aggressive gating may make the DAG’s critical path as long as the linear chain. At $d_{\text{dag}} \geq 4\text{--}5$ parallel branches, gate overhead cancels expected savings.

Prior Art and Related Work

- **Highway Networks** (Srivastava et al., 2015) [134]: learned bypass gates; foundational.
- **SkipNet** (Wang et al., 2018) [137]: layer-skipping gates trained with RL.
- **BlockDrop** (Wu et al., CVPR 2018) [143]: block dropping for residual networks.
- **DARTS** [117]: differentiable NAS for the gate-learning problem.
- **Mixture of Depths (MoD)** [128]: token-level layer bypass—closest validated analog.
- **Conditional Computation** (Bengio et al., 2015) [58]: foundational conditional compute theory.
- **D2NN** (AAAI 2018): dynamic 2D neural network; per-token routing in a 2D layer grid.
- **Deja Vu** (Liu et al., ICML 2023) [113]: contextual sparsity; identifies active neurons before computation—*missing from original draft; identified by review as critical omission*.
- **LayerSkip** [72]: layer-level skipping; self-speculative decoding.

- **MegaBlocks** [75]: block-sparse expert computation—*missing from original draft*.
- **ACM (AAAI 2025)**: adaptive computation masking.
- **GateSkip** [102]: gate-based layer skipping; validated on small LMs.

Missing from original draft: Deja Vu [113] (ICML 2023), ACT [184], PowerInfer (Song 2023), LayerSkip [72], MegaBlocks. Overlap revised to $\sim 70\text{--}75\%$ after review. *Prior art overlap: partial ($\sim 70\text{--}75\%$).*

Complexity Analysis

Table 18: *Complexity impact of Idea 3.5 (Gated Internal DAG).*

Baseline	TPOT change	Notes
A1	$\downarrow$ up to $1.33\times$ (theoretical)	Gate overhead crossover at $d_{\text{dag}} \approx 4\text{--}5$; realistic gain $<10\%$
A2	$\downarrow$ up to $1.33\times$ (theoretical)	Same; no published LLM-scale results
B	marginal	MoE $k/E = 2.1\%$ vs. $p_{\text{active}} = 30\text{--}50\%$ in DAG—gap unexplained

Verdict

candidate for investigation (medium priority, subsumed into 3.6 if 3.6 proceeds). No published LLM-scale results. Sequential DAG critical path is a fundamental latency obstacle at batch=1. Recommend: first characterise gate-overhead crossover empirically, then decide whether to pursue full DAG or fall back to MoD-style bypass.

Confidence scores: *Overall*: medium

Full analysis: research_3_5_gated_internal_dag.md in the research corpus.

3.21 Idea 3.6: Recursive Internal DAG

Mechanism

Combines the recurrent iteration of Idea 3.4 with the DAG routing of Idea 3.5 and an external persistent state, creating a model that both iterates *and* routes dynamically within each decode step. Three jointly-trained components: (1) T recurrent iterations over a DAG-structured middle zone; (2) per-iteration token routing through DAG branches; (3) a persistent external state (key-value memory or external vector) updated across iterations and across tokens.

TPOT increases by $\sim 2.5\times$ at $T = 4$ (arithmetic correction from review: $1 + 3 \times 0.5 = 2.5\times$, not $2\times$). KV bandwidth: a missing term from the original draft is $T \cdot L_{\text{mid}} \cdot s \cdot d_{kv}$ for the recurrent KV materialisation across T iterations.

“Turing-complete” characterisation in earlier drafts is overstated: the system is *bounded-Turing-complete* (computation is bounded by $T_{\max}$ and L). KV cache vs. A1 arrow corrected: should be $\uparrow$ (increases) not $=$ (as earlier drafts stated).

Three anonymous citation entries in the original research document require author/venue resolution before publication.

Prior Art and Related Work

- **NTM (Graves et al., 2014) [85]**: Neural Turing Machine; external differentiable memory; foundational.
- **DNC (Graves et al., Nature 2016) [86]**: Differentiable Neural Computer; loop + memory.
- **ACT [184]**: halting mechanism for the loop component.
- **Universal Transformers [68]**: recurrent transformer.
- **Pérez et al. (ICLR 2019) [125]**: Turing completeness of transformers with hard attention; theoretical bound.
- **Huggins [78]**: loop component validated at 3.5B; directly relevant. arXiv: exttt2501.17157
- **Ouro (Zhu et al., 2025) [159]**: ouroboros recurrence; loop mechanism.
- **Giannou et al. (ICML 2023) [79]**: looped transformer as programmer; complexity analysis.
- **Cyclic NN (Yang et al., NeurIPS 2024) [145]**: cyclic connections + DAG routing combined; closest published antecedent.
- **SpiralFormer (Yu et al., 2026) [149]**: spiral recurrent + branching.
- **LoopFormer (Jeddi et al., ICLR 2026) [98]**: looped transformer at ICLR 2026.
- **TM Simulation (Li & Wang, ICLR 2026) [242]**: Turing machine simulation via transformer loops.
- **ANIRA (Moosa et al., 2026) [122]**: adaptive nested iteration with routing.

Novel gap: the DAG + persistent state *combination* has no published experiments; loop-only (Huggins, Ouro) is validated. *Prior art overlap: partial.*

Complexity Analysis

Verdict

candidate for investigation (but TPOT is efficiency-negative; only justified for quality-premium workloads). Recommend splitting into: (a) loop-only validation (already done in Huggins), and (b) DAG routing on top of loop (novel, Phase 3). Persistent state integration is the hardest subproblem; requires new training infrastructure.

Table 19: Complexity impact of Idea 3.6 (Recursive Internal DAG). TPOT increases; KV cache increases.

Baseline	TPOT change	KV change	Notes
A1	$\uparrow \sim 2.5\times$ at $T=4$	$\uparrow$	Recurrent KV materialisation; $T \cdot L_{\text{mid}} \cdot s \cdot d_{kv}$ bandwidth term
A2	$\uparrow \sim 2.5\times$ at $T=4$	$\uparrow$	Same
B	$\uparrow \sim 2.5\times$	$\uparrow$	MoE dispatch repeated T times per decode step

Confidence scores: *Overall*: medium-low (TPOT increases; high implementation complexity)

Full analysis: `research_3_6_recursive_internal_dag.md` in the research corpus.

3.22 Idea 3.7: Learnable State Machine (FSM Block Router)

Mechanism

Maintain an N -state discrete finite state machine (FSM) alongside the main transformer whose state $s_t \in \{1, \dots, N\}$ evolves across the token sequence via a learned transition matrix $A \in \mathbb{R}^{N \times N}$. The FSM state gates which computation blocks execute at each position: state 1 might activate “reasoning blocks,” state 2 “copy blocks,” etc. In theory, this allows the model to learn discourse-level “modes” and activate only mode-relevant computation.

TPOT speedup formula (verified by review): $C_{\text{avg}} = (1 - p(1 - r)) \times C_{\text{baseline}}$, where p is the fraction of tokens in active-computation states and r is the activation ratio within those states. FSM overhead: $\sim 0.025\%$ of total TPOT (negligible). Estimated TPOT speedup: $\sim 1.33\times$ if routing is effective.

Critical gap: BPTT through FSM recurrence (the gradient path $\partial s_t / \partial A$) is not discussed in the original research document—a critical training concern identified by review. Routing collapse (all tokens converging to one FSM state) is the primary risk. No published experiment validates FSM routing in a generative LLM.

Prior Art and Related Work

- **SR-RNNs (Wang & Niepert, ICML 2019) [316]**: state-regularised RNNs with discrete FSM structure; validated on language modelling.
- **Linear FSM (Li et al., NeurIPS 2020) [109]**: linear-time FSM integration in neural networks.
- **Shortcuts to Automata (Liu et al., ICLR 2023) [114]**: transformers can implement automata; theoretical foundation.
- **Merrill et al. (ICML 2024) [119]**: FSM expressivity bounds for transformer layers.

- **Terzić et al. (NeurIPS 2025, Spotlight)** [136]: learned discrete FSM in sequence models; closest published antecedent.
- **Mordvintsev (2022)**: differentiable FSM baseline.
- **Heakl et al. (Dr.LLM, ICLR 2026)** [92]: FSM gating in LLM inference; directly relevant.
- **MoM (Gong et al., EMNLP 2024)** [82]: mixture of modules with state-based routing.
- **Memory³ (Yang et al., 2024)** [147]: memory-augmented state routing.
- **H-NSM (Lei et al., 2021)** [108]: hierarchical neural state machines; validated on structured prediction.
- **Mixture of Depths (MoD, Raposo et al., ICML 2024)** [128]: per-layer routing baseline—*missing from original draft, identified by review*.
- **RASP (Weiss et al., ICML 2021)** [141]: Restricted Access Sequence Processing; automata-equivalent formalism—*missing from original draft*.

Missing: MoD [128], RASP [141], SCAN benchmark. *Prior art overlap: partial (~60%).*

Complexity Analysis

Table 20: *Complexity impact of Idea 3.7 (Learnable State Machine / FSM Block Router).*

Baseline	TPOT change	Notes
A1	↓ $\sim 1.33\times$ (ideal)	FSM overhead $\sim 0.025\%$; routing collapse risk HIGH
A2	↓ $\sim 1.33\times$ (ideal)	Same
B	↓ $\sim 1.33\times$ (ideal)	MoE routing already provides token-level selection; FSM adds discourse-level routing

Verdict

candidate for investigation. Merge with Idea 4.1 (FiLM conditioning) into a *Unified Learnable FSM Investigation*: both require the same FSM core; the distinction is whether the FSM controls routing (3.7) or conditioning (4.1). No published experiment validates FSM routing in a generative LLM at 1B+ scale. Phase 2: validate on 1B model with formal BPTT gradient analysis; measure routing collapse rate.

Confidence scores: *Overall*: medium (TPOT formula verified; BPTT gap critical; no LLM-scale evidence)

Full analysis: research_3_7_learnable_state_machine.md in the research corpus.

3.23 Idea 3.8: Trainable Activation Functions

Mechanism

Replace fixed activation functions (SiLU, GELU) with per-neuron learned activation functions parameterised by a small number of scalars. PolyNorm (baseline: SwiGLU with 3-matrix structure) adds a degree-2 polynomial scaling: $\text{act}(x) = \text{clamp}(ax + b, \alpha, \beta)$, where (a, b, α, β) are per-neuron learned scalars. This adds $4 \times d_{\text{ff}} \times L$ extra parameters—less than 0.1% of total model parameters.

Overhead correction from review: earlier drafts stated 0.04% overhead; correct value is 0.026% for the 3-matrix SwiGLU structure (not 2-matrix as assumed). Windowed variant (learning activations only in a context window): fundamental flaw—hidden neurons are permutation-invariant, so windowing is not semantically meaningful. Post-training fold-in: the (a, b) parameters can be folded into the upstream weight matrix W_{up} at inference time, leaving only (α, β) (clamping bounds) as runtime-active parameters—reducing the overhead to near-zero at inference.

Quality improvement: +1.21% average downstream accuracy at 1B scale (PolyNorm). TPOT impact: +0.026% (negligible; dominated by (a, b) fold-in).

Prior Art and Related Work

- **PReLU (He et al., ICCV 2015) [90]**: per-channel learned slope; well-validated in vision.
- **APL (Agostinelli et al., 2015) [52]**: adaptive piecewise linear activations.
- **Padé activations (Molina et al., ICLR 2020) [121]**: rational function activations; expressive but numerically sensitive.
- **KAN (Liu et al., ICLR 2025) [115]**: Kolmogorov-Arnold networks; per-edge learnable functions; +1.0% accuracy vs. MLP on small benchmarks.
- **KAT (Yang et al., 2024) [146]**: KAN-inspired transformer activations.
- **Swish (Ramachandran et al., 2017) [127]**: learned β in $x \cdot \sigma(\beta x)$; simple trainable activation.
- **SwiGLU (Shazeer, 2020) [287]**: gated linear unit variant; standard baseline for modern LLMs.
- **ReLU Strikes Back (Mirzadeh et al., ICLR 2024 Oral) [120]**: ReLU outperforms SiLU for sparsity-aware inference; activates fewer neurons.
- **ReLU² (Zhang et al., 2024) [154]**: squared ReLU for structured sparsity.
- **PolyNorm (Zhuo et al., ICLR 2025) [161]**: polynomial activation; +1.21% at 1B scale.
- **DiTAC (Chelly et al., ECCV 2024) [60]**: differentiable task-aware activation clipping.
- **HeLU (Kimhi et al., NeurIPS Workshop 2024) [103]**: hybrid exponential-linear unit.
- **PACT (Choi et al., 2018) [64]**: parameterised clipping for quantisation-aware training; adjacent mechanism.

- **Maxout** (Goodfellow et al., ICML 2013) [83]: piecewise linear learnable activation—*missing from original draft; identified by review.*
- **ProSparse** (Song et al., 2024) [132]: progressive sparsification via activation tuning—*missing from original draft; identified by review.*

Prior art overlap: exists (PolyNorm, PReLU, PACT all validated).

Complexity Analysis

Table 21: *Complexity impact of Idea 3.8 (Trainable Activation Functions).*

Baseline	TPOT change	Notes
A1	↑ 0.026% (negligible)	(a, b) foldable into W_{up} ; only (α, β) remain at inference
A2	↑ 0.026% (negligible)	Same
B	↑ 0.026% (negligible)	Per-expert activation parameters; foldable

TTFT: unchanged. KV cache: unchanged. Parameter count increase: <0.1% of total.

Verdict

lower-priority. No inference speedup. Pure quality-add: +1.21% average downstream at 1B scale. Compatible with all other ideas (no architectural conflicts). Worth including as a “free” quality improvement in any new pretraining run at near-zero cost.

Confidence scores: *Overall:* high confidence (lower-priority confirmed; mechanism well-understood)

Full analysis: research_3_8_trainable_activation.md in the research corpus.

3.24 Idea 4.1: Learned FSM State as Universal Conditioning

Mechanism

Idea 4.1 introduces a discrete latent state vector $\mathbf{s} \in \mathbb{R}^N$ (with $N = 16\text{--}256$ soft-categorical dimensions) maintained across the entire forward pass and updated once per token step. Rather than appending a dedicated memory layer or modifying positional encodings, the FSM state conditions every transformer layer via FiLM-style (feature-wise linear modulation) affine transforms: for each layer l , a small learned projection $\phi_l(\mathbf{s}) = (\gamma_l, \beta_l)$ computes per-channel scale and shift vectors applied to the residual stream after the layer-norm step. The update rule $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{h}_t^{(L)})$ uses the final-layer hidden state $\mathbf{h}_t^{(L)}$ to steer the state, creating a recurrent bottleneck orthogonal to the transformer depth.

During prefill the state vector evolves sequentially—each token’s hidden state updates the FSM before the next token’s forward pass can begin, introducing a sequential dependency of $O(s \cdot (L + 1) \cdot N \cdot d)$

over the context length s . At decode, however, the overhead is a single matrix-vector product per layer, amounting to approximately $2 \cdot L \cdot N \cdot d$ FLOPs per token. At $N=64$, $L=64$, $d=5120$ (Baseline A2), the FiLM overhead is roughly 168 M FLOPs/token—approximately 2% of A2 TPOT, and thus negligible in terms of latency.

The relationship to Idea 3.7 (Static Gate Layer Pruning) is complementary and synergistic. Idea 3.7 gates *which* layers execute; Idea 4.1 conditions *how* those layers compute. A merged design applies FSM conditioning only to the active layers selected by the 3.7 gate, with the FSM advancing only when a layer fires, reducing FiLM overhead proportional to the gate sparsity.

The dominant extra-parameter cost is $L \cdot 2 \cdot N \cdot d$ for the FiLM projection matrices (scale and shift separately), plus the state update network. At $N=64$ this yields roughly 170 M additional parameters for A2—under 0.6% of the base model weight. The KV cache is unaffected; the FSM state is $O(N)$ and does not grow with context.

Prior Art and Related Work

Neural Turing Machines (Graves et al. 2014 [188]) and Differentiable Neural Computers ([187]) pioneered differentiable memory controllers that maintain and update a discrete latent state while reading and writing to an external memory bank. Recurrent State-Space Models and the Mamba architecture (Gu & Dao 2023 [191]) show that selective SSM transitions are learnable without explicit FSM supervision. RWKV (Peng et al. 2023 [269]) demonstrates that learned gating can replace full self-attention while preserving sequence modeling capacity. FiLM conditioning originates in Perez et al. 2018 [271].

Complexity Analysis

Table 22: *Idea 4.1 complexity vs baselines (per token, batch = 1, decode).*

Metric	A1 (Qwen3.5-27B Hybrid)	A2 (Qwen3-32B Dense)	B (Qwen3.5-397B-A17B)	This Idea
TPOT overhead	–	–	–	$\approx +2\%$ (FiLM proj.)
TTFT overhead	–	–	–	Sequential; $O(s)$
Extra params	–	–	–	$\sim 170\text{M}$ (A2), $\sim 140\text{M}$
KV cache	unchanged	unchanged	unchanged	unchanged

The FiLM projection FLOPs per token are $2 \cdot L \cdot N \cdot d$ (scale) $+ 2 \cdot L \cdot N \cdot d$ (shift) $= 4 \cdot L \cdot N \cdot d$. At A2 parameters: $4 \times 64 \times 64 \times 5120 \approx 84\text{M}$ —but the correct figure accounting for both directions is $\approx 168\text{M}$ FLOPs/token, or roughly 2% of A2 TPOT. The sequential prefill dependency is the primary risk for long-context applications.

Verdict

candidate for investigation. Confidence: 7/10. The mechanism is theoretically sound, parameter-efficient, and adds negligible decode overhead. The primary open question is whether the sequential

prefill dependency is tolerable at 262K context and whether the FSM state provides quality lift over a vanilla baseline.

Full analysis: `research_4_1_state_machine_core.md` in the research corpus.

3.25 Idea 4.2: Cross-Layer Shared Weights with Per-Layer LoRA

Mechanism

Idea 4.2 proposes reusing a single shared weight tensor W_{shared} across all L layers of the transformer, with each layer augmented by a lightweight LoRA adapter (A_l, B_l) of rank r . The per-layer forward pass becomes $W_l \approx W_{\text{shared}} + A_l B_l$, where $A_l \in \mathbb{R}^{d \times r}$ and $B_l \in \mathbb{R}^{r \times d_{\text{ff}}}$. The primary appeal is a dramatic reduction in weight storage: the $O(L \cdot d \cdot d_{\text{ff}})$ parameter budget collapses to $O(d \cdot d_{\text{ff}} + L \cdot r \cdot d)$, with each layer retaining its individual character through the low-rank delta.

TPOT at batch = 1 is unchanged in the basic formulation—the materialized weight W_l must still be loaded from HBM for every layer in every token step. An early-exit variant can reduce effective depth and thus TPOT, but this is a separate mechanism. TTFT scales proportionally with the active layers. The KV cache is completely unaffected since no attention mechanism is modified.

The training challenge is gradient accumulation: W_{shared} receives gradients from all L layers simultaneously, causing gradient magnitudes to scale as $O(L)$ relative to a per-layer parameter. A recommended mitigation is to use a per-layer learning rate ratio of $\sim 1/\sqrt{L}$ for W_{shared} relative to the adapters. MoR (Mixture-of-Ranks) variants, where different tokens route to different rank subsets, can further reduce the effective KV cache requirement—correctly at $\sim 67\%$ of the full-attention KV footprint at $N_r = 3$ active ranks (not 50% as misquoted in the source document).

Prior Art and Related Work

Universal Transformers (Dehghani et al. 2018 [41]) are the seminal architecture that shares weights across all transformer layers (identical, not LoRA-augmented). ALBERT (Lan et al. 2019 [213]) applies cross-layer parameter sharing to BERT with a factored embedding layer, demonstrating that shared weights can match per-layer quality on GLUE/SQuAD at substantially lower parameter count. LoRA (Hu et al. 2022 [201]) introduces the rank-decomposed adapter framework from which Idea 4.2 draws its per-layer delta. Bae et al. (ICLR 2025 [5]) provide an analysis of shared-weight transformers for language modeling. Wei et al. (2024 [319]) study BlockDense and related structured FFN layers trained from scratch. AdaLoRA (Zhang et al. 2023 [345]) and DyLoRA (Valipour et al. 2022 [310]) both provide adaptive rank allocation schemes directly relevant to the per-layer rank selection in Idea 4.2.

Complexity Analysis

The review corrected the parameter reduction formula: the LoRA adapter count per layer is $L \times 3 \times r \times (d + d_{\text{ff}})$ (not $L \times 3 \times 2 \times r \times d$ as originally written), giving the corrected ratios above. Full-model storage compression (weights to disk) is $\sim 17\times$ at $r=64$ (the original document claimed $35\text{--}54\times$). The MoR KV claim of 50% reduction was corrected to $\sim 67\%$ at $N_r=3$.

Table 23: *Idea 4.2 parameter reduction at varying rank r (A2: $L=64$, $d=5120$, $d_{\text{ff}}=25600$).*

Rank r	A1 (param. reduction)	A2 (param. reduction)	B (param. reduction)	TPOT vs dense
64	$\sim 31\times$	$\sim 32.7\times$	(expert-weighted)	unchanged
128	$\sim 20\times$	$\sim 21.9\times$	–	unchanged
256	$\sim 12\times$	$\sim 13.2\times$	–	unchanged
512	$\sim 7\times$	$\sim 7.4\times$	–	unchanged

Verdict

candidate for investigation. Confidence: 6/10. Parameter efficiency is real and well-supported. The core risk is training instability from shared-weight gradient accumulation and the absence of TPOT improvement at decode time without an accompanying early-exit or layer-dropping mechanism.

Full analysis: `research_4_2_shared_weights_lora.md` in the research corpus.

3.26 Idea 4.3: Low-Rank Factorization Everywhere (LoRA-Everywhere)

Mechanism

Idea 4.3 examines three realizations of replacing full-rank weight matrices with low-rank products. Variant 4.3-A is equivalent to Idea 4.2 (shared base plus per-layer adapters). Variant 4.3-B adds a trainable full-rank W_{core} alongside the low-rank product $A \cdot B$; this retains all parameters and provides no inference benefit. Variant 4.3-C—the productive case—eliminates W_{core} entirely, storing and computing only $A \cdot B$ for each weight matrix. At decode (batch = 1), loading $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{ff}}}$ costs $2 \cdot r \cdot d$ bytes versus $d \cdot d_{\text{ff}}$ bytes for the full matrix, yielding a bandwidth reduction of $d_{\text{ff}}/(2r)$ —for example $12.8\times$ at $r=1000$, $d_{\text{ff}}=25600$. The two-step matrix-vector product at decode ($y = A(Bx)$) is memory-bandwidth-bound and can be realized without materializing $W = AB$.

TTFT is approximately $2\times$ *faster* for pure $A \cdot B$ at compute-equal ranks (fewer FLOPs because $r \ll d$)—a sign inversion from the original document, which incorrectly stated $1.5\times$ *slower*. Training FLOPs are approximately $0.5\times$ relative to dense. The key empirical evidence from SLTrain, Flora, and WeLore is sobering: uniform low-rank pretraining is insufficient. WeLore shows that uniform 50% rank allocation yields PPL 1836.62 versus 11.87 for non-uniform rank, confirming that per-layer adaptive rank is essential.

Prior Art and Related Work

LoRA (Hu et al. 2022 [201]) and the intrinsic dimensionality analysis of Aghajanyan et al. 2021 [1]) motivate the low-rank assumption. GaLore ([348]) proposes gradient low-rank projection for memory-efficient training but does not eliminate full-rank weights at inference. SLTrain ([290]) demonstrates that pure low-rank pretraining without a full-rank component is insufficient for competitive language modeling quality. WeLore ([322]) provides the crucial non-uniform rank evidence (PPL 1836.62 uniform vs. 11.87 non-uniform). CoLA (Liu 2025 [235]) and Monarch matrices (Dao et al. 2022 [35])

provide structured alternatives to naive SVD factorization.

Complexity Analysis

Table 24: *Idea 4.3-C inference bandwidth reduction (A2 MLP: $d=5120$, $d_{ff}=25600$).*

Rank r	BW ratio vs dense	TTFT	TPOT (batch=1)	Quality risk
500	$25.6\times$	$\sim 2\times$ faster	$\sim 25\times$ better	HIGH
1000	$12.8\times$	$\sim 2\times$ faster	$\sim 12\times$ better	MEDIUM
2000	$6.4\times$	$\sim 2\times$ faster	$\sim 6\times$ better	LOW-MED

Activation memory at large batch: at A2 scale with rank $r=1000$, intermediate activations from the two-step product add approximately 21 GB of extra activation memory at batch=32. Variants 4.3-A (= Idea 4.2) and 4.3-B are superseded; only 4.3-C is recommended for investigation.

Verdict

candidate for investigation (4.3-C only). Confidence: 7/10. The bandwidth reduction is real and large. The viability gate is whether non-uniform rank allocation can recover sufficient quality for the target model family; WeLore results are encouraging but require replication at Qwen3 scale.

Full analysis: `research_4_3_lora_everywhere.md` in the research corpus.

3.27 Idea 4.4: Exponentially-Spaced Skip Connections (Skip-List Layers)

Mechanism

Idea 4.4 replaces per-layer residual connections with exponentially-spaced long-range skip connections placed at layer indices $L/2$, $L/4$, $L/8$, $\dots$. The intuition is to provide gradient highways spanning progressively longer stretches of depth, analogous to a skip list data structure. The skip buffers store layer activations and add them to later layers, with no change to TPOT or TTFT—the elementwise additions are $O(d)$ per skip and represent a ratio of approximately 1 FLOP per 50,000 FLOP of MLP compute.

The primary risk is gradient attenuation in the plain sub-block between skip connections. A 32-layer sub-block without any skip connection exhibits gradient magnitudes of approximately 2.3×10^{-10} relative to the output layer, which can cause training to fail for the affected parameters. Hyper-Connections (Sun et al. 2024 [297]) is identified as the closest modern work, using a learned linear combination of all previous layer outputs; Idea 4.4 is a sparse, structured special case of this formulation. AltUp (Baykal et al. NeurIPS 2023 [10]) provides a closely related alternating update mechanism.

Skip buffer activations add approximately 503 MB of extra activation memory at A2 scale ($L=64$, $\log_2 64=6$ skip buffers, each $d=5120$ floats at BF16, times the sequence length at prefill).

Prior Art and Related Work

ResNets (He et al. 2016 [197]) and DenseNet (Huang et al. 2017 [200]) establish per-layer and all-to-all skip connections for vision models. DenseFormer (Pagliardini et al. 2024 [266]) applies dense cross-layer weighted averaging to transformers and shows quality improvement. Hyper-Connections ([297]) is the most directly comparable published system. Attention Residuals (Kimi Team 2026 [207]) show that cross-layer residuals help at scale. AltUp ([10]) is the critical missing citation identified in the review.

Complexity Analysis

Table 25: *Idea 4.4 cost profile vs baselines.*

Metric	A1	A2	B	This Idea
TPOT	baseline	baseline	baseline	unchanged
TTFT	baseline	baseline	baseline	unchanged
Activation memory	baseline	baseline	baseline	+503 MB (A2)
Extra params	0	0	0	0

Verdict

lower-priority (strong form); pursue as initialization variant of Hyper-Connections. Confidence: 5/10. The mechanism adds no inference cost but also no inference benefit, and the severe gradient attenuation risk in long sub-blocks requires empirical validation before committing resources.

Full analysis: `research_4_4_skip_list_layers.md` in the research corpus.

3.28 Idea 4.5: Learned Residual Flow (Static-Gate Layer Pruning)

Mechanism

Idea 4.5 equips each transformer layer with a scalar gate $g_i \in [0, 1]$ learned during training and frozen at inference. Layers whose gate hard-zeros at zero are physically removed (*model surgery*), yielding a smaller model with lower TPOT and TTFT proportional to p (the fraction of active layers). At $p=0.75$, TPOT is $0.75\times$ the full-model baseline. The ShortGPT (Men et al. 2024 [249]) Block Influence (BI) metric empirically shows that 24–27% of layers in modern LLMs are removable with less than 5% MMLU degradation.

Gate collapse (all gates converging to zero or one during training) is managed by adaptive L_1 regularization following the GateSkip (2024 [180]) approach. The training overhead from scalar gates is approximately $1.00\times$ (not the $1.05\text{--}1.15\times$ erroneously attributed to gates in the source document). A multiple-stream variant with per-stream gates is assessed as LOW feasibility due to routing complexity.

Prior Art and Related Work

Lottery Ticket Hypothesis (Frankle & Carbin 2019 [170]) establishes that sparse subnetworks exist within dense networks. LayerDrop (Fan et al. 2020 [167]) is a critical missing citation: it trains with random layer dropping, enabling structured pruning at inference. Mixture of Depths (Raposo et al. 2024 [276])—the most directly comparable published system—routes tokens to a subset of layers via a per-token routing decision. DenseFormer ([266]) and Hyper-Connections ([297]) are related cross-layer mixing approaches. ShortGPT ([249]) and GateSkip ([180]) provide the empirical base for gate-based pruning.

Complexity Analysis

Table 26: *Idea 4.5 TPOT improvement at varying pruning fraction p (A2 baseline).*

Active fraction p	A1 TPOT	A2 TPOT	B TPOT	Weight memory
1.00	1.00×	1.00×	1.00×	full
0.75	0.75×	0.75×	0.75×	0.75×
0.50	0.50×	0.50×	0.50×	0.50×

Verdict

candidate for investigation. Confidence: 7/10. The inference gain (proportional to pruning fraction) is direct and well-supported by ShortGPT empirical evidence. The primary risk is quality degradation beyond the 5% MMLU threshold, especially for reasoning-intensive tasks that rely on deeper layers.

Full analysis: `research_4_5_learned_residual_flow.md` in the research corpus.

3.29 Idea 4.6: Mixture of Models (MoMoRA)

Mechanism

MoMoRA (Mixture of Models with Recurrent Adapters) extends the MoE paradigm from expert weight matrices to full heterogeneous models. A shared embedding layer and output projection are used across all pool models; a GRU-based state fusion layer maintains inter-token context; a hierarchical router groups pool models into Light/Medium/Heavy tiers and routes each token via top- k selection. An Adaptive Computation Time (ACT) (Graves 2016 [186]) style gated emission allows halting computation early.

The architecture imposes a *shared* d_{model} constraint: all pool models must have identical embedding dimension to share the embedding matrix and output projection. This prevents leveraging existing pretrained heterogeneous models of different sizes, substantially increasing the barrier to entry. Four additional high barriers were identified: (1) dynamic halting is incompatible with `torch.compile`/CUDA graphs/vLLM continuous batching; (2) joint training requires 648+ GB

peak memory; (3) token-level routing quality is unvalidated; (4) the shared d_{model} constraint forces a homogeneous pool.

Prior Art and Related Work

Mixture of Experts (Shazeer et al. 2017 [288]) and Switch Transformers (Fedus et al. 2022 [169]) establish token-level routing to weight subsets. FrugalGPT (Chen et al. 2023 [24]) cascades API-served models for cost reduction—a related but query-level (not token-level) routing approach. RouteLLM (Ong et al. 2024 [265]) routes full queries to strong or weak models. Speculative decoding (Leviathan et al. 2023 [220]) uses a small draft model to generate candidates verified by a large model—token-level interaction between heterogeneous models. ACT ([186]) and PonderNet (Banino et al. 2021 [9]) provide the adaptive halting framework. Branch-Train-Merge (Li et al. 2022 [224]) and Depth-Adaptive Transformer (Elbayad et al. 2020 [50]) are additional missing prior art identified by the review.

Complexity Analysis

Table 27: *Idea 4.6 cost profile relative to baselines.*

Baseline	Decode TPOT	Notes
A1 (27B Hybrid)	Routing overhead <1%	DeltaNet layers route without KV
A2 (32B Dense)	KV $\approx$ 8.59 GB at 32K	Largest decode bottleneck; MoM helps
B (397B MoE)	KV $\approx$ 1.0 GB at 32K	Already small; MoM marginal gain

Verdict

candidate for investigation — CONDITIONAL. Confidence: 5/10. The concept is novel at the token-level model routing granularity, but four simultaneous high barriers make near-term realization unlikely without significant infrastructure investment.

Full analysis: research_4_6_mixture_of_models.md in the research corpus.

3.30 Idea 4.7: Context-Conditioned Vocabulary Compression

Mechanism

The LM head output projection $W_{\text{out}} \in \mathbb{R}^{V \times d}$ dominates memory bandwidth at decode for large-vocabulary models. Idea 4.7 activates only $V_{\text{active}} \ll V$ vocabulary entries per token, selected either by static frequency ranking, context-conditioned draft-model prediction, per-entry structural compression, or a combined approach. At $V_{\text{active}}=10,000$ from $V=151,936$ (A2), the LM head bandwidth reduces $15\times$. DynaSpec (2024 [49]) achieves a $2.23\times$ throughput improvement via an MLP meta-classifier routing to $\sim 28,000$ token clusters.

The corrected LM head bandwidths from the review are: A1: 2.547 GB (vocab 248,320, $d=5120$); A2: 1.308 GB (vocab 151,936, $d=5120$); B: 2.034 GB (vocab 248,320, $d=4096$). For Baseline B, the TPOT improvement from $15\times$ LM head compression is approximately 5.7%—substantially larger than the $\sim 0.2\%$ figure in the source document, which used an incorrect LM head bandwidth. The corrected A1 TPOT improvement is $\sim 0.955\times$ (not $\sim 0.977\times$).

Prior Art and Related Work

Differentiated Softmax (Chen et al. NAACL 2016 [23]) is the foundational work on vocabulary partitioning at the output layer. SVD-Softmax ([300]) applies low-rank decomposition to the output projection, with an author attribution error flagged (“Jongwuk Shim” likely incorrect; verify “Jongwuk Lee”). Adaptive Softmax (Grave et al. 2017 [183]) uses frequency-binned cluster softmax. DynaSpec ([49]) provides the most direct production evidence. Tree-Structured Diffusion LM (arXiv:2604.03537, year corrected to 2026 [307]) is a related hierarchical output approach.

Complexity Analysis

Table 28: *Idea 4.7 LM head bandwidth and TPOT impact by baseline.*

Metric	A1	A2	B	This Idea
Full LM head BW	2.547 GB	1.308 GB	2.034 GB	$V_{\text{active}}/V \times \text{full}$
TPOT at $V_{\text{active}}=10\text{K}$	$\sim 0.955\times$	$\sim 0.93\times$	$\sim 0.943\times$	per above
KV cache	unchanged	unchanged	unchanged	unchanged

Verdict

candidate for investigation. Confidence: 7/10. The mechanism is straightforward, benefits are real (especially for B-model), and DynaSpec provides production evidence. The context-conditioned sub-variant requires a draft model with associated KV overhead.

Full analysis: research_4_7_compressed_dictionary.md in the research corpus.

3.31 Idea 5.1: KV Cache Quantization-Aware Training (TurboQuant Extension)

Mechanism

TurboQuant (Zandieh et al. ICLR 2026 [341]) is a post-hoc, training-free KV cache quantization method combining two innovations: PolarQuant, which applies a rotation to the KV tensors so that quantization error concentrates as a Beta distribution (enabling 3.5-bit representation with near-zero LongBench degradation), and QJL, which corrects the quantized residual using the 1-bit sign of a Gaussian projection. At 3.5-bit, LongBench score is 50.06 (zero loss); at 2.5-bit, 49.44.

Idea 5.1 proposes extending TurboQuant with Quantization-Aware Training (QAT) scoped specifically to the KV cache interface, allowing the model to adapt its key/value representations to better

tolerate quantization error. The QAT scope is narrow: only the W_K and W_V projections and their layer norms see quantized gradients; all other weights train normally. This minimizes QAT overhead while targeting the bottleneck.

The primary challenge is QJL differentiability: the sign function used in QJL correction has zero gradient almost everywhere, making naïve STE (straight-through estimator) degenerate. The review recommends Phase 1 restrict to PolarQuant QAT (rotation + scalar quantization, no QJL) to avoid this issue.

Prior Art and Related Work

LLM-QAT (Liu et al. 2023 [237]) provides the QAT framework for LLM weight and activation quantization. SpinQuant (Liu et al. 2024 [241]) applies learned rotation matrices to weights before quantization, directly analogous to PolarQuant’s rotation step. ScissorHands (Liu et al. 2023 [239]) selectively retains KV entries based on attention pivot patterns. QServe (Lin et al. 2024 [232]) combines W4A8KV4 quantization in a unified serving system, achieving $2.4\text{--}3.5\times$ throughput on Qwen1.5-72B. PV-Tuning (Tseng et al. 2024 [309]) fine-tunes the model to restore quality after post-training vector quantization.

Complexity Analysis

Table 29: *Idea 5.1 KV cache size and TPOT impact at 3.5-bit quantization.*

Metric	A1 (Hybrid)	A2 (Dense)	B (MoE)	At 3.5-bit
KV cache BF16	~17 GB (262K)	~8 GB (32K)	varies	baseline
KV cache 3.5-bit	~3.7 GB	~1.75 GB	varies	~4.6× smaller
TPOT (decode, 32K)	~1.5–2× better	~1.5–2× better	–	realistic
QAT training cost	–	–	–	~1.1× (W_K/W_V only)

Note: A1 KV cache is approximately 17 GB (head_dim=256, 4 KV heads per GatedAttn layer)—the source document used head_dim=128, halving the A1 figures. A2 at 32K context is ~8 GB .

Verdict

candidate for investigation. Confidence: 7/10. TurboQuant’s post-training results are strong. The QAT extension is well-motivated and has a clear narrow scope. The primary open question is whether QJL’s non-differentiability fundamentally limits the QAT gain or whether PolarQuant-only QAT is sufficient.

Full analysis: research_5_1_turbo_quant.md in the research corpus.

3.32 Idea 5.2: LSTM-Gated Attention (KV Cache Elimination)

Mechanism

Idea 5.2 replaces softmax self-attention with a Gated Linear Attention (GLA) formulation (Yang et al. ICML 2024 [331]), enabling a pure recurrent forward pass at decode. The GLA recurrence is $S_t = G_t \odot S_{t-1} + k_t \otimes v_t$, where the state $S_t \in \mathbb{R}^{d_k \times d_v}$ replaces the growing KV cache. For A2 (Qwen3-32B Dense), this converts the 8.59 GB KV cache (at 32K context) to a fixed ~ 134 MB recurrent state—a $64\times$ reduction in memory footprint independent of sequence length.

For Baseline A1 (Qwen3.5-27B Hybrid), only the 16 full-attention layers need conversion; the 48 DeltaNet layers already use a recurrent form. The corrected A1 post-conversion recurrent state is ~ 50 MB (not 13 MB as stated in the source document, because A1 GatedAttn uses head_dim=256 and 4 KV heads). The TPOT improvement at A2 32K context is $\sim 51\%$ (decode bandwidth reduction from eliminating KV reads). The TTFT changes from $O(s^2d)$ to $O(s \cdot d^2)$ —linear in sequence length for the recurrent form, but with a larger constant.

The FLOPs reduction relative to softmax attention at A2 scale is approximately $32\text{--}85\times$ (not the $250\times$ stated in the source, which ignored multi-head structure).

Prior Art and Related Work

RWKV ([269]), RetNet (Sun et al. 2023 [298]), GLA ([331]), Griffin (De et al. 2024 [38]), HGRN2 ([198]), xLSTM (Beck et al. NeurIPS 2024 [12]), Mamba ([191]), and BASED (Arora et al. ICLR 2024 [3]) all provide related recurrent or linear attention formulations. The NVlabs/GatedDeltaNet reference implementation is the recommended starting point for A1 integration.

Complexity Analysis

Table 30: Idea 5.2 KV cache and recurrent state comparison.

Metric	A1 (Hybrid)	A2 (Dense)	B (MoE)	This Idea
KV cache (32K ctx, BF16)	~ 17 GB	~ 8.6 GB	varies	0 (recurrent)
Recurrent state	N/A	N/A	N/A	~ 50 MB (A1), ~ 134 MB (A2)
TPOT at 32K	baseline	baseline	baseline	$\sim 51\%$ better (A2 decode)
TTFT	$O(s^2)$	$O(s^2)$	$O(s^2)$	$O(s \cdot d^2)$

Verdict

ENDORSE. Confidence: 8/10. KV cache elimination is the highest-leverage KV reduction available, and the GLA formulation is well-validated at scale. The recurrent prefill trade-off requires evaluation for long-context use cases.

Full analysis: research_5_2_lstm_gated_attention.md in the research corpus.

3.33 Idea 5.3: Grammar/FSM-Structured Sparse Attention

Mechanism

Idea 5.3 constrains the attention pattern using a grammar or finite-state machine (FSM) to determine which KV entries each query is permitted to attend. Only grammar-permitted KV entries are materialized, reducing KV cache from $O(s \cdot d_{kv})$ to $O(|S| \cdot d_{kv})$ where $|S|$ is the number of FSM states (grammar-accessible tokens). A practical decomposition uses 70% sliding window attention ($w=512$), 20% learned content-selected attention ($k=64$ per query), and 10% attention sinks ($g=8$ tokens), yielding approximately 1.78% total KV retention.

The primary implementation challenges are: (1) a per-token CFG parse overhead of $O(s)$ that can dominate the KV savings at long contexts; (2) random KV selection produces cache-line misses, requiring block-aligned selection (page granularity) for hardware efficiency. The MoSA (2024) claim of 27% perplexity improvement is flagged as speculative and from an unverified preprint.

Prior Art and Related Work

Reformer (Kitaev et al. 2020 [209]) uses locality-sensitive hashing to reduce attention to $O(s \log s)$. Longformer (Beltagy et al. 2020 [13]) and BigBird (Zaheer et al. 2020 [339]) combine sliding window, global, and random attention. Structformer (Wang et al. 2021 [317]) explicitly integrates syntactic structure into attention. Willard & Louf (2023 [324]) provide foundational FSM grammar compilation, a critical missing citation in the source document. StreamingLLM ([328]) introduces attention sinks for infinite-length generation.

Complexity Analysis

Table 31: *Idea 5.3 KV reduction and TPOT impact (A2 baseline, 32K context).*

Metric	A1	A2 (32K)	B	At 1.78% KV retention
KV cache BF16	~1.07 GB	~4.3 GB	varies	×retention
TPOT at 32K	baseline	~1.03×	baseline	benefit grows at longer context

Verdict

candidate for investigation with major caveats. Confidence: 5/10. Grammar-structured attention has theoretical elegance but faces severe implementation challenges (CFG overhead, cache misses). The actual TPOT improvement at practical contexts is modest (~3% at 32K).

Full analysis: research_5_3_grammar_attention.md in the research corpus.

3.34 Idea 5.4: Dynamic Relevance-Based KV Eviction (Linked Attention)

Mechanism

Idea 5.4 dynamically evicts low-relevance KV entries from the cache, retaining only the top- k most relevant entries per query. Two eviction strategies are analyzed: score-based (H2O, SnapKV, Quest)—which uses cumulative attention scores to identify pivots but does not reduce current-step FLOPs, only future KV capacity; and score-free recency-based (StreamingLLM)—which evicts oldest entries and does reduce current-step FLOPs from the reduced cache size. Page-level granularity is essential for hardware efficiency; token-level scatter-gather achieves only 10–40% of peak memory bandwidth.

At 10% KV retention on A2 (32K context): KV cache reduces from ~ 8.6 GB to ~ 0.86 GB. The TPOT improvement from page-level eviction is approximately $5\text{--}8\times$ bandwidth reduction on the KV read path, with overall model TPOT improvement of $\sim 1.11\times$ at 32K (not $1.9\times$ as stated in the source).

The primary benefit of this idea is KV *capacity* reduction (enabling longer contexts within a fixed memory budget), rather than TPOT reduction at typical short contexts.

Prior Art and Related Work

H2O (Zhang et al. 2023 [346]), ScissorHands (Liu et al. 2023 [240]), SnapKV (Li et al. 2024 [233]), Quest (Tang et al. 2024 [303]), DuoAttention (Xiao et al. 2024 [327]), MagicPIG (He et al. 2024 [196]), StreamingLLM ([328]), and PyramidKV ([273]) all provide specific eviction strategies. Missing citations identified by the review include KVQuant, CaM ([21]), and RazorAttention ([277]).

Complexity Analysis

Table 32: *Idea 5.4 KV cache at varying retention rates (A2 at 32K context).*

Retention	A1 KV	A2 KV	B KV	TPOT improvement
100%	~ 17 GB	~ 8.6 GB	varies	$1.0\times$
50%	~ 8.5 GB	~ 4.3 GB	varies	$\sim 1.05\times$
10%	~ 1.7 GB	~ 0.86 GB	varies	$\sim 1.11\times$

The review corrects: A2 KV cache at 32K is ~ 8 GB . The H2O $29\times$ performance claim is an extreme constraint peak; realistic token-level scatter-gather achieves 10–40% of peak bandwidth.

Verdict

candidate for investigation. Confidence: 7/10. Strong prior art base, clear capacity benefit. TPOT benefit at moderate contexts is more modest than the source document implied, but long-context capacity is independently valuable.

Full analysis: research_5_4_linked_attention.md in the research corpus.

3.35 Idea 5.5: Per-Token Variable Lookback Windows (Ragged Window Attention)

Mechanism

Idea 5.5 assigns each token a variable lookback window width w_i , replacing the fixed sliding-window of Longformer or BigBird. If the average window width $w_{\text{avg}}=2,000$ on a sequence of length $s=32,000$, prefill FLOPs reduce from $O(s^2)$ to $O(s \cdot w_{\text{avg}})$ —a $16\times$ reduction. KV bandwidth at decode reduces proportionally to the per-token window average.

A critical conceptual error in the source document was identified by the review: KV *storage* requires allocation proportional to w_{max} (the maximum window any token uses), not w_{avg} . With a hard w_{max} cap, the scheme is equivalent to a fixed sliding-window attention at w_{max} . The genuine benefit is compute savings at prefill (computing attention over $w_i < w_{\text{max}}$ entries per token), not KV storage reduction.

Prior Art and Related Work

Adaptive Attention Span (Sukhbaatar et al. 2019 [294]) learns a continuous per-head lookback masking function, directly implementing per-head variable attention span. Longformer ([13]) and BigBird ([339]) use fixed sliding windows. LM-Infinite (Han et al. 2024 [194]) and StreamingLLM ([328]) enable infinite-length generation with fixed attention sinks. Mixture of Attention (MoA; Fu et al. 2024 [179]) heterogeneously mixes attention spans across heads. NSA (Yuan et al. 2025 [338]) implements native sparse attention in hardware. Missing citations identified by the review: Sparse Transformer ([26]), Mixture of Depths ([276]), PagedAttention ([210]), FlexAttention. SATA 2026 (arXiv:2601.20267) could not be verified.

Complexity Analysis

Table 33: *Idea 5.5 complexity (corrected KV storage uses w_{max} , not w_{avg}).*

Metric	A1	A2	B	This Idea
KV storage	~ 2.0 GB (32K)	~ 8.0 GB (32K)	varies	$\approx w_{\text{max}}$ -limited
Prefill FLOPs	$O(s^2)$	$O(s^2)$	$O(s^2)$	$O(s \cdot w_{\text{avg}})$
Decode BW	proportional to s	proportional to s	proportional to s	proportional to w_i

Verdict

candidate for investigation. Confidence: 6/10. The prefill FLOPs benefit is real and can be large. The KV storage benefit requires combining with page-level eviction. The mechanism is well-supported by Adaptive Attention Span prior art.

Full analysis: research_5_5_ragged_window_attention.md in the research corpus.

3.36 Idea 5.6: Double Attention (Two Sequential Self-Attention Passes)

Mechanism

Idea 5.6 inserts two sequential self-attention passes per decoder block. The second attention can refine or gate the output of the first, potentially enabling shallower networks (fewer total blocks) with equivalent expressive power due to the increased per-block computation. Three variants are analyzed: (1) fully shared Q/K/V and KV cache; (2) separate Q via LoRA, shared K/V and KV cache (recommended); (3) fully separate projections and KV cache (doubles memory cost).

The shared-KV variant (recommended) does not increase KV cache memory, but doubles KV cache *reads* per block—increasing attention-related memory bandwidth linearly with the number of attention passes. TTFT increase is proportional to the attention FLOPs fraction; TPOT increase is proportional to KV read fraction. At A2 scale (dense, 64 layers), the TTFT overhead at 8K context is approximately 1.1×; at 32K, approximately 1.29×. The depth-reduction hypothesis—that two attention passes per block allows removing k blocks while maintaining quality—is the key unknown quality gate.

Prior Art and Related Work

DaViT (Ding et al. 2022 [47]) is the direct vision-domain existence proof, applying dual attention (window + channel attention) per block. Looped Transformers (Yang et al. 2023 [332]) apply the same set of attention layers multiple times, directly related to the depth-reuse hypothesis. Attention-over-Attention (Cui et al. ACL 2017 [33]) applies a second attention step to re-weight the first attention matrix. Universal Transformers ([41]) share weights across repeated passes. Prior art coverage in the source document is assessed at ~65–70% by the review.

Complexity Analysis

Table 34: *Idea 5.6 overhead (shared-KV variant) vs baselines.*

Metric	A1	A2	B	Shared-KV Double Attn
KV cache	unchanged	unchanged	unchanged	unchanged
KV reads per block	1×	1×	1×	2×
TTFT at 8K	1.0×	1.0×	1.0×	~1.1×
TTFT at 32K	1.0×	1.0×	1.0×	~1.29×
TPOT	1.0×	1.0×	1.0×	~1.1–1.2× (attn fraction)

Verdict

candidate for investigation. Confidence: 6/10. The depth-reduction hypothesis is the key unknown. If two attention passes allow removing $\geq 25\%$ of blocks with equivalent quality, the overhead is net-positive. This requires a direct empirical test on a 1–3B pilot model.

Full analysis: `research_5_6_double_attention.md` in the research corpus.

3.37 Idea 5.7: Block-Level Compressed Weights

Mechanism

Idea 5.7 applies compression at the sub-matrix tile level, partitioning each weight matrix into tiles $B_1, B_2, \dots, B_n$ and applying independent compression to each tile. Two primary compression families are analyzed: (a) block-wise quantization, where each tile of $g=128$ contiguous weights shares one scale/zero-point at INT4 (GPTQ, AWQ, ATOM style); and (b) block-wise low-rank factorization, where each $B \times B$ tile is stored as $U_b \cdot V_b$ with per-tile rank $r_b \ll B$. A combined variant ($W \approx Q_block + L_block \times R_block$, all quantized) targets the sub-2.5-bit regime as in CALDERA.

At decode (batch = 1, GEMV), weight bytes loaded per token are: INT4 with $g=128$: $\approx 0.25 \times$ FP16 bytes (+1.5% scale overhead), giving $\sim 3.9 \times$ ideal bandwidth reduction, $\sim 2\text{--}3 \times$ practical (QServe: $2.4\text{--}3.5 \times$ on Qwen1.5-72B). Block low-rank with $r_b=16$, $B=128$: $2r_b/B=0.25 \times$ bytes, $\sim 4 \times$ ideal; in practice $\sim 2\text{--}3 \times$ due to sequential two-step GEMV dependency. The review corrects a FLOPs factor error: the source stated “ $8 \times$ fewer FLOPs” for $r_b=8$, $B=64$; the correct factor is $B/(2r_b)=4 \times$.

For hardware efficiency, the minimum Tensor Core-compatible tile for $4 \times$ compression is $r_b=16$, $B=128$ (not $r_b=8$, $B=64$ as in the source document, which underutilizes warp dimensions). The ATOM $7.73 \times$ throughput figure is for multi-user serving (batch > 1), not batch = 1 TPOT; MARLIN’s $2.8 \times$ at batch = 1 is the appropriate benchmark.

Prior Art and Related Work

GPTQ (Frantar et al. ICLR 2023 [174]), AWQ (Lin et al. MLSys 2024 [231]), ATOM (Zhao et al. MLSys 2024 [350]), and OmniQuant (Shao et al. ICLR 2024 [285]) cover block-wise quantization at column-group to transformer-block granularity. MARLIN (Frantar et al. 2024 [175]) and QServe ([232]) provide production-ready kernels. CALDERA (IST Austria / ETH Zurich team, NeurIPS 2024 [20]), LoftQ (Li et al. ICLR 2024 [226]), ASVD (Yuan et al. 2023 [337]), and SVD-LLM (Wang et al. ICLR 2025 [318]) cover per-matrix low-rank compression. Monarch ([35]) and BLAST (Lee et al. NeurIPS 2024 [217]) cover block-adaptive structured matrices. QuIP# ([308]) and SqueezeLLM (Kim et al. ICML 2023 [208]) are high-priority missing citations added by the review.

Complexity Analysis

Table 35: Idea 5.7 TPOT improvement at INT4 and block low-rank ($B=128$, $r_b=16$).

Variant	A1 TPOT	A2 TPOT	B TPOT	Practical range
INT4 block-wise	$\sim 2.8 \times$	$\sim 2.4\text{--}3.0 \times$	$\sim 2.0 \times$ (experts)	$2\text{--}3 \times$
Block low-rank	$\sim 2\text{--}3 \times$	$\sim 2\text{--}3 \times$	$\sim 2 \times$	estimated
Combined Q+LR	$\sim 3\text{--}4 \times$	$\sim 3\text{--}4 \times$	–	research

Verdict

high-priority (INT4 sub-variant immediately; block low-rank as prototype). Confidence: 8/10 for INT4; 6/10 for block low-rank. The INT4 path has production-ready kernels (MARLIN, QServe) validated on Qwen-family models.

Full analysis: `research_5_7_block_compressed_weights.md` in the research corpus.

3.38 Idea 5.8: Block Sparse Weights

Mechanism

Idea 5.8 zeroes out entire contiguous blocks of weight matrices, enabling hardware-efficient sparse computation via two paths: (a) NVIDIA 2:4 structured sparsity (50% sparsity, natively accelerated on Ampere/Hopper Sparse Tensor Cores); and (b) arbitrary block-sparse rows (BSR, $\geq 50\%$ sparsity, $\geq 32 \times 32$ blocks required for hardware advantage). The distinction from Idea 5.7 is the information loss: 5.7 retains all information in compressed form; 5.8 permanently discards pruned weights.

At 2:4 sparsity (Path A): practical end-to-end TPOT improvement for 32B-class models at batch = 1 is approximately $1.24\text{--}1.4\times$ (Wanda end-to-end, LLaMA-7B: $1.24\times$; conservative upper bound from bandwidth analysis: $1.6\times$). Weight memory: $0.625\times$ dense (50% weights + 12.5% metadata). MaskLLM (Fang et al. NeurIPS 2024 [166]) substantially recovers the quality drop: LLaMA-2-7B at 2:4 reduces from SparseGPT PPL 10.42 to MaskLLM PPL 6.72 (dense: 5.12).

At BSR $\geq 50\%$ (Path B): block-32 on A100 beats cuBLAS at density $< 50\%$. Theoretical bandwidth reduction at 75% BSR: $\sim 3.6\times$; practical with overhead: $\sim 2\text{--}3\times$. BLaST (NeurIPS 2025 [16]) reports $1.6\times$ end-to-end for a 1B model at 95% sparsity—not at 50% on 32B class; the review flags this as a claim conflation in the source document.

Combined 4-bit quantization + 2:4 pruning: Antmicro (2024) reports $> 2.75\times$ batch = 1 speedup for Mistral-7B/Phi-2. The quality drop without fine-tuning at 2:4 is severe (MMLU: $0.667 \rightarrow 0.404$); with fine-tuning it partially recovers to 0.604.

Prior Art and Related Work

NVIDIA 2:4 sparsity specification (Mishra et al. 2021 [253]), SparseGPT (Frantar & Alistarh, ICML 2023 [176]), Wanda (Sun et al. ICLR 2024 [299]), and Thanos (Ilin & Richtarik 2025 [203]) cover the weight pruning pipeline. MaskLLM ([166]) and LTE (2024 [246]) cover learnable mask discovery. Missing citations from the review: Deja Vu (Liu et al. NeurIPS 2023 [236]) for activation sparsity; PowerInfer (Song et al. 2023 [292]) for hot-neuron CPU offload; ReLU Strikes Back (Mirzadeh et al. 2024 [252]) as complementary activation sparsity.

Complexity Analysis

The TPOT range $1.27\text{--}1.6\times$ in the source document conflates different conditions: $1.27\times$ is a multi-batch throughput figure (not batch = 1 latency); $1.6\times$ is at 95% sparsity on a 1B model (not

Table 36: *Idea 5.8 TPOT and memory at 2:4 sparsity (A2, batch=1 decode).*

Metric	A1	A2	B	2:4 Sparse
Weight memory	~54 GB	~64 GB	~794 GB (total)	0.625× dense
TPOT (batch=1)	baseline	baseline	baseline	~1.24–1.4×
TTFT	baseline	baseline	baseline	≤2× (non-GEMM dilutes)

50% on 32B). The corrected batch = 1 decode range for 50% BSR on Qwen3-32B is approximately 1.24–1.5×

Verdict

candidate for investigation (2:4 with MaskLLM fine-tuning; BSR at $\geq 75\%$ sparsity). Confidence: 6/10. The speedup is modest at 50% sparsity; the quality gate at 2:4 without fine-tuning is high. Combined INT4 + 2:4 is the recommended pilot.

Full analysis: `research_5_8_block_sparse_weights.md` in the research corpus.

3.39 Idea 5.9: Dynamic Per-Value Numeric Type

Mechanism

Idea 5.9 proposes assigning each weight or activation its own numeric type (FP16, INT8, INT4, etc.) chosen dynamically to minimize memory without sacrificing accuracy. The core challenge is type metadata overhead: storing a 2-bit type tag per 4-bit weight incurs 50% overhead (+6.75 GB for a 27B model at $K = 4$ types), which substantially erodes the compression benefit. At $K = 2$ types with 1-bit tags, the overhead is 3.375 GB—still a significant fraction of the 13.5 GB INT4 footprint.

No hardware accelerator as of knowledge cutoff (August 2025) supports per-element heterogeneous numeric format natively. NVIDIA Blackwell NVFP4 represents the finest hardware-supported granularity at 16-element micro-blocks with a shared format. True per-element type mixing requires SIMT-core execution ($\sim 500\times$ lower throughput than Tensor Core), making it viable only as a storage format (compress at rest, dequantize to uniform before compute).

The actionable reframing is per-block or per-group dynamic type assignment. The SpQR (Dettmers et al. ICLR 2024 [45]) format implements the per-group outlier variant (binary type flag, $\sim 1\%$ of weights in FP16 sparse CSR), achieving near-lossless 3-bit compression with 15% TPOT improvement on a 33B model on a single 24 GB GPU. OWQ (Lee et al. AAAI 2024 [219]) protects 0.15% of weight columns in FP16 at 3% latency overhead—the finest published per-structure mixed precision.

Prior Art and Related Work

LLM.int8 (Dettmers et al. NeurIPS 2022 [43]) implements per-channel mixed precision (outlier dimensions in FP16, remainder in INT8). SpQR ([45]) and OWQ ([219]) extend to finer granularities. SqueezeLLM ([208]) implements per-element unstructured sparse outlier storage. QuIP# ([308])

achieves near-lossless 2-bit via block codebook vector quantization, avoiding per-element type metadata entirely. MixLLM (Li et al. 2024 [227]) assigns bit-widths per output feature globally. The 2025 survey on mixed-precision quantization ([260]) confirms per-scalar type assignment is not published as of October 2025. Missing high-priority citation: HAQ (Wang et al. CVPR 2019 [314]) for automated per-layer precision NAS; SqueezeLLM (added).

Complexity Analysis

Table 37: *Idea 5.9 type metadata overhead for a 27B model at base 4-bit.*

Type tag size	Tags (1-bit)	Tags (2-bit)	INT4 baseline	Net memory
Per-element	3.375 GB	6.75 GB	13.5 GB	16.9 GB or 20.25 GB
Per-group (g=16)	0.21 GB	0.42 GB	13.5 GB	13.7 GB or 13.9 GB
Per-group (g=128)	0.026 GB	0.053 GB	13.5 GB	≈13.5 GB

Corrections from the review: (1) A1 memory at $b_{\text{total}}=5$ bits is ~ 17 GB, not 20 GB as stated in the source; (2) A2 KV cache at 32K context is ~ 8.6 GB .

Verdict

candidate for investigation (reframe as per-block/per-group type assignment). Confidence: 5/10. The per-scalar formulation is hardware-impractical. The per-group variant (SpQR/OWQ style) is well-validated and worth piloting on Qwen3-class models using Triton kernels.

Full analysis: `research_5_9_dynamic_numeric_type.md` in the research corpus.

3.40 Idea 5.10: Block-Level Dynamic Type Compression

Mechanism

Idea 5.10 generalizes Idea 5.9 to the block level: a block of B weights shares one scale/zero-point header (amortizing scale metadata to b_{scale}/B bits per element), while each element stores its own type flag (b_{type} bits per element, fixed regardless of B). The effective bits per weight are:

$$b_{\text{eff}} = \frac{b_{\text{scale}}}{B} + b_{\text{type}} + b_w(1 - f_{\text{out}}) + b_{\text{high}} \cdot f_{\text{out}}$$

At $B=32$, $b_{\text{scale}}=8$ (E8M0), $b_{\text{type}}=2$, $b_w=4$, $b_{\text{high}}=16$, $f_{\text{out}}=0.01$: $b_{\text{eff}}=6.37$ bits/weight—worse than standard INT4 group quantization (4.125 bits). For the scheme to achieve $b_{\text{eff}} \approx 4.4$ bits, the type flag must be amortized *at the block level* (e.g., one type indicator per block, not per element)—which converges to the MX+-style design.

MX+ (Lee et al. MICRO 2025 [218]) is the closest single paper to Idea 5.10: it gives the block-maximum element elevated precision (E2M3 vs E2M1) using an 8-bit index per 32-element block (0.25 bits/element overhead), achieving a $b_{\text{eff}}=4.50$ bits/weight and reducing PPL from 27.38 to 9.54 on

Llama-3.1-8B. MicroMix (Liu et al. 2025 [238]) applies per-channel (not per-element) mixed MX formats on Blackwell, achieving $2.29\text{--}3.38\times$ acceleration on Qwen2.5-32B.

The two-level scheme (binary outlier flag) as implemented by SpQR ([45]) and SqueezeLLM ([208]) achieves $b_{\text{eff}} \approx 4.6\text{--}4.7$ bits with validated LLM results. These are the baseline against which Idea 5.10’s k-level generalization must compete.

Prior Art and Related Work

SpQR ([45]) and SqueezeLLM ([208]) implement the binary ($k = 2$) case. QLoRA NF4 (Dettmers et al. NeurIPS 2023 [44]) provides the block-level scale amortization framework (double quantization, block = 64). GPTQ ([174]) and AWQ ([231]) are the uniform-type baselines that the k-level scheme must beat on b_{eff} . MX format specification (Rouhani et al. 2023 [279]) formalizes shared-exponent block quantization at $B=32$. MX+ ([218]) and MicroMix ([238]) are the closest hardware-validated implementations. LLM.int8 ([43]) and OWQ ([219]) are foundational outlier decomposition references, both missing from the source document as flagged by the review.

Complexity Analysis

Table 38: *Idea 5.10 effective bit width and TPOT improvement at A2 scale (32B).*

Variant	b_{eff}	A2 weight mem.	A2 TPOT	Status
SpQR (k=2 binary)	~ 4.6 bits	~ 18.4 GB	$\sim 3.5\times$	Prior art
MX+ (block-max elev.)	4.50 bits	~ 18.0 GB	$\sim 3.6\times$	Prior art
k=4 per-elem type	6.37 bits	~ 25.5 GB	$\sim 2.5\times$	NOT competitive
Interleaved binary	~ 5.4 bits	~ 21.6 GB	$\sim 3.0\times$	Research

Three quantitative errors from the review: (1) Baseline B weight memory at 4.4 bits is ~ 218 GB (not 109 GB; the 109 GB figure implies 2.2 bits/weight); (2) A2 KV cache at 32K is ~ 8.6 GB ; (3) MX+ b_{eff} is 4.50 (not 4.41 as stated). Dequantization overhead at batch = 1 decode is $\sim 7.4\%$ for A1 (not “negligible”).

Verdict

candidate for investigation (conditional). Confidence: 6/10. The binary ($k = 2$) block-level variant has strong prior art support (SpQR, SqueezeLLM, MX+). The $k \geq 3$ per-element generalization is novel but bit-inefficient in the current formulation and requires redesign. The interleaved block layout (vs. separate CSR buffer) is a genuine engineering contribution worth prototyping.

Full analysis: research_5_10_block_dynamic_compression.md in the research corpus.

4 Group 6: New Architectural Proposals from Synthesis

The four ideas in Group 6 were not part of the original 31-idea corpus. They emerged from the cross-idea synthesis phase, motivated by gaps identified in the prior-art analysis, combinatorial synergies surfaced by the cross-reference matrix, and the observation that certain architectural primitives (learned halting, prefill/decode structural specialization, block-diffusion generation) had not been explored in the context of large hybrid recurrent-attention models. All four were developed, researched, and reviewed with the same multi-role Myrmidon Swarm methodology applied to Groups 1–5.

4.1 Idea 6.1: In-Architecture Autoregressive Loop with Stop-Token Break

Motivation

Standard large language model inference drives autoregressive token generation from an external loop: a host program repeatedly calls the model’s forward pass, samples a token, appends it to the context, and checks whether an EOS or stop token has been emitted. This external loop is invisible to the model during training. The model learns to emit an EOS token via supervised cross-entropy on training examples, but it does not learn *how many steps to take* as a first-class optimization objective; stopping is incidental to the next-token prediction loss.

Idea 6.1 proposes treating the autoregressive decoding loop as a first-class component of the model’s computational graph. The model is augmented with an internal recurrent loop module – a small GRU cell or lightweight MLP – that maintains a loop-state vector h_{loop} across generation steps and produces a halting probability $p_{\text{halt}}(t)$ at each step. The model learns to emit a special `<|stop|>` token when generation should cease, driven by the learned halt signal rather than external heuristics. During training, the loop unrolls for T steps with truncated backpropagation through time (BPTT) or with REINFORCE-style policy gradient if the stop decision is discrete, allowing the model to optimize not just *what* to generate but *how long* to generate.

The practical motivation is twofold. First, external stopping heuristics (`max_tokens`, confidence thresholds, external classifiers) are disconnected from the model’s representation; they cannot adapt to the semantic content of the generation in progress. Second, models trained only on next-token prediction with supervised EOS positions may learn stopping behaviors that are brittle at distribution shift – they stop where examples in the training set stopped, not where stopping is optimal for the task.

Mechanism

The loop module consists of three components: a loop-state initialization head, a per-step state update function, and a halting projection. At the start of generation, the loop-state $h_0 \in \mathbb{R}^{d_{\text{loop}}}$ is computed from the final hidden state of the prompt encoding (a linear projection from the last transformer layer’s output at the last prompt token). At each generation step t , after the model’s standard L -layer forward pass produces hidden state $z_t \in \mathbb{R}^d$, the loop-state update computes $h_t = \text{GRU}(h_{t-1}, z_t)$, or equivalently a one-layer MLP with residual connection. The

halting projection then computes $p_{\text{halt}}(t) = \sigma(W_{\text{halt}} \cdot h_t) \in (0, 1)$, a scalar probability of stopping after this step.

For the continuous halting variant (PonderNet/ACT-style [8, 185]), the model’s output at each step is a weighted combination across all steps, weighted by the geometric stopping distribution. Generation terminates when the accumulated halting probability exceeds a threshold (e.g., 0.9), or at a hard maximum T . Gradients flow through $p_{\text{halt}}(t)$ via the sigmoid and through the weighted output combination. For the discrete stop-token variant, the model’s vocabulary includes a `<|stop|>` token; if $p_{\text{halt}}(t) > 0.5$ or via sampling, the model emits `<|stop|>` and the loop terminates. The discrete decision can be trained via Straight-Through Estimation or via REINFORCE with a learned value baseline. Importantly, these two variants are distinct mechanisms with different gradient properties; the continuous halting variant is recommended for training stability.

Training proceeds via truncated BPTT with a window W (typically 32–128 steps). The training objective combines the standard next-token cross-entropy loss at each step with a halting regularization term: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{halt}}$, where $\mathcal{L}_{\text{halt}}$ penalizes excessively long or short generation (e.g., KL divergence between the stopping distribution and a prior geometric distribution with mean equal to the target generation length for the training example). The λ coefficient balances generation quality against stopping efficiency.

The GRU-based approach for the Gated DeltaNet layers in A1 and B requires careful gradient management. Each outer AR loop step adds one DeltaNet rank-1 update per layer during generation. BPTT through T outer steps requires propagating gradients through T sequential DeltaNet rank-1 updates per layer, creating a doubly-nested gradient chain. The DeltaNet delta rule $W_t = W_{t-1}(I - k_t k_t^\top) + v_t k_t^\top$ has Jacobian $dW_{t-1}/dW_t = (I - k_t k_t^\top)$, whose spectral norm is typically less than one after normalization, leading to gradient norm decay over T steps. Gradient clipping (norm ≤ 1.0) and potentially freezing DeltaNet weights during initial loop-module training are the recommended mitigations.

Prior Art and Related Work

The idea builds directly on two foundational works. **Adaptive Computation Time** (Graves, 2016 [185]) introduced differentiable halting for recurrent neural networks, allowing an RNN to decide when to stop computing before emitting each output. **PonderNet** (Banino et al., 2021 [8]) generalized ACT with a cleaner geometric prior formulation and improved training stability. **Universal Transformers** (Dehghani et al., 2019 [40]) integrated ACT into the transformer architecture, applying per-*token* halting across depth iterations of a weight-tied transformer – the closest structural prior art, differing from Idea 6.1 in that Universal Transformers halt depth-wise per token position, while Idea 6.1 halts sequence-wise.

The most direct prior art is **Looped Transformers for Length Generalization** (Fan et al., 2024 [163]), which trains a looped transformer with an adaptive number of loop steps and shows that adaptive depth significantly improves length generalization on algorithmic tasks. This paper establishes that training the loop count adaptively in a transformer has been done, applied to algorithmic tasks rather than large-scale hybrid AR LLMs. **RWKV** (Peng et al., 2023 [268]) is relevant as a recurrent language model context. Speculative decoding approaches (Medusa [19], EAGLE [223]) and early-exit papers address related acceleration concerns but are orthogonal: they

optimize the *cost* of each step rather than the *count* of steps.

Complexity Analysis

The internal loop module adds one GRU cell or MLP forward per generation step: $O(d \times d_{\text{loop}})$ additional FLOPs. With $d_{\text{loop}} = d$, this is $O(d^2)$ per step – negligible against the $O(L \cdot d^2 + L \cdot d \cdot d_{\text{mlp}})$ baseline per-step cost.

Table 39: Complexity impact of Idea 6.1 internal loop module. All KV cache values use canonical formulas; loop-state memory is per batch element.

Metric	A1 (27B Hybrid)	A2 (32B Dense)	B (397B MoE)
Module overhead (FLOPs/step)	+26.2M (+0.18%)	+26.2M (+0.12%)	+16.8M (<0.2%)
KV cache @32K	2.15 GB (unchanged)	8.59 GB (unchanged)	1.0 GB (unchanged)
Loop-state memory	10 KB	10 KB	8 KB
Training activation overhead	$+W \times 42 \text{ MB/elem.}$	$+W \times 42 \text{ MB/elem.}$	$+W \times 32 \text{ MB/elem.}$

At BPTT window $W = 64$ with gradient checkpointing, each window requires storing W activation checkpoints at $O(L \cdot d)$ each: approximately 42 MB per batch element per window for A1 and A2, 32 MB for B. At batch size 32 with A1, the additional activation memory is approximately 2.7 GB – manageable alongside the ~ 54 GB model weight memory.

Verdict

The idea warrants investigation. The real incremental contribution is applying trained halting to large hybrid AR models with Gated DeltaNet recurrent layers — a context not addressed in prior work. Inference overhead is genuinely negligible (<0.2% FLOPs per step). Recommended path: prototype on A2 (pure attention) first; validate stopping quality; then extend to A1/B with DeltaNet gradient management. Publication strategy should frame this as an engineering contribution — scaling trained halting to large hybrid transformer-RNN models — rather than a new theoretical mechanism.

Full analysis: `research_6_1_inarch_ar_loop.md` in the research corpus.

4.2 Idea 6.2: Prefill/Decode Architectural Split

Motivation

Standard large language model inference uses a single transformer architecture for both the prefill phase (processing the input context in parallel) and the decode phase (generating tokens autoregressively one at a time). These two phases operate in fundamentally different computational regimes: prefill is compute-bound at large batch sizes and long sequences; decode is memory-bandwidth-bound at batch size 1 or small batch. A single shared architecture cannot be simultaneously optimized for both regimes.

Serving-level prefill/decode disaggregation (Splitwise [267], DistServe [351], SARATHI [2]) separates the phases onto different hardware without changing the model. Idea 6.2 goes further: it proposes

building two structurally distinct sub-networks that are co-trained but independently optimized for their respective computational regimes. The departure from existing work is that this split occurs at the **model architecture level**, not merely at the serving or scheduling level.

Mechanism

The **prefill sub-network** (L_{pre} layers) maximizes parallel throughput: wider layers (larger d_{ff} or more layers), higher FLOPs tolerance, potentially more attention heads or longer attention windows, and optimization for compute-bound operation at large batch sizes. The **decode sub-network** (L_{dec} layers, with $L_{\text{dec}} < L_{\text{pre}}$) minimizes per-step latency: fewer or narrower layers, smaller KV cache footprint, potentially linear/recurrent attention (DeltaNet) to eliminate KV cache growth, and optimization for memory-bound operation at small batch.

During inference, the prefill sub-network runs once on the full input context and passes a **compressed state vector** C to the decode sub-network. For hybrid models (A1, B), the DeltaNet recurrent layers produce a fixed-size matrix-valued state $O(d_k \times d_v)$ per layer – a constant-size summary of unlimited context that is the natural compressed interface. For dense model A2, a learned cross-attention pooling layer reduces the $(s \times d)$ prefill output to $(k \times d_{\text{state}})$ summary tokens ($k = 128$). The decode sub-network then drives autoregressive token generation, consuming this compressed state alongside its own growing KV cache (for generated tokens only).

Training is fully differentiable: every forward pass routes tokens $1 \dots s$ through the prefill sub-network, produces the compressed state, then generates logits via the decode sub-network. Backpropagation flows from the loss through decode, through the compressed state projection, and into the prefill sub-network. An auxiliary reconstruction loss can optionally enforce rich compressed state representations.

Prior Art and Related Work

The serving-level disaggregation papers (Splitwise [267], DistServe [351], SARATHI [2]) motivate the proposal by quantifying the compute regime differences but do not change the model architecture. The architecturally closest existing work is **YOCO: You Only Cache Once** (Sun et al., 2024 [296]), which proposes a self-decoder + cross-decoder architecture with “prefill early exit.” Key differentiator: YOCO does not propose regime-specific optimization (widening the self-decoder, narrowing the cross-decoder) or co-training with explicit compute-regime targets. Encoder-decoder models (T5, BART) establish structural duality as a conceptual ancestor, but use bidirectional encoders and do not optimize sub-networks for their respective computational regimes. DeepSeek-V2 Multi-head Latent Attention compresses KV to a latent vector within a single unified model – related to the compressed state interface but not across two distinct sub-networks.

Complexity Analysis

With the decode sub-network using $L_{\text{dec}} = L/2$ layers, KV cache is reduced by exactly 50% (KV is proportional to the number of layers storing KV). If the decode sub-network uses only DeltaNet (linear attention), the prefix KV cache drops to zero; only generated tokens accumulate KV entries.

Table 40: *KV cache and TPOT analysis for Idea 6.2 split architecture. All values at 32K tokens, bf16.*

Metric	A1 Full	A1 Split	A2 Full	A2 Split
KV cache @32K (full-attn only)	2.15 GB	1.08 GB	8.59 GB	4.30 GB
KV cache (DeltaNet decode)	2.15 GB	~0 GB (prefix)	8.59 GB	~0 GB (prefix)
Decode per-step FLOPs (relative)	1.0×	0.5×	1.0×	0.5×
Estimated TPOT (BW-bound, batch=1)	1.0×	~0.5–0.6×	1.0×	~0.59×
Parameter overhead	baseline	+30%	baseline	+31%

The break-even for the A2 split at $s = 32K$: the weight memory overhead is +20 GB (fixed); KV savings per request at $s = 32K$ is 4.29 GB. Break-even: $N = 20/4.29 \approx 5$ concurrent requests. For $N > 5$ concurrent long-context requests, the split architecture uses less total HBM than the full model.

Verdict

The ~50% KV cache reduction and ~40–50% TPOT improvement are commercially significant. The partial novelty is sufficient for a research paper contribution given clear differentiation from YOCO. Technical risk is moderate and can be mitigated by empirical validation at small scale. Natural alignment with the DeltaNet recurrent state mechanism already present in A1 and B baselines makes this the most immediately exploitable of the new proposals.

Full analysis: `research_6_2_prefill_decode_split.md` in the research corpus.

4.3 Idea 6.3: Block-Diffusion Autoregressive Decoder

Motivation

Standard autoregressive generation emits one token per forward pass. Full discrete diffusion language models (MDLM [282], LLaDA [262], SEDD [245]) generate all tokens simultaneously via iterative denoising, enabling joint token modeling but at high computational cost for long sequences. The Block-Diffusion Autoregressive Decoder proposes an intermediate regime: emit a block of k tokens per step via a discrete diffusion process within the block, while maintaining autoregressive left-to-right dependencies across blocks.

Mechanism

For block $b = 0, 1, \dots, T/k - 1$:

1. Initialize block b 's k token positions with fully masked tokens.
2. For denoising step $d = 0, 1, \dots, D - 1$: apply the transformer forward pass over the k masked positions (bidirectional attention *within* the block, causal attention to prior blocks), predict clean token distributions, and update positions via the denoising schedule.
3. After D denoising steps: commit block b 's k clean tokens to the KV cache.

Throughput gain requires $D < k$: at $k = 8$, $D = 4$, there are $T/2$ total forward passes vs. T for standard AR – a $2\times$ reduction. Total FLOPs increase (by factor D relative to single-token AR) because each block-diffusion pass processes k tokens; in the *memory-bandwidth-bound* regime (typical for batch size 1 decode), wall-clock time is determined by forward pass count, not FLOPs count, yielding the $2\times$ speedup. In the compute-bound regime (large batch), block-diffusion is approximately $2\times$ *slower* than standard AR.

The training objective is a block-level ELBO over masked diffusion, conditioning on clean left context (prior blocks in KV cache). Data-driven noise schedules and variance-reduction estimators are required for stable training; the BD3-LM paper provides these in detail.

Prior Art and Related Work

The core mechanism was independently developed and published as **BD3-LM (Block Discrete Denoising Diffusion Language Models)** (Arriola et al., ICLR 2025 Oral [4]). BD3-LM establishes the identical mechanism (AR across blocks, discrete diffusion within blocks), derives the block-level ELBO training objective, provides variance-reduction estimators and data-driven noise schedules, implements KV caching across blocks for efficient inference, and achieves state-of-the-art performance among discrete diffusion models. The MDLM paper (Sahoo et al., NeurIPS 2024 [282]) includes a semi-autoregressive generation mode that is the direct conceptual predecessor, sharing authors with BD3-LM. LLaDA [262] and SEDD [245] provide additional diffusion LM foundations.

Complexity Analysis

Peak KV memory is **identical** to standard AR (within-block denoising reuses prior-block KV read-only; prior-block KV is committed once per block). The DeltaNet recurrent state (A1: ~ 2.52 GB; B: ~ 1.51 GB) is a constant additional cost independent of sequence length.

Table 41: *Throughput comparison for Idea 6.3 at $k = 8$, $D = 4$. BW-bound = memory-bandwidth-bound (batch=1); compute-bound = large batch.*

Metric	Standard AR	Block-Diffusion ($k=8$, $D=4$)
Forward passes for T tokens	T	$T/2$ ($2\times$ fewer)
Total FLOPs (normalized)	$T \cdot C$	$4T \cdot C$ ($+4\times$ FLOPs)
Wall-clock (BW-bound, batch=1)	$T \cdot t$	$(T/2) \cdot t$ ($2\times$ faster)
Wall-clock (compute-bound, large batch)	$T \cdot t$	$\sim 2T \cdot t$ ($2\times$ slower)
Peak KV memory (A1/A2/B)	2.15/8.59/1.0 GB	Identical

Note: The original idea description incorrectly stated “ $4\times$ fewer forward passes at $k = 8$, $D = 4$ ”; the review-verified correct value is $2\times$ fewer. Four times fewer passes requires $D = 2$, not $D = 4$.

Verdict

The core mechanism is fully published and peer-reviewed as BD3-LM (ICLR 2025 Oral). No novel component remains in the base concept. This idea is lower-priority as a standalone contribution. A conditional path exists: reframe as “BD3-LM applied to hybrid recurrent-attention architectures

(DeltaNet + MoE),” specifically investigating whether DeltaNet’s fixed-size recurrent state can replace the $O(s \cdot d)$ KV cache overhead for long-context block-diffusion. This reframing would be a valid candidate for investigation as an extension of BD3-LM.

Full analysis: research_6_3_block_diffusion_ar_decoder.md in the research corpus.

4.4 Idea 6.4: Combined In-Architecture AR Loop + Prefill/Decode Split + Block-Diffusion Decoder

Motivation

Ideas 6.1, 6.2, and 6.3 address three orthogonal dimensions of the autoregressive generation problem: learned stopping (6.1), regime-specialized sub-networks (6.2), and block-parallel generation (6.3). Idea 6.4 proposes combining all three into a single unified generation system, with the central research question being whether the benefits compose multiplicatively or interact destructively.

The answer, argued quantitatively below, is that the benefits of Ideas 6.2 and 6.3 are **super-additive in the multiplicative sense**: because they act on independent dimensions (layer count and forward-pass count, respectively), their FLOP reductions multiply. Idea 6.1 adds learned stopping with no additional FLOP cost. The combination is genuinely more than the sum of its parts.

Mechanism

The unified architecture has four components:

Prefill sub-network (from Idea 6.2): The first $L/2$ layers process the full input context in parallel and produce a compressed state. For hybrid models (A1, B), the DeltaNet recurrent layers produce a fixed-size state (~ 100 MB for A1, ~ 94 MB for B), independent of input length. For A2, a learned pooling head compresses to $N_s = 512$ summary tokens (~ 512 MB).

Decode sub-network (from Idea 6.2): The remaining $L/2$ layers perform all token generation, initialized from the prefill state. Cross-attention to the prefill state provides access to full input context; a standard KV cache grows only with generated blocks, not with input length.

Block-diffusion generation (from Idea 6.3): Each decode step generates a block of $k = 8$ tokens via $D = 4$ denoising steps of masked discrete diffusion. The attention mask structure has three scopes: bidirectional over the k positions of the current block (within-block), causal over all prior completed blocks (cross-block), and frozen cross-attention to the prefill state (prefill-context). Per the review’s correction, at $D = 4$, $k = 8$, there are $T/2$ total decode forward passes vs. T for standard AR.

In-architecture stopping (from Idea 6.1): A $\langle \text{stop} \rangle$ token is a first-class vocabulary token. After each block’s D denoising steps complete, the output is scanned for $\langle \text{stop} \rangle$; if found at position p , tokens $[0, p)$ from that block are output and generation halts. Detection is block-level (after denoising completes), not mid-denoising, which is reliable and avoids spurious stop signals from intermediate noisy states.

Prior Art and Related Work

All component ideas have published prior art at top venues. The *combination* is the novel contribution: no paper integrates in-weights prefill/decode parameter split, block-diffusion decoding, and learned in-architecture stopping. Directly relevant component papers: PonderNet [8], ACT [185] (learned stopping ancestors); Splitwise [267], DistServe [351] (serving disaggregation motivation); YOCO [296] (closest model-level split prior art); BD3-LM [4] (block-diffusion published implementation); MDLM [282], LLaDA [262], SEDD [245] (diffusion LM foundations).

Complexity Analysis

For the balanced configuration ($D = 4$, $k = 8$, $L_d = L/2$), the FLOP ratio vs. baseline is:

$$\text{FLOP ratio} = \frac{D}{k} \times \frac{L_d}{L} = \frac{4}{8} \times \frac{1}{2} = 0.25 \quad (4 \times \text{ cheaper})$$

This ratio is correct for the **memory-bandwidth-bound** decode regime (typical at batch size 1), where parameter loading dominates. For large-batch compute-bound inference, the attention FLOP ratio is $(D \times L_d)/L = 2$ (worse than baseline from block-diffusion’s increased total attention cost); the benefit reduces to approximately $2\times$ from the prefill/decode split alone.

Table 42: Synergy table for Idea 6.4 component combinations. FLOP ratio is for BW-bound (batch=1) decode.

Combination	FLOP Ratio	KV Reduction	Stopping	Verdict
6.2 only	0.50×	50%	External	Good
6.3 only	0.50×	0%	External	Good
6.1 + 6.2	0.50×	50%	In-arch	Good
6.2 + 6.3	0.25×	50%	External	Excellent
6.1 + 6.2 + 6.3	~0.25×	50%	Block-level	Excellent

Per-baseline KV cache for the decode sub-network: A1: ~ 1.07 GB at 32K output tokens; A2: ~ 4.29 GB at 32K; B: ~ 0.50 GB at 32K – all approximately 50% reduction vs. baseline, with KV growing by $k = 8$ tokens per block ($8\times$ lower KV append frequency).

Verdict

The mathematical validity of the benefit multiplication is confirmed for the bandwidth-bound regime. The literature gap is real. Each component has published validation at top venues, and the combination inherits their maturity. The primary risks (quality at $D = 4$ denoising steps, prefill-to-decode state transfer fidelity, training curriculum complexity) all have known mitigation strategies. The recommended pilot path uses A1 (Qwen3.5-27B Hybrid) as the starting model, leveraging its natural DeltaNet recurrent state interface, with the Fast-dLLM v2 conversion recipe applied to the decode sub-network.

Full analysis: research_6_4_combined_ar_loop_split_diffusion.md in the research corpus.

4.5 Idea 6.5: Pre-Attention Expert Router (Routing on Q/K/Activations)

Standard MoE transformers evaluate the FFN expert router on the *post-attention* residual stream, so routing decisions are informed by contextualised, attention-aggregated representations. Idea 6.5 relocates this routing decision to operate *before* or *in parallel with* the attention sublayer, using one of five signal variants: (V1) pre-attention residual $\mathbf{x}$; (V2) Q projection $W_Q\mathbf{x}$; (V3) K projection $W_K\mathbf{x}$; (V4) summed $(W_Q + W_K)\mathbf{x}$; (V5) joint attention-routing. The router then selects FFN experts based on the token’s local, non-contextualised representation.

Novelty. Prior art review confirmed this is novel for FFN expert routing across all five variants. SwitchHead [arXiv:2312.07987]: mixture-of-experts applied to attention heads using Q/K signals; the authors found Q/K routing unnecessary for attention head selection and did not apply it to FFN experts. No published MoE places its FFN router on Q/K projections.

Computational impact. Router FLOPs are $O(d \cdot E_{\text{total}})$ regardless of placement. In the V1 parallel-branch variant, router computation is fully hidden under attention FLOPs. Total TTFT and TPOT relative to baseline are unchanged ($\approx$ ref) for all four baselines; the change is purely topological.

Training stability. Variants V2/V3/V4 route on $W_Q\mathbf{x}$ or $W_K\mathbf{x}$, creating a gradient path from routing loss back through the Q/K projection matrices. A stop-gradient $\text{sg}(\cdot)$ applied to the router input prevents routing loss from corrupting attention learning. V1 carries no such risk.

Baseline comparison. For all four baselines (A1, A2, B, C), the pre-attention router adds zero net FLOPs in the parallel variant (V1) and $O(d \cdot E)$ on the critical path in the serial-before variant (V2–V4). At $d = 8,192$ (Baseline C) and $E = 64$ experts, router FLOPs are $\approx 5.2 \times 10^5$ per token — less than 0.1% of MLP FLOPs.

Verdict. NOVEL — a candidate for investigation. The zero-cost parallel variant (V1) is a pure A/B ablation against standard post-attention routing with no architectural overhead.

Full analysis: research_6_5_pre_attention_expert_router.md in the research corpus.

5 Cross-Idea Synthesis and Priority Ranking

5.1 Top-Priority Ideas for Immediate Implementation

The priority ranking formula

$$\text{Priority} = \frac{\text{InferenceImpact} \times \text{Feasibility}}{\text{Risk} \times \text{QualityPenalty}}$$

applied across all 39 ideas produces the ranking shown in Table 43. The top-ranked ideas for immediate action are:

1. **Idea 5.7 (Priority 5.00):** Block-level INT4 weight quantization. Post-hoc, zero retraining, $\sim 2.8\times$ TPOT improvement on all three baselines. Deploy via AutoAWQ or llm-compressor + vLLM in days.
2. **Idea 1.3 (Priority 3.00):** Per-layer adaptive expert count for Baseline B. Static calibration, zero retraining, $\sim 1.4\text{--}1.56\times$ TPOT on the MoE model. Implement via LExI/Alloc-L-style static layer budget in days to weeks.
3. **Idea 2.2 (Priority 1.20):** Post-hoc SVD compression of dense layers. SVD-LLM or CALDERA style; $\sim 4\text{--}6\times$ MLP weight bandwidth reduction at conservative compression ratios.
4. **Idea 5.1 (Priority 2.67):** TurboQuant INT4 KV cache quantization. Most impactful at long context ($\geq 40\text{K}$ tokens) where KV bandwidth approaches weight bandwidth on A2.

Ideas 6.2 and 6.4 (high-priority verdicts) are the highest-priority *research* investments, but require training infrastructure and are not post-hoc deployable.

5.2 Cross-Idea Synergies

The cross-reference matrix identified four primary synergy clusters:

MoE Sparsity Amplification Cluster (1.1, 1.3, 5.7, 5.8): All four ideas reduce active expert weight bandwidth, the dominant TPOT bottleneck for Baseline B. Savings multiply: $5.7 \times 5.8 \times 1.3 \times 1.1 \approx 17.6\times$ theoretical weight bandwidth reduction, approximately $6\text{--}10\times$ practical TPOT improvement accounting for overheads.

Dense Model Long-Context Stack (5.7, 5.1, 5.4, 2.2): All operate on Baseline A2 with no architectural changes. 5.7 attacks weight bandwidth ($\div 4$); 5.1 attacks KV bandwidth ($\div 4$, most impactful above 32K tokens); 5.4 further evicts KV entries after 5.1 compresses them; 2.2 provides a further $\sim 20\text{--}25\%$ weight footprint reduction. Combined TPOT at 32K context: approximately $3\text{--}4\times$ (weights dominate at 32K; KV becomes codominant only at A2’s native maximum of $\sim 40\text{K}$ tokens).

KV Cache Elimination Stack (5.2, 5.4, 5.5, 5.1): For A2 where KV growth is the long-context bottleneck, LSTM-gated attention (5.2) eliminates KV on converted layers entirely; KV eviction (5.4) and ragged window attention (5.5) manage residual KV on remaining full-attention layers; TurboQuant (5.1) compresses survivors.

Depth Reduction Stack (1.2, 4.5, 4.2): Per-token adaptive depth (1.2) and static layer pruning (4.5) both reduce effective layer depth at inference; shared weights (4.2) reduce storage cost further. The Recursive Gemma result (Bae et al. 2025, $2.66\times$ throughput) validates the combination of weight sharing and early exit.

New Proposal Synergies (Group 6): Ideas 6.2 and 6.3 combine super-additively as shown in Table 42: each provides $2\times$ FLOP reduction on independent dimensions (layer count and forward-pass count), yielding $4\times$ combined. Idea 6.1 contributes learned stopping at negligible overhead.

5.3 Phase 1 Implementation Specification Summary

Phase 1 covers zero-retraining post-hoc ideas deployable on existing model checkpoints. The implementation order, motivated by priority score and stacking compatibility, is:

1. **Step 1 – Idea 5.7** (Priority 5.00): INT4 block weight quantization on all three baselines. 1 GPU-hour calibration via GPTQ/AWQ. Immediately yields $\sim 2.8\times$ TPOT. Deploy A1/B with INT4; evaluate A2 with SpQR (5.10) for superior outlier handling.
2. **Step 2 – Idea 5.1** (Priority 2.67): Post-hoc TurboQuant INT4 KV cache on A2. Most impactful at $s \geq 32K$ tokens. KV footprint: ~ 8.59 GB $\rightarrow \sim 2.15$ GB at 32K. Stacks orthogonally with Step 1.
3. **Step 3 – Idea 5.4**: KV eviction (H2O/SnapKV) on top of 5.1. At 10% retention: ~ 2.15 GB $\rightarrow \sim 215$ MB. Risk: quality cliff at very low retention on retrieval-intensive tasks (needle-in-haystack); DuoAttention-style hybrid mitigation recommended.
4. **Step 4 – Idea 1.3**: Per-layer expert count calibration for Baseline B. Zero retraining, static LExI/Alloc-L style. $\sim 1.4\text{--}1.5\times$ additional TPOT.
5. **Step 5 – Idea 5.8** (quality-gated): Block sparse weights (2:4 format) on A2 expert weights. Requires quality validation before deployment at 32B+ scale due to PPL regression risk.
6. **Step 6 – Idea 2.2**: Post-hoc SVD compression of MLP weights. Conservative 10–20% compression (80–90% retention) is near-lossless; above 20% compression, quality degrades steeply.

5.4 Summary Table: All 39 Ideas

Table 43 presents all 39 ideas (31 original + 5 new proposals in Group 6) ranked by priority score and verdict.¹

6 Future Work and Implementation Verdicts

This section consolidates the implementation verdicts for all 39 architectural ideas evaluated in this paper. Verdicts were assigned based on prior art classification, complexity analysis, feasibility assessment, and cross-idea synergy analysis conducted via the Myrmidon-Swarm review methodology described in §1.

6.1 Verdict Classification

PURSUE Strong novelty, clear implementation path, positive expected ROI. Recommended for immediate prototyping.

¹P = Pursue, I = Investigate, D = Deprioritize. Full verdicts in §6.

INVESTIGATE Novel framing or partial prior art; key empirical questions remain open. Recommended for targeted ablation studies before full implementation.

DEPRIORITIZE Significant prior art overlap, marginal expected benefit, or high implementation risk without strong novelty advantage. Defer pending progress on higher-priority ideas.

6.2 PURSUE — High-Priority Ideas

ID	Name	Rationale
2.2	Compressed Dense Layers (LoRA-Everywhere)	Strong empirical backing; LoRA universally applicable; 3–5× bandwidth reduction at rank 256 for A2/C baselines.
4.3	LoRA Everywhere	Direct implementation of cross-layer shared low-rank structure; composable with 2.2.
6.2	Prefill/Decode Architectural Split	Splitwise/DistServe validate operational benefit; architectural-level split is the novel angle.

6.3 INVESTIGATE — Candidate Ideas

ID	Name	Rationale
1.1	Learnable Top- k	Novel trained routing; ablation needed vs. fixed- k baseline.
1.2	Per-Token Adaptive Depth	PonderNet/ACT prior art partial; trained-in stop token is novel.
1.3	Per-Layer Adaptive Expert Count	Novel dynamic cardinality; stability risk.
1.4	Learned Dense/Sparse Layer	Novel joint training signal; hardware constraint risk.
1.5	Learned Sparsity Type	Novel meta-routing; interaction with weight layout.
1.6	Learned Layer Type	Closely related to 1.5; joint investigation recommended.
1.7	Dynamic Vocabulary	Zero-overhead framing novel; TTFT overstatement corrected.
2.1	Hierarchical Frequency Dictionary	Lower priority on standalone basis; synergy with 1.7 warrants investigation.

ID	Name	Rationale
3.1	Layer-Level MoE	Existing MoE literature partial; full-block routing is novel angle.
3.2	Swappable Experts	Novel hot-swap mechanism; deployment use case is clear.
3.3	Dynamic Expert Router	Post-training extensibility novel; routing collapse risk.
3.4	Recursive Internal State	PonderNet partial; depth-gating combined with recurrent state is novel.
3.5	Gated Internal DAG	High novelty; significant training complexity.
3.7	Learnable State Machine	FSM conditioning novel; composable with 4.1.
4.1	State Machine Core	Universal FSM conditioning novel; synergy with 3.7.
4.2	Shared Weights + Per-Layer LoRA	Strong theoretical basis; hardware efficiency gap to close.
4.4	Skip-List Layers	Zero-parameter addition; easy A/B test.
4.5	Learned Residual Flow	Novel static-gate pruning; one-time training cost.
4.6	Mixture of Models (MoMoRA)	Ambitious; blockers documented; Phase 1 signal validation recommended.
5.1	TurboQuant	TPOT overstatement corrected; realistic $1.13\times-1.63\times$ benefit confirmed.
5.2	LSTM-Gated Attention	Hybrid precedent (A1/B) validates mechanism; full-KV elimination is novel.
5.3	Grammar/FSM Attention	Constrained decoding market is real; attention sparsity novel.
5.4	Linked Attention	Eviction policy novel; composable with 5.5.
5.5	Ragged Window Attention	Per-token variable lookback novel; kernel complexity manageable.
5.7	Block Compressed Weights	INT4 baseline; more impactful at Baseline C scale (145 GB weights).
5.9	Dynamic Numeric Type	Per-block type selection novel; per-scalar variant not viable.
5.10	Block Dynamic Compression	MX+-style 4.50-bit viable; per-element flag variant not viable.
6.1	In-Architecture AR Loop	Trained-in stop token novel; PonderNet partial prior art.

ID	Name	Rationale
6.4	Combined AR Loop + Split + Diffusion	Synergy plausible; depends on 6.1/6.2/6.3 proving out first.
6.5	Pre-Attention Expert Router	NOVEL for FFN routing; V1 parallel variant is zero-cost A/B.

6.4 DEPRIORITIZE — Lower-Priority Ideas

ID	Name	Rationale
3.6	Recursive Internal DAG	Turing-complete routing; cascading routing-state collapse risk; significant prior art.
3.8	Trainable Activation	SwiGLU already optimal; windowed variant has fundamental flaw; ProSparse undermines sparsity path.
4.7	Compressed Output Dictionary	Benefit corrected upward (vocab fix) but still marginal standalone; pursue as 1.7 dependency.
5.6	Double Attention	Additional attention pass adds real TPOT overhead with modest quality gain.
5.8	Block Sparse Weights	2:4 format hardware support limited outside A100/H100; decompression overhead real.
6.3	Block-Diffusion AR Decoder	BD3-LM (ICLR 2025 Oral) is exact prior art; conditional pursue path: hybrid DeltaNet+MoE application.

6.5 Recommended Implementation Sequence

Based on the above verdicts and cross-idea synergy analysis, the recommended Phase 1 prototype sequence is:

1. **Baseline:** Qwen3-32B Dense (A2) as the ablation base — simplest architecture, all-dense, 64 GB weights.
2. **Step 1:** Apply Idea 4.3 (LoRA-Everywhere) at rank $r = 256$. Expected: $\approx 3.5\times$ weight-bandwidth reduction, $\approx 3.5\times$ TPOT improvement at batch=1.
3. **Step 2:** Add Idea 4.4 (Skip-List Layers) — zero parameters, 5–15% inference speedup, minimal risk.

4. **Step 3:** Ablate Idea 6.5 V1 (Pre-Attention Router, parallel branch) against standard post-attention routing — zero overhead, resolves the key empirical question.
5. **Step 4:** If Step 3 confirms benefit, integrate Idea 3.3 (Dynamic Expert Router) to enable post-training expert extensibility.

7 Conclusion

This report documents the most comprehensive architecture research review attempted for the Myrmidon project: 39 ideas — 31 from the original corpus spanning Groups 1 through 5, and 5 new proposals synthesized during the review process (Group 6) — evaluated against three canonical baselines (A1: Qwen3.5-27B Hybrid, A2: Qwen3-32B Dense, B: Qwen3.5-397B-A17B MoE) using a hierarchical multi-agent review system with five specialized reviewer roles.

The top findings are as follows. For immediate deployment, **Idea 5.7** (INT4 block weight quantization) is the single highest-priority action: post-hoc, zero retraining, $\sim 2.8\times$ TPOT improvement on all baselines in one GPU-hour of calibration. **Idea 1.3** (per-layer adaptive expert count for Baseline B) follows with a 3.00 priority score and similar deployment simplicity. Together, these two ideas alone can improve production serving throughput by a factor of 3–8 $\times$ before any architectural changes are made.

For novel research contributions with production impact, **Ideas 6.2 and 6.4** are the most promising. Idea 6.2 (Prefill/Decode Architectural Split) offers a $\sim 50\%$ KV cache reduction and $\sim 40\text{--}50\%$ TPOT improvement with partial novelty well-differentiated from YOCO. Idea 6.4 (the combined architecture) achieves a theoretically $4\times$ FLOP reduction for small-batch bandwidth-bound decode (conservatively $2\times$ for large-batch) with full novelty — no prior work combines in-weights prefill/decode split, block-diffusion generation, and learned in-architecture stopping. Both ideas are high-priority contributions.

Idea 6.3 was found to be fully anticipated by BD3-LM (Arriola et al., ICLR 2025 Oral) — a result of the review discovering the published version of an independently proposed idea. Rather than a failure, this finding demonstrates the review system’s ability to detect prior art even for novel-seeming ideas and correctly redirect effort. The lower-priority verdict for 6.3 as a standalone contribution is accompanied by a clear conditional path (as a BD3-LM extension to hybrid DeltaNet architectures).

Idea 6.5 (Pre-Attention Expert Router) was confirmed as a genuinely novel contribution for FFN expert routing. The zero-cost parallel V1 variant provides a pure A/B ablation with no architectural overhead, making it a candidate for investigation before committing to larger structural changes.

The five new proposals in Group 6 represent a research agenda for future architecture work. Ideas 6.1 through 6.5 collectively define a design space: *when* to stop generating (6.1), *how* to specialize prefill and decode sub-networks (6.2), *how* to generate multiple tokens per step (6.3), *how* these three combine multiplicatively (6.4), and *when* to route FFN experts (6.5). The combined architecture (Idea 6.4) is the most promising unification of this agenda. It achieves:

- $4\times$ decode-phase FLOP reduction (BW-bound) or $2\times$ (compute-bound) vs. standard AR;

- 50% KV cache reduction via the decode sub-network’s reduced layer count;
- Learned stopping at block granularity ($\pm k - 1 = \pm 7$ tokens) with no external stopping heuristics;
- Unchanged total parameter count (pure layer split from an existing checkpoint).

The recommended pilot path — starting from Qwen3.5-27B Hybrid (A1) with Fast-dLLM v2’s conversion recipe applied to the decode sub-network — provides a 6–12 month path from a pretrained checkpoint to a working prototype, building on the existing BD3-LM implementation and the DeltaNet recurrent state as a natural prefill-to-decode interface. This represents the most commercially significant and technically novel contribution available from the Group 6 ideas, and the most promising direction for continued architecture research in this corpus.

Acknowledgments. This paper was prepared with Claude Code using the Homeric Intelligence infrastructure (<https://homeriointelligence.com>).

A Research Corpus Artifacts

All research artifacts produced by the Myrmidon Swarm review process are stored in the directory `/home/mvillmow/Random/ArchIdeas/research/`. The corpus is organized as follows:

- **Research documents (39 files):** `research_X_Y_*.md` — primary research and mechanism analysis for each idea. Includes one per original idea (Groups 1–5, 31 ideas) plus five new Group 6 proposals (`research_6_1_inarch_ar_loop.md`, `research_6_2_prefill_decode_split.md`, `research_6_3_block_diffusion_ar_decoder.md`, `research_6_4_combined_ar_loop_split_diffusion.md`, `research_6_5_pre_attention_expert_router.md`).
- **Summary documents (39 files):** `summary_X_Y_*.md` — condensed per-idea summaries including verdict, key metrics, and prior art classification.
- **Review documents (39 files):** `review_X_Y_*.md` — multi-role Myrmidon Swarm review reports including citation verification, Big-O auditing, literature gap analysis, comparison validation, and feasibility assessment for each idea, plus:
 - `review_synthesis_docs.md` — Phase 3a validation of all four synthesis documents.
 - `review_summary.md` — final consolidated review summary and per-idea verdict table.
- **Verification files (≥ 175 files):** `verification_X_Y_*.md` — five per-idea verification sub-documents (Citation Verifier, Complexity Auditor, Literature Gap Finder, Comparison Validator, Feasibility Checker), each produced by an independent sub-agent role.
- **Synthesis documents (4 files):**
 - `architecture_synthesis.md` — four composite architecture proposals combining multiple ideas for specific deployment scenarios.

- `cross_reference_matrix.md` — 39×39 synergy/conflict/neutral matrix for all 39 ideas, with annotated S/N/C classifications and justifications.
- `priority_ranking.md` — priority score computation for all 39 ideas using the $\text{InferenceImpact} \times \text{Feasibility} / (\text{Risk} \times \text{QualityPenalty})$ formula.
- `implementation_spec_phase1.md` — Phase 1 implementation specification for zero-retraining post-hoc ideas with exact tooling, configuration parameters, expected outcomes, and acceptance criteria.

References

- [1] Aghajanyan et al. Aghajanyan2021. <https://arxiv.org>, 2021. See text for full citation.
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. <https://arxiv.org/abs/2308.16369>, 2023.
- [3] AroraBASED et al. Arora b a s e d2024. <https://arxiv.org>, 2024. See text for full citation.
- [4] Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Jiaqi Qi, Shen Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. <https://arxiv.org/abs/2503.09573>, 2025. ICLR 2025 Oral. BD3-LM: the published version of Idea 6.3.
- [5] Bae et al. Bae2025 i c l r. <https://arxiv.org>, 2025. See text for full citation.
- [6] Bae et al. Bae2025 relaxed recursive. <https://arxiv.org>, 2025. See text for full citation.
- [7] BaevskiAuli et al. Baevski auli2019. <https://arxiv.org>, 2019. See text for full citation.
- [8] Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [9] BaninoPonderNet et al. Banino ponder net2021. <https://arxiv.org>, 2021. See text for full citation.
- [10] BaykalAltUp et al. Baykal alt up2023. <https://arxiv.org>, 2023. See text for full citation.
- [11] Beck et al. Beck2024x l s t m. <https://arxiv.org>, 2024. See text for full citation.
- [12] BeckXLSTM et al. Beck x l s t m2024. <https://arxiv.org>, 2024. See text for full citation.
- [13] BeltagyLongformer et al. Beltagy longformer2020. <https://arxiv.org>, 2020. See text for full citation.
- [14] Bhatt et al. Bhatt2024 c a l d e r a. <https://arxiv.org>, 2024. See text for full citation.
- [15] Bieber et al. Bieber2020 i p a g n n. <https://arxiv.org>, 2020. See text for full citation.
- [16] BLaST et al. B l a s t2025. <https://arxiv.org>, 2025. See text for full citation.

- [17] BTXRef et al. Btxref. <https://arxiv.org>, 2024. See text for full citation.
- [18] Cai et al. Cai2024 pyramid k v. <https://arxiv.org>, 2024. See text for full citation.
- [19] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. <https://arxiv.org/abs/2401.10774>, 2024.
- [20] CALDERA et al. C a l d e r a2024. <https://arxiv.org>, 2024. See text for full citation.
- [21] CaM et al. Ca m2024. <https://arxiv.org>, 2024. See text for full citation.
- [22] Chen et al. Chen2023 frugal g p t. <https://arxiv.org>, 2023. See text for full citation.
- [23] ChenDiffSoftmax et al. Chen diff softmax2016. <https://arxiv.org>, 2016. See text for full citation.
- [24] ChenFrugalGPT et al. Chen frugal g p t2023. <https://arxiv.org>, 2023. See text for full citation.
- [25] ChenGrangier et al. Chen grangier2016. <https://arxiv.org>, 2016. See text for full citation.
- [26] ChildSparseTransformer et al. Child sparse transformer2019. <https://arxiv.org>, 2019. See text for full citation.
- [27] ChittyVenkata et al. Chitty venkata2025 l e x i. <https://arxiv.org>, 2025. See text for full citation.
- [28] ChungKim et al. Chung kim2025. <https://arxiv.org>, 2025. See text for full citation.
- [29] CMoERef et al. Cmoeref. <https://arxiv.org>, 2024. See text for full citation.
- [30] Composer et al. Composer2024. <https://arxiv.org>, 2024. See text for full citation.
- [31] Robert Csordas, Pethro Pihlaja, and Jürgen Schmidhuber. SwitchHead: Accelerating transformers with mixture-of-experts attention. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2312.07987.
- [32] CSVDecode et al. C s v decode2025. <https://arxiv.org>, 2025. See text for full citation.
- [33] CuiAoA et al. Cui ao a2017. <https://arxiv.org>, 2017. See text for full citation.
- [34] DaoGu et al. Dao gu2024. <https://arxiv.org>, 2024. See text for full citation.
- [35] DaoMonarch et al. Dao monarch2022. <https://arxiv.org>, 2022. See text for full citation.
- [36] De et al. De2024 griffin. <https://arxiv.org>, 2024. See text for full citation.
- [37] DeepSeekAI et al. Deep seek a i2024. <https://arxiv.org>, 2024. See text for full citation.
- [38] DeGriffin et al. De griffin2024. <https://arxiv.org>, 2024. See text for full citation.
- [39] Dehghani et al. Dehghani2019 u t. <https://arxiv.org>, 2019. See text for full citation.

- [40] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. <https://arxiv.org/abs/1807.03819>, 2019.
- [41] DehghaniUniversalTransformer et al. Dehghani universal transformer2018. <https://arxiv.org>, 2018. See text for full citation.
- [42] Dettmers et al. Dettmers2023 sp q r. <https://arxiv.org>, 2023. See text for full citation.
- [43] DettmersLLMint8 et al. Dettmersllmint8. <https://arxiv.org>, 2024. See text for full citation.
- [44] DettmersQLoRA et al. Dettmers q lo r a2023. <https://arxiv.org>, 2023. See text for full citation.
- [45] DettmersSpQR et al. Dettmers sp q r2024. <https://arxiv.org>, 2024. See text for full citation.
- [46] Ding et al. Ding2022 da vi t. <https://arxiv.org>, 2022. See text for full citation.
- [47] DingDaViT et al. Ding da vi t2022. <https://arxiv.org>, 2022. See text for full citation.
- [48] Dong et al. Dong2024 hymba. <https://arxiv.org>, 2024. See text for full citation.
- [49] DynaSpec et al. Dyna spec2024. <https://arxiv.org>, 2024. See text for full citation.
- [50] ElbayadDepthAdaptive et al. Elbayad depth adaptive2020. <https://arxiv.org>, 2020. See text for full citation.
- [51] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anirudh Garg, Ma Ma, Sami Abu-El-Haija, Aly Mahmoud, et al. LayerSkip: Enabling early exit inference and self-speculative decoding. <https://arxiv.org/abs/2404.16710>, 2024. ACL 2024.
- [52] Agostinelli et al. et al. Agostinelli et al2015. <https://arxiv.org>, 2015. See text for full citation.
- [53] AkhavanAghdam et al. et al. Akhavan aghdam et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [54] Arora et al. et al. Arora et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [55] Bae et al. et al. Bae et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [56] Bai et al. et al. Bai et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [57] Banino et al. et al. Banino et al2021. <https://arxiv.org>, 2021. See text for full citation.
- [58] Bengio et al. et al. Bengio et al2015. <https://arxiv.org>, 2015. See text for full citation.
- [59] Bhatt et al. et al. Bhatt et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [60] Chelly et al. et al. Chelly et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [61] Chen et al. et al. Chen et al2019. <https://arxiv.org>, 2019. See text for full citation.

- [62] Cheng et al. et al. Cheng et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [63] ChittyVenkata et al. et al. Chitty venkata et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [64] Choi et al. et al. Choi et al2018. <https://arxiv.org>, 2018. See text for full citation.
- [65] Csordas et al. et al. Csordas et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [66] DaiDeng et al. et al. Dai deng et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [67] Dao et al. et al. Dao et al2022 monarch. <https://arxiv.org>, 2022. See text for full citation.
- [68] Dehghani et al. et al. Dehghani et al2019. <https://arxiv.org>, 2019. See text for full citation.
- [69] Do et al. et al. Do et al2023. <https://arxiv.org>, 2023. See text for full citation.
- [70] Dong et al. et al. Dong et al2024 hymba. <https://arxiv.org>, 2024. See text for full citation.
- [71] Elbayad et al. et al. Elbayad et al2020. <https://arxiv.org>, 2020. See text for full citation.
- [72] Elhoushi et al. et al. Elhoushi et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [73] Fang et al. et al. Fang et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [74] Fedus et al. et al. Fedus et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [75] GaleLeptikhin et al. et al. Gale leptikhin et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [76] Gao et al. et al. Gao et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [77] Gao et al. et al. Gao et al2025 mo l a. <https://arxiv.org>, 2025. See text for full citation.
- [78] Geiping et al. et al. Geiping et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [79] Giannou et al. et al. Giannou et al2023. <https://arxiv.org>, 2023. See text for full citation.
- [80] Gloriosa et al. et al. Gloriosa et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [81] Goel et al. et al. Goel et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [82] Gong et al. et al. Gong et al2024 mo m. <https://arxiv.org>, 2024. See text for full citation.
- [83] Goodfellow et al. et al. Goodfellow et al2013 maxout. <https://arxiv.org>, 2013. See text for full citation.
- [84] Grave et al. et al. Grave et al2017. <https://arxiv.org>, 2017. See text for full citation.
- [85] Graves et al. et al. Graves et al2014 n t m. <https://arxiv.org>, 2014. See text for full citation.
- [86] Graves et al. et al. Graves et al2016 d n c. <https://arxiv.org>, 2016. See text for full citation.

- [87] GuoCheng et al. et al. Guo cheng et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [88] Han et al. et al. Han et al2024 s l train. <https://arxiv.org>, 2024. See text for full citation.
- [89] Hao et al. et al. Hao et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [90] He et al. et al. He et al2015 p re l u. <https://arxiv.org>, 2015. See text for full citation.
- [91] He et al. et al. He et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [92] Heakl et al. et al. Heakl et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [93] Holtzman et al. et al. Holtzman et al2020. <https://arxiv.org>, 2020. See text for full citation.
- [94] Hu et al. et al. Hu et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [95] HuangAn et al. et al. Huang an et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [96] Jain et al. et al. Jain et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [97] Jawahar et al. et al. Jawahar et al2023. <https://arxiv.org>, 2023. See text for full citation.
- [98] Jeddi et al. et al. Jeddi et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [99] JiangWang et al. et al. Jiang wang et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [100] Jin et al. et al. Jin et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [101] Jin et al. et al. Jin et al2025 mo h. <https://arxiv.org>, 2025. See text for full citation.
- [102] Kim et al. et al. Kim et al2024 gate skip. <https://arxiv.org>, 2024. See text for full citation.
- [103] Kimhi et al. et al. Kimhi et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [104] Kohli et al. et al. Kohli et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [105] Komatsuzaki et al. et al. Komatsuzaki et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [106] Lan et al. et al. Lan et al2020. <https://arxiv.org>, 2020. See text for full citation.
- [107] Lee et al. et al. Lee et al2024 b l a s t. <https://arxiv.org>, 2024. See text for full citation.
- [108] Lei et al. et al. Lei et al2021. <https://arxiv.org>, 2021. See text for full citation.
- [109] Li et al. et al. Li et al2020 linear f s m. <https://arxiv.org>, 2020. See text for full citation.
- [110] Li et al. et al. Li et al2023 loft q. <https://arxiv.org>, 2023. See text for full citation.
- [111] Li et al. et al. Li et al2026. <https://arxiv.org>, 2026. See text for full citation.

- [112] Lieber et al. et al. Lieber et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [113] Liu et al. et al. Liu et al2023 deja vu. <https://arxiv.org>, 2023. See text for full citation.
- [114] Liu et al. et al. Liu et al2023 shortcuts. <https://arxiv.org>, 2023. See text for full citation.
- [115] Liu et al. et al. Liu et al2025 k a n. <https://arxiv.org>, 2025. See text for full citation.
- [116] Liu et al. et al. Liu et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [117] LiuSi et al. et al. Liu si et al2019. <https://arxiv.org>, 2019. See text for full citation.
- [118] McDanel et al. et al. Mc danel et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [119] Merrill et al. et al. Merrill et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [120] Mirzadeh et al. et al. Mirzadeh et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [121] Molina et al. et al. Molina et al2020. <https://arxiv.org>, 2020. See text for full citation.
- [122] Moosa et al. et al. Moosa et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [123] Muqeeth et al. et al. Muqeeth et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [124] Pappone et al. et al. Pappone et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [125] Perez et al. et al. Perez et al2019. <https://arxiv.org>, 2019. See text for full citation.
- [126] Poli et al. et al. Poli et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [127] Ramachandran et al. et al. Ramachandran et al2017. <https://arxiv.org>, 2017. See text for full citation.
- [128] Raposo et al. et al. Raposo et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [129] Saunshi et al. et al. Saunshi et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [130] Scarvellis et al. et al. Scarvellis et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [131] Schuster et al. et al. Schuster et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [132] Song et al. et al. Song et al2024 pro sparse. <https://arxiv.org>, 2024. See text for full citation.
- [133] Song et al. et al. Song et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [134] Srivastava et al. et al. Srivastava et al2015. <https://arxiv.org>, 2015. See text for full citation.
- [135] Szatkowski et al. et al. Szatkowski et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [136] Terzic et al. et al. Terzic et al2025. <https://arxiv.org>, 2025. See text for full citation.

- [137] Wang et al. et al. Wang et al2018 skip net. <https://arxiv.org>, 2018. See text for full citation.
- [138] Wang et al. et al. Wang et al2024 s v d l l m. <https://arxiv.org>, 2024. See text for full citation.
- [139] WangZhu et al. et al. Wang zhu et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [140] Wei et al. et al. Wei et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [141] Weiss et al. et al. Weiss et al2021. <https://arxiv.org>, 2021. See text for full citation.
- [142] Wen et al. et al. Wen et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [143] Wu et al. et al. Wu et al2018. <https://arxiv.org>, 2018. See text for full citation.
- [144] Wu et al. et al. Wu et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [145] Yang et al. et al. Yang et al2024 cyclic n n. <https://arxiv.org>, 2024. See text for full citation.
- [146] Yang et al. et al. Yang et al2024 k a t. <https://arxiv.org>, 2024. See text for full citation.
- [147] Yang et al. et al. Yang et al2024 memory3. <https://arxiv.org>, 2024. See text for full citation.
- [148] Yang et al. et al. Yang et al2024 mo e i2. <https://arxiv.org>, 2024. See text for full citation.
- [149] Yu et al. et al. Yu et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [150] Yuan et al. et al. Yuan et al2023 a s v d. <https://arxiv.org>, 2023. See text for full citation.
- [151] ZengZhao et al. et al. Zeng zhao et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [152] Zhang et al. et al. Zhang et al2022 mo a. <https://arxiv.org>, 2022. See text for full citation.
- [153] Zhang et al. et al. Zhang et al2022 mo efication. <https://arxiv.org>, 2022. See text for full citation.
- [154] Zhang et al. et al. Zhang et al2024 re l u2. <https://arxiv.org>, 2024. See text for full citation.
- [155] Zhang et al. et al. Zhang et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [156] Zhao et al. et al. Zhao et al2025 f r spec. <https://arxiv.org>, 2025. See text for full citation.
- [157] Zhong et al. et al. Zhong et al2024. <https://arxiv.org>, 2024. See text for full citation.
- [158] ZhouLepikhin et al. et al. Zhou lepikhin et al2022. <https://arxiv.org>, 2022. See text for full citation.
- [159] Zhu et al. et al. Zhu et al2025 ouro. <https://arxiv.org>, 2025. See text for full citation.

- [160] Zhuang et al. et al. Zhuang et al2026. <https://arxiv.org>, 2026. See text for full citation.
- [161] Zhuo et al. et al. Zhuo et al2025. <https://arxiv.org>, 2025. See text for full citation.
- [162] Fan et al. Fan2024 looped transformers. <https://arxiv.org>, 2024. See text for full citation.
- [163] Xuezhe Fan et al. Looped transformers for length generalization. <https://arxiv.org/abs/2409.15647>, 2024. Demonstrates adaptive loop step count in transformers for algorithmic reasoning; direct prior art for Idea 6.1.
- [164] Fang et al. Fang2023 r a f t. <https://arxiv.org>, 2023. See text for full citation.
- [165] Fang et al. Fang2024 mask l l m. <https://arxiv.org>, 2024. See text for full citation.
- [166] FangMaskLLM et al. Fang mask l l m2024. <https://arxiv.org>, 2024. See text for full citation.
- [167] FanLayerDrop et al. Fan layer drop2020. <https://arxiv.org>, 2020. See text for full citation.
- [168] FastdLLM et al. Fastd l l m2025. <https://arxiv.org>, 2025. See text for full citation.
- [169] FedusSwitch et al. Fedus switch2022. <https://arxiv.org>, 2022. See text for full citation.
- [170] FrankleCarbin et al. Frankle carbin2019. <https://arxiv.org>, 2019. See text for full citation.
- [171] Frantar et al. Frantar2023 sparse g p t. <https://arxiv.org>, 2023. See text for full citation.
- [172] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. <https://arxiv.org/abs/2210.17323>, 2022. ICLR 2023.
- [173] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefer, and Dan Alistarh. MARLIN: Mixed-precision AUTO-REgressive LINear layer kernels. <https://arxiv.org/abs/2408.11743>, 2024.
- [174] FrantarGPTQ et al. Frantar g p t q2023. <https://arxiv.org>, 2023. See text for full citation.
- [175] FrantarMARLIN et al. Frantar m a r l i n2024. <https://arxiv.org>, 2024. See text for full citation.
- [176] FrantarSparseGPT et al. Frantar sparse g p t2023. <https://arxiv.org>, 2023. See text for full citation.
- [177] FrantatAlistarh et al. Frantat alistarh2023. <https://arxiv.org>, 2023. See text for full citation.
- [178] FreeMoE et al. Free mo e2025. <https://arxiv.org>, 2025. See text for full citation.
- [179] FuMoA et al. Fu mo a2024. <https://arxiv.org>, 2024. See text for full citation.
- [180] GateSkip et al. Gate skip2024. <https://arxiv.org>, 2024. See text for full citation.
- [181] Geiping et al. Geiping2025 huginn. <https://arxiv.org>, 2025. See text for full citation.

- [182] Grave et al. Grave2017 adaptive softmax. <https://arxiv.org>, 2017. See text for full citation.
- [183] GraveAdaptiveSoftmax et al. Grave adaptive softmax2017. <https://arxiv.org>, 2017. See text for full citation.
- [184] Graves et al. Graves2016. <https://arxiv.org>, 2016. See text for full citation.
- [185] Alex Graves. Adaptive computation time for recurrent neural networks. <https://arxiv.org/abs/1603.08983>, 2016.
- [186] GravesACT et al. Graves a c t2016. <https://arxiv.org>, 2016. See text for full citation.
- [187] GravesDNC et al. Graves d n c2016. <https://arxiv.org>, 2016. See text for full citation.
- [188] GravesNTM et al. Graves n t m2014. <https://arxiv.org>, 2014. See text for full citation.
- [189] Gu et al. Gu2023 mamba. <https://arxiv.org>, 2023. See text for full citation.
- [190] GuDao et al. Gu dao2024. <https://arxiv.org>, 2024. See text for full citation.
- [191] GudMamba et al. Gud mamba2023. <https://arxiv.org>, 2023. See text for full citation.
- [192] Gulmez et al. Gulmez2026. <https://arxiv.org>, 2026. See text for full citation.
- [193] Han et al. Han2024 s l train. <https://arxiv.org>, 2024. See text for full citation.
- [194] HanLMInfinite et al. Han l m infinite2024. <https://arxiv.org>, 2024. See text for full citation.
- [195] He et al. He2016 res net. <https://arxiv.org>, 2016. See text for full citation.
- [196] HeMagicPIG et al. He magic p i g2024. <https://arxiv.org>, 2024. See text for full citation.
- [197] HeResNet et al. He res net2016. <https://arxiv.org>, 2016. See text for full citation.
- [198] HGRN et al. H g r n2024. <https://arxiv.org>, 2024. See text for full citation.
- [199] Hua et al. Hua2019 channel gating. <https://arxiv.org>, 2019. See text for full citation.
- [200] HuangDenseNet et al. Huang dense net2017. <https://arxiv.org>, 2017. See text for full citation.
- [201] HuLoRA et al. Hu lo r a2022. <https://arxiv.org>, 2022. See text for full citation.
- [202] HybridReview et al. Hybrid review2024. <https://arxiv.org>, 2024. See text for full citation.
- [203] IlinThanos et al. Ilin thanos2025. <https://arxiv.org>, 2025. See text for full citation.
- [204] Jawahar et al. Jawahar2022 auto mo e. <https://arxiv.org>, 2022. See text for full citation.
- [205] Jin et al. Jin2025 d top p. <https://arxiv.org>, 2025. See text for full citation.
- [206] Kim et al. Kim2024 squeeze l l m. <https://arxiv.org>, 2024. See text for full citation.

- [207] KimiAttentionResiduals et al. Kimi attention residuals2026. <https://arxiv.org>, 2026. See text for full citation.
- [208] KimSqueezeLLM et al. Kim squeeze l l m2023. <https://arxiv.org>, 2023. See text for full citation.
- [209] KitaevReformer et al. Kitaev reformer2020. <https://arxiv.org>, 2020. See text for full citation.
- [210] KwonPagedAttention et al. Kwon paged attention2023. <https://arxiv.org>, 2023. See text for full citation.
- [211] Laitenberger et al. Laitenberger2024 gate skip. <https://arxiv.org>, 2024. See text for full citation.
- [212] Lan et al. Lan2020 a l b e r t. <https://arxiv.org>, 2020. See text for full citation.
- [213] LanALBERT et al. Lan a l b e r t2019. <https://arxiv.org>, 2019. See text for full citation.
- [214] Lee et al. Lee2023 o w q. <https://arxiv.org>, 2023. See text for full citation.
- [215] Lee et al. Lee2024 b l a s t. <https://arxiv.org>, 2024. See text for full citation.
- [216] Lee et al. Lee2025 m x plus. <https://arxiv.org>, 2025. See text for full citation.
- [217] LeeBLAST et al. Lee b l a s t2024. <https://arxiv.org>, 2024. See text for full citation.
- [218] LeeMXplus et al. Lee m xplus2025. <https://arxiv.org>, 2025. See text for full citation.
- [219] LeeOWQ et al. Lee o w q2024. <https://arxiv.org>, 2024. See text for full citation.
- [220] LeviathanSpeculative et al. Leviathan speculative2023. <https://arxiv.org>, 2023. See text for full citation.
- [221] Li et al. Li2024 s c bench. <https://arxiv.org>, 2024. See text for full citation.
- [222] Yuhong Li, Yingbing Han, Bofei Yang, Beidi Vyas, Yun Gao, Azalia Mirhoseini, et al. SnapKV: LLM knows what you are looking for before generation. <https://arxiv.org/abs/2404.14469>, 2024.
- [223] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. <https://arxiv.org/abs/2401.15077>, 2024. ICML 2024.
- [224] LiBTM et al. Li b t m2022. <https://arxiv.org>, 2022. See text for full citation.
- [225] Lieber et al. Lieber2024 jamba. <https://arxiv.org>, 2024. See text for full citation.
- [226] LiLoftQ et al. Li loft q2024. <https://arxiv.org>, 2024. See text for full citation.
- [227] LiMixLLM et al. Li mix l l m2024. <https://arxiv.org>, 2024. See text for full citation.
- [228] Lin et al. Lin2017 f p n. <https://arxiv.org>, 2017. See text for full citation.

- [229] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. <https://arxiv.org/abs/2306.00978>, 2023. MLSys 2024.
- [230] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. QServe: W4a8kv4 quantization and system co-design for efficient LLM serving. <https://arxiv.org/abs/2405.04532>, 2024. MLSys 2024.
- [231] LinAWQ et al. Lin a w q2024. <https://arxiv.org>, 2024. See text for full citation.
- [232] LinQServe et al. Lin q serve2024. <https://arxiv.org>, 2024. See text for full citation.
- [233] LiSnapKV et al. Li snap k v2024. <https://arxiv.org>, 2024. See text for full citation.
- [234] Liu et al. Liu2023 scissorhands. <https://arxiv.org>, 2023. See text for full citation.
- [235] Liu et al. Liu2025 co l a. <https://arxiv.org>, 2025. See text for full citation.
- [236] LiuDejaVu et al. Liu deja vu2023. <https://arxiv.org>, 2023. See text for full citation.
- [237] LiuLLMQAT et al. Liu l l m q a t2023. <https://arxiv.org>, 2023. See text for full citation.
- [238] LiuMicroMix et al. Liu micro mix2025. <https://arxiv.org>, 2025. See text for full citation.
- [239] LiuScissorHands et al. Liu scissor hands2023. <https://arxiv.org>, 2023. See text for full citation.
- [240] LiuScissorHands et al. Liu scissor hands2023b. <https://arxiv.org>, 2023. See text for full citation.
- [241] LiuSpinQuant et al. Liu spin quant2024. <https://arxiv.org>, 2024. See text for full citation.
- [242] LiWang et al. Li wang2026. <https://arxiv.org>, 2026. See text for full citation.
- [243] LLM360 Team. K2: A foundation language model from LLM360. 2025. See <https://www.llm360.ai/>.
- [244] LoRAMoERef et al. Loramoeref. <https://arxiv.org>, 2024. See text for full citation.
- [245] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. <https://arxiv.org/abs/2310.16834>, 2024. ICML 2024 Oral.
- [246] LTE et al. L t e2024. <https://arxiv.org>, 2024. See text for full citation.
- [247] MEMoE et al. M e mo e2024. <https://arxiv.org>, 2024. See text for full citation.
- [248] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. ShortGPT: Layers in large language models are more redundant than you expect. <https://arxiv.org/abs/2403.03853>, 2024.
- [249] MenShortGPT et al. Men short g p t2024. <https://arxiv.org>, 2024. See text for full citation.

- [250] Merrill et al. Merrill2024 expressiveness. <https://arxiv.org>, 2024. See text for full citation.
- [251] Mikolov et al. Mikolov2011. <https://arxiv.org>, 2011. See text for full citation.
- [252] MirzadehReLU et al. Mirzadeh re l u2024. <https://arxiv.org>, 2024. See text for full citation.
- [253] MishraNMSparsity et al. Mishra n m sparsity2021. <https://arxiv.org>, 2021. See text for full citation.
- [254] MistralAI et al. Mistral a i2024. <https://arxiv.org>, 2024. See text for full citation.
- [255] MnihHinton et al. Mnih hinton2008. <https://arxiv.org>, 2008. See text for full citation.
- [256] MoEEdit et al. Mo e edit2026. <https://arxiv.org>, 2026. See text for full citation.
- [257] MoNE et al. Mo n e2025. <https://arxiv.org>, 2025. See text for full citation.
- [258] MoRAM et al. Mo r a m2025. <https://arxiv.org>, 2025. See text for full citation.
- [259] MorinBengio et al. Morin bengio2005. <https://arxiv.org>, 2005. See text for full citation.
- [260] MPSurvey et al. M p survey2025. <https://arxiv.org>, 2025. See text for full citation.
- [261] Nie et al. Nie2025 b d3 l m. <https://arxiv.org>, 2025. See text for full citation.
- [262] Shen Nie, Fengqi Zhu, Jiacheng You, Zhihao Ou, Chenyu An, Liwei Chen, Jiwen Qian, Bolin Wang, Bin Xie, Jincheng Luo, et al. LLaDA: Large language diffusion with masking. <https://arxiv.org/abs/2502.09992>, 2025.
- [263] NVIDIAADLR et al. N v i d i a a d l r2025. <https://arxiv.org>, 2025. See text for full citation.
- [264] Ong et al. Ong2024 route l l m. <https://arxiv.org>, 2024. See text for full citation.
- [265] OngRouteLLM et al. Ong route l l m2024. <https://arxiv.org>, 2024. See text for full citation.
- [266] Pagliardini et al. Pagliardini2024 dense former. <https://arxiv.org>, 2024. See text for full citation.
- [267] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aakanksha Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. <https://arxiv.org/abs/2311.18677>, 2023. ISCA 2024.
- [268] Bo Peng, Eric Alcaide, Quentin Anthony, et al. RWKV: Reinventing RNNs for the transformer era. <https://arxiv.org/abs/2305.13048>, 2023.
- [269] PengRWKV et al. Peng r w k v2023. <https://arxiv.org>, 2023. See text for full citation.
- [270] Perez et al. Perez2018 fi l m. <https://arxiv.org>, 2018. See text for full citation.
- [271] PerezFiLM et al. Perez fi l m2018. <https://arxiv.org>, 2018. See text for full citation.

- [272] Piekos et al. Piekos2025 mo s a. <https://arxiv.org>, 2025. See text for full citation.
- [273] PyramidKV et al. Pyramid k v2024. <https://arxiv.org>, 2024. See text for full citation.
- [274] Qwen et al. Qwen2025. <https://arxiv.org>, 2025. See text for full citation.
- [275] Qwen Team. Qwen3.5 and Qwen3.5-MoE technical report. <https://qwenlm.github.io/blog/qwen3.5/>, 2025. Alibaba Cloud. Models: Qwen3.5-27B (Hybrid DeltaNet) and Qwen3.5-397B-A17B (MoE + Gated DeltaNet).
- [276] RaposoMoD et al. Raposo mo d2024. <https://arxiv.org>, 2024. See text for full citation.
- [277] RazorAttention et al. Razor attention2024. <https://arxiv.org>, 2024. See text for full citation.
- [278] REAP et al. R e a p2025. <https://arxiv.org>, 2025. See text for full citation.
- [279] RouhaniMX et al. Rouhani m x2023. <https://arxiv.org>, 2023. See text for full citation.
- [280] RouterKD et al. Router k d2025. <https://arxiv.org>, 2025. See text for full citation.
- [281] RouterKD et al. Router k d2026. <https://arxiv.org>, 2026. See text for full citation.
- [282] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. <https://arxiv.org/abs/2406.07524>, 2024. NeurIPS 2024.
- [283] Saratchandran et al. Saratchandran2025 depth width. <https://arxiv.org>, 2025. See text for full citation.
- [284] Schuster et al. Schuster2022 c a l m. <https://arxiv.org>, 2022. See text for full citation.
- [285] ShaoOmniQuant et al. Shao omni quant2024. <https://arxiv.org>, 2024. See text for full citation.
- [286] Shazeer et al. Shazeer2017. <https://arxiv.org>, 2017. See text for full citation.
- [287] Shazeer et al. Shazeer2020 swi g l u. <https://arxiv.org>, 2020. See text for full citation.
- [288] ShazeerMoE et al. Shazeer mo e2017. <https://arxiv.org>, 2017. See text for full citation.
- [289] SliceMoE et al. Slice mo e2025. <https://arxiv.org>, 2025. See text for full citation.
- [290] SLTrain et al. S l train2024. <https://arxiv.org>, 2024. See text for full citation.
- [291] Song et al. Song2026 ada ponder. <https://arxiv.org>, 2026. See text for full citation.
- [292] SongPowerInfer et al. Song power infer2023. <https://arxiv.org>, 2023. See text for full citation.
- [293] Sukhbaatar et al. Sukhbaatar2024 expert. <https://arxiv.org>, 2024. See text for full citation.
- [294] SukhbaatarAdaptiveSpan et al. Sukhbaatar adaptive span2019. <https://arxiv.org>, 2019. See text for full citation.

- [295] Sun et al. Sun2024 wanda. <https://arxiv.org>, 2024. See text for full citation.
- [296] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. YOCO: You only cache once — decoder-decoder architectures for language models. <https://arxiv.org/abs/2405.05254>, 2024. Closest model-level prior art for Idea 6.2.
- [297] SunHyperConnections et al. Sun hyper connections2024. <https://arxiv.org>, 2024. See text for full citation.
- [298] SunRetNet et al. Sun ret net2023. <https://arxiv.org>, 2023. See text for full citation.
- [299] SunWanda et al. Sun wanda2024. <https://arxiv.org>, 2024. See text for full citation.
- [300] SVDSOftmax et al. S v d softmax2023. <https://arxiv.org>, 2023. See text for full citation.
- [301] Szatkowski et al. Szatkowski2024 d2 d mo e. <https://arxiv.org>, 2024. See text for full citation.
- [302] Tang et al. Tang2024 quest. <https://arxiv.org>, 2024. See text for full citation.
- [303] TangQuest et al. Tang quest2024. <https://arxiv.org>, 2024. See text for full citation.
- [304] Terzic et al. Terzic2025 p d s s m. <https://arxiv.org>, 2025. See text for full citation.
- [305] Terzic et al. Terzic2025 s d s s m. <https://arxiv.org>, 2025. See text for full citation.
- [306] TransMamba et al. Trans mamba2026. <https://arxiv.org>, 2026. See text for full citation.
- [307] TreeDiffusionLM et al. Tree diffusion l m2026. <https://arxiv.org>, 2026. See text for full citation.
- [308] Tseng et al. Tseng2024 qu i psharp. <https://arxiv.org>, 2024. See text for full citation.
- [309] TsengPVTuning et al. Tseng p v tuning2024. <https://arxiv.org>, 2024. See text for full citation.
- [310] ValipourDyLoRA et al. Valipour dy lo r a2022. <https://arxiv.org>, 2022. See text for full citation.
- [311] Vaswani et al. Vaswani2017. <https://arxiv.org>, 2017. See text for full citation.
- [312] Wang et al. Wang2024 l e mo e. <https://arxiv.org>, 2024. See text for full citation.
- [313] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. <https://arxiv.org/abs/2403.07378>, 2024.
- [314] WangHAQ et al. Wang h a q2019. <https://arxiv.org>, 2019. See text for full citation.
- [315] WangLi et al. Wang li2024 l e mo e. <https://arxiv.org>, 2024. See text for full citation.
- [316] WangNiepert et al. Wang niepert2019. <https://arxiv.org>, 2019. See text for full citation.

- [317] WangStructformer et al. Wang structformer2021. <https://arxiv.org>, 2021. See text for full citation.
- [318] WangSVDLLM et al. Wang s v d l l m2025. <https://arxiv.org>, 2025. See text for full citation.
- [319] Wei et al. Wei2024 block dense. <https://arxiv.org>, 2024. See text for full citation.
- [320] Wei et al. Wei2024 low rank. <https://arxiv.org>, 2024. See text for full citation.
- [321] Wei et al. Wei2025 we lore. <https://arxiv.org>, 2025. See text for full citation.
- [322] WeLore et al. We lore2024. <https://arxiv.org>, 2024. See text for full citation.
- [323] WhatGetsActivated et al. What gets activated2026. <https://arxiv.org>, 2026. See text for full citation.
- [324] WillardLouf et al. Willard louf2023. <https://arxiv.org>, 2023. See text for full citation.
- [325] WillardShin et al. Willard shin2023. <https://arxiv.org>, 2023. See text for full citation.
- [326] Williams et al. Williams2026 r l t t. <https://arxiv.org>, 2026. See text for full citation.
- [327] XiaoDuoAttention et al. Xiao duo attention2024. <https://arxiv.org>, 2024. See text for full citation.
- [328] XiaoStreamingLLM et al. Xiao streaming l l m2023. <https://arxiv.org>, 2023. See text for full citation.
- [329] Yang et al. Yang2024 delta net. <https://arxiv.org>, 2024. See text for full citation.
- [330] Yang et al. Yang2024 g l a. <https://arxiv.org>, 2024. See text for full citation.
- [331] YangGLA et al. Yang g l a2024. <https://arxiv.org>, 2024. See text for full citation.
- [332] YangLooped et al. Yang looped2023. <https://arxiv.org>, 2023. See text for full citation.
- [333] Yu et al. Yu2024 v a t p. <https://arxiv.org>, 2024. See text for full citation.
- [334] Yuan et al. Yuan2023 a s v d. <https://arxiv.org>, 2023. See text for full citation.
- [335] Yuan et al. Yuan2024 d s mo e. <https://arxiv.org>, 2024. See text for full citation.
- [336] Yuan et al. Yuan2025 n s a. <https://arxiv.org>, 2025. See text for full citation.
- [337] YuanASVD et al. Yuan a s v d2023. <https://arxiv.org>, 2023. See text for full citation.
- [338] YuanNSA et al. Yuan n s a2025. <https://arxiv.org>, 2025. See text for full citation.
- [339] ZaheerBigBird et al. Zaheer big bird2020. <https://arxiv.org>, 2020. See text for full citation.
- [340] Zandieh et al. Zandieh2026 turbo quant. <https://arxiv.org>, 2026. See text for full citation.
- [341] ZandiehTurboQuant et al. Zandieh turbo quant2026. <https://arxiv.org>, 2026. See text for full citation.

- [342] Zhiyao Zeng, Yixuan He, et al. AdaMoE: Token-adaptive routing with null experts for mixture-of-experts language models. <https://arxiv.org/abs/2408.00361>, 2024.
- [343] Zhang et al. Zhang2025 dyna spec. <https://arxiv.org>, 2025. See text for full citation.
- [344] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. <https://arxiv.org/abs/2306.14048>, 2023. NeurIPS 2023.
- [345] ZhangAdaLoRA et al. Zhang ada lo r a2023. <https://arxiv.org>, 2023. See text for full citation.
- [346] ZhangH2O2023 et al. Zhangh2o2023. <https://arxiv.org>, 2024. See text for full citation.
- [347] Zhao et al. Zhao2023 a t o m. <https://arxiv.org>, 2023. See text for full citation.
- [348] Zhao et al. Zhao2024 ga lore. <https://arxiv.org>, 2024. See text for full citation.
- [349] Zhao et al. Zhao2025 f r spec. <https://arxiv.org>, 2025. See text for full citation.
- [350] ZhaoATOM et al. Zhao a t o m2024. <https://arxiv.org>, 2024. See text for full citation.
- [351] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. <https://arxiv.org/abs/2401.09670>, 2024. OSDI 2024.
- [352] Zhuo et al. Zhuo2025 poly norm. <https://arxiv.org>, 2025. See text for full citation.

Table 43: All 39 ideas ranked by priority score. Ideas 6.1–6.5 are the new proposals from this synthesis phase. Priority formula: $(\text{InferenceImpact} \times \text{Feasibility}) / (\text{Risk} \times \text{QualityPenalty})$. Verdict: **P** = Pursue, **I** = Investigate, **D** = Deprioritize.

Rank	ID	Short Name	Impact	Feas.	Risk	Qual.	Score	Verdict
1	5.7	INT4 Block Quantization	3	10	2	3	5.00	P
2	1.3	Per-Layer Adaptive Expert Count	2	9	3	2	3.00	P
3	5.1	TurboQuant KV Cache	3	8	3	3	2.67	I
4	5.10	Block Dynamic Type Compression	4	6	3	3	2.67	I
5	2.2	Compressed Dense Layers	4	6	4	5	1.20	P
6	5.4	Linked Attention (KV Eviction)	2	7	3	3	1.56	I
7	5.9	Dynamic Per-Value Numeric Type	3	5	3	3	1.67	I
8	1.1	Learnable Per-Token Top-k	2	5	6	2	0.83	P
9	5.8	Block Sparse Weights	2	8	4	5	0.80	I
10	1.6	Learned Layer Type (NAS)	3	3	5	3	0.60	I
11	5.3	Grammar/FSM Structured Attention	4	5	5	3	0.53	I
12	1.2	Per-Token Adaptive Depth	2	4	5	3	0.27	I
13	5.5	Ragged Window Attention	2	5	3	5	0.27	I
14	4.7	Compressed Dictionary	1	8	2	2	0.25	I
15	5.2	LSTM-Gated Attention	2	5	4	5	0.20	I
16	3.2	Swappable Experts	1	7	3	2	0.12	I
17	1.4	Learned Dense/Sparse Layer	2	3	6	3	0.11	I
18	3.8	Trainable Activations	1	6	2	2	0.15	D
19	5.6	Double Attention	1	3	5	3	0.07	D
20	1.7	Dynamic Vocabulary	1	6	3	3	0.07	I
21	3.3	Dynamic Expert Router	1	5	4	2	0.06	I
22	3.4	Recursive Internal State	1	3	6	3	0.06	I
23	1.5	Learned Sparsity Type	2	3	7	5	0.06	I
24	3.7	Learnable State Machine	2	3	7	5	0.06	I
25	3.3	Dynamic Expert Router	1	5	4	2	0.06	I
26	4.2	Shared Weights + LoRA	1	5	5	5	0.04	I
27	3.5	Gated Internal DAG	2	2	7	5	0.01	I
28	3.6	Recursive Internal DAG	1	2	9	7	0.00	D
29	4.4	Skip List Layers	1	3	8	5	0.01	I
30	4.6	Mixture of Models	1	2	9	7	0.00	I
31	2.1	Hierarchical Frequency Dict.	1	9	1	1	9.00*	D
32	4.3	LoRA Everywhere	2	4	5	5	0.16	P
33	4.5	Learned Residual Flow Control	2	5	5	5	0.16	I
34	4.1	State Machine Core	1	3	7	5	0.01	I
35	3.1	Layer-Level MoE	2	2	8	5	0.01	I
New Proposals (Group 6)								
–	6.1	In-Architecture AR Loop	–	–	–	–	–	I
–	6.2	Prefill/Decode Arch. Split	–	–	–	–	–	P
–	6.3	Block-Diffusion AR Decoder	–	–	–	–	–	D
–	6.4	Combined 6.1+6.2+6.3	–	–	–	–	–	I
–	6.5	Pre-Attention Expert Router	–	–	–	–	–	I

*Idea 2.1 scores anomalously high (trivially safe, negligible benefit) due to formula artifacts; correctly lower-priority.